

Pi Developer Architecture Guide

A source-grounded path from first contribution to extension authoring

Pi maintainers

2026-07-12

Table of contents

1	Welcome to the Pi codebase	9
1.1	Prerequisites and development setup	9
1.1.1	Quick start	10
1.1.2	Editor setup	10
1.1.3	TypeScript configuration for extension authors	10
1.2	What Pi is	11
1.2.1	How the npm workspace works	11
1.2.2	Import rules at a glance	11
1.3	A mental model in one diagram	12
1.4	Reading order and outcomes	12
1.5	Source map used for this guide	13
2	Reading and changing Pi's TypeScript	14
2.1	How to use this chapter	14
2.2	What you should know first	14
2.3	The five questions to ask when reading Pi code	14
2.3.1	0. Set up the workspace and run TypeScript	15
2.4	Part 1: First contact with a TypeScript file	17
2.4.1	1. Read a <code>.ts</code> file in 60 seconds	17
2.4.2	2. Files, exports, and imports	19
2.4.3	3. Describe values: annotations, inference, and shapes	21
2.5	Part 2: Choose and protect a value's shape	25
2.5.1	4. Unions, narrowing, and exhaustive branches	25
2.5.2	5. Mutability, snapshots, and copies	30
2.6	Part 3: Functions, reusable types, and data boundaries	32
2.6.1	6. Functions, callbacks, and closures	32
2.6.2	7. Generics and the utility types worth recognizing	35
2.6.3	8. Untrusted data: <code>unknown</code> , <code>any</code> , and casts	38
2.6.4	9. Runtime schemas with <code>TypeBox</code>	41
2.7	Part 4: Async work and object lifecycle	45
2.7.1	10. Promises, async functions, and streams	45
2.7.2	11. Cancellation with <code>AbortSignal</code>	47
2.7.3	12. Classes, inheritance, and cleanup	49
2.8	Part 5: Apply the skills to Pi	52
2.8.1	13. Pi's runtime and developer workflow	52
2.8.2	14. Guided source tour: a custom tool	53
2.8.3	15. Guided source tour: an agent event	54
2.8.4	16. Guided source tour: state and lifecycle	54
2.8.5	17. Make a small safe change	55
2.9	Reference: use when you get stuck	56
2.9.1	R1. Common TypeScript errors	56

2.9.2	R2. Utility type quick reference	59
2.9.3	R3. Glossary	59
2.9.4	R4. Cumulative self-check	60
2.9.5	R5. Type-checking a standalone extension	62
2.10	Closing	62
3	Architecture: packages, ownership, and entry points	64
3.1	What you will learn first	64
3.2	Dependency direction	64
3.3	Repository-level conventions	65
3.4	Component walkthrough	65
3.4.1	packages/tui: terminal rendering primitives	65
3.4.2	packages/ai: a provider-neutral streaming contract	65
3.4.3	packages/agent: generic conversation and tool loop	66
3.4.4	packages/coding-agent: the Pi application	66
3.4.5	packages/orchestrator: experimental process orchestration	67
3.5	Constructing a running coding-agent session	67
3.6	Persistence and context are different things	68
3.7	Where to make a change	68
3.8	Beginner checkpoint	69
4	Prompt lifecycle: from Enter to agent settlement	70
4.1	A small vocabulary for the trace	70
4.2	The complete path	70
4.3	Stage 1: Terminal input enters the TUI	71
4.4	Stage 2: Commands, keybindings, and input interception	71
4.5	Stage 3: Agent-session orchestration constructs a turn	71
4.6	Stage 4: Context and system prompt cross three construction boundaries	72
4.7	Stage 5: Provider dispatch for the already-chosen model	73
4.8	Stage 6: Streaming response becomes agent events and TUI updates	73
4.8.1	Compact event reference	74
4.9	Stage 7: Tool calls are parsed, validated, gated, and executed	74
4.10	Stage 8: Results create the next agent action	75
4.11	Stage 9: Persistence and transcript updates settle the turn	75
4.12	Stage 10: Errors, cancellation, retries, compaction, and shutdown	75
4.13	Trace this yourself	76
5	Agent core: runs, events, queues, and tool batches	77
5.1	The smallest useful mental model	77
5.2	The three layers inside the package	77
5.3	Agent state and snapshots	78
5.4	Starting a prompt versus continuing a context	78
5.5	Active-run lifecycle and cancellation	79
5.6	Event stream as the observable protocol	79
5.7	One provider turn in detail	80
5.8	Tool argument preparation and validation	80
5.9	Sequential and parallel tool execution	81
5.10	Before and after tool hooks	81
5.11	Termination, steering, and follow-up	82

5.12	Next-turn refresh hook	82
5.13	Error semantics are intentionally normalized	82
5.14	Embedding the agent package	83
5.15	Agent-core debugging checklist	83
6	Built-in tools: filesystem, shell, and discovery	84
6.1	Start with the tool contract	84
6.2	Tool registry and default selection	84
6.3	Shared design principles	85
6.4	Output limits protect the next provider turn	85
6.5	<code>read</code> : text ranges and image attachments	85
6.5.1	Read operations are pluggable	86
6.6	<code>write</code> : create or replace a complete file	86
6.7	<code>edit</code> : exact replacement and queued mutation	87
6.8	<code>bash</code> : process execution with streamed tail output	87
6.8.1	Streaming output	87
6.9	<code>grep</code> , <code>find</code> , and <code>ls</code> : bounded repository discovery	88
6.10	Paths, <code>cwd</code> , and remote operations	88
6.11	User shell commands are separate from model <code>bash</code>	89
6.12	Tool rendering is a first-class interface	89
6.13	Tool lifecycle and extension interception	89
6.14	A safe custom file-mutation skeleton	90
6.15	Tool-author review questions	90
7	Extensions: change Pi without forking it	92
7.1	Learning path and first vocabulary	92
7.2	Choose the right customization mechanism	92
7.3	Discovery, trust, and loading	93
7.4	The lifecycle you can rely on	93
7.4.1	Event groups and common use	94
7.5	A complete first extension	94
7.6	What <code>ExtensionAPI</code> and context provide	96
7.7	Tools: make the contract for the AI model explicit	97
7.8	Understanding <code>TypeBox</code> schemas for tool parameters	97
7.8.1	Common schema patterns	97
7.8.2	How to think about schemas as a JavaScript developer	98
7.8.3	<code>StringEnum</code> vs <code>Type.Union</code> for string literals	98
7.9	Commands, UI, and rendering	98
7.10	State, sessions, and queues	99
7.11	Distribute an extension as a Pi package	99
7.11.1	Step-by-step: from zero to a shareable extension package	100
7.11.2	Package layout conventions	101
7.12	Debugging extensions	102
7.12.1	Common pitfalls	102
8	Sessions, branches, and durable context	104
8.1	Vocabulary before the file format	104
8.2	Why persistence is a tree	104
8.3	Files, headers, and locations	105

8.4	The entry vocabulary	106
8.5	Message shapes at the provider boundary	106
8.6	SessionManager as the persistence boundary	107
8.7	Building context from a selected leaf	107
8.8	Compaction is a controlled replacement of old context	108
8.8.1	Split turns and tool continuity	108
8.9	Branch navigation and branch summaries	109
8.10	Session replacement tears down the old runtime	109
8.11	Session names, labels, and human navigation	110
8.12	Extension persistence patterns	110
8.12.1	Stateful tool results	110
8.12.2	Private durable data	110
8.12.3	Persistent model context	110
8.13	Session debugging checklist	111
9	Resources, settings, and project trust	112
9.1	The four words to keep separate	112
9.2	The startup services boundary	112
9.3	Global and project settings	113
9.4	Trust is an execution policy, not a cosmetic prompt	113
9.5	Resource categories	114
9.6	Explicit resource paths and patterns	114
9.7	Skills and prompt templates on the input path	115
9.8	Context files and system-prompt construction	115
9.9	Settings that alter runtime behavior	116
9.9.1	Model and thinking	116
9.9.2	Compaction and branch summary	116
9.9.3	Retry and delivery	116
9.9.4	Shell, images, and terminal	116
9.10	Project configuration that an extension should respect	116
9.11	Packages are resource delivery, not a new execution mode	117
9.12	Themes and the TUI boundary	117
9.13	Reload semantics	118
9.14	Pre-commit security and reproducibility checklist	118
10	Models, providers, and authentication	119
10.1	Vocabulary and a concrete comparison	119
10.2	The <code>Model</code> contract	119
10.3	Model registry assembly	120
10.4	Selecting an initial model	120
10.5	Thinking levels are Pi vocabulary	120
10.6	Custom models in <code>models.json</code>	121
10.7	Provider overrides and model overrides	122
10.8	Config values and secrets	123
10.9	Credential precedence and storage	124
10.9.1	OAuth lifecycle	124
10.10	Provider headers and request layering	125
10.11	The unified streaming API	125
10.12	Payload and response hooks	125

10.13	Transport, timeout, and retry	126
10.14	Custom providers from an extension	126
10.15	Provider debugging workflow	126
11	Terminal UI and interactive mode	128
11.1	Vocabulary before rendering details	128
11.2	The TUI package owns terminal mechanics	128
11.3	Differential rendering in practical terms	129
11.4	Components are composition, not application policy	129
11.5	Terminal lifecycle and capabilities	129
11.6	Keyboard input is parsed before it becomes a binding	130
11.7	Editor submission and queue semantics	130
11.8	Transcript rendering from agent events	130
11.9	Tools in the transcript	131
11.10	Extension UI surface	131
11.11	Custom components and focus	132
11.12	Autocomplete composition	133
11.13	Themes and semantic styling	133
11.14	Images and rendered width	133
11.15	Debugging visual issues	134
12	Customization patterns and contributor workflow	135
12.1	Start with the decision, not the API	135
12.2	Configuration before code	135
12.3	Pattern: a safety gate around a built-in tool	136
12.4	Pattern: inject focused context, not a second system	136
12.5	Pattern: state that survives restart and branching	137
12.6	Pattern: dynamic tools and active-tool policy	138
12.7	Pattern: provider customization	138
12.8	Pattern: custom TUI that still works outside the TUI	139
12.9	Pattern: package a team customization	140
12.10	Common mistakes and the correct boundary	140
12.11	A practical verification ladder	140
13	Testing, debugging, and contribution workflow	142
13.1	Three kinds of evidence	142
13.2	Localize the failure first	142
13.3	Static checking versus tests	143
13.4	Test at the correct package	143
13.5	High-value test cases by subsystem	143
13.5.1	Extensions	143
13.5.2	Tools	144
13.5.3	Sessions and compaction	144
13.5.4	Provider/model registry	144
13.6	Interactive TUI smoke testing	144
13.7	/debug and diagnostic data	145
13.8	Error classification before retrying	145
13.9	Source navigation habits	145
13.10	Git hygiene in a shared worktree	146

13.11	Documentation as a tested interface	146
13.12	Pre-review checklist	146
14	Guided source tours and architecture exercises	147
14.1	How to do a source tour	147
14.2	Tour 1: What launches when <code>pi</code> starts?	147
14.3	Tour 2: Why can a session change <code>cwd</code> safely?	148
14.4	Tour 3: How do resources reach a system prompt?	148
14.5	Tour 4: What happens to a user message before a provider sees it?	149
14.6	Tour 5: Follow one streamed token to the terminal	149
14.7	Tour 6: Follow one tool call through validation	150
14.8	Tour 7: Study parallel tool execution without guessing	150
14.9	Tour 8: Explain a model request without opening a network trace	151
14.10	Tour 9: Trace a resume-context discrepancy	151
14.11	Tour 10: Add a narrowly scoped extension feature	151
14.12	A weekly reading routine for new contributors	152
15	Extension event and API reference	153
15.1	Use this chapter as an index	153
15.2	Choose an event by the representation you need	153
15.3	Startup and trust events	154
15.3.1	<code>project_trust</code>	154
15.3.2	<code>resources_discover</code>	155
15.4	Session events	155
15.4.1	<code>session_start</code> and <code>session_shutdown</code>	155
15.4.2	<code>session_before_switch</code> and <code>session_before_fork</code>	155
15.4.3	Compaction and tree hooks	155
15.4.4	Session information	157
15.5	Input and agent-run events	157
15.5.1	<code>input</code>	157
15.5.2	<code>before_agent_start</code>	157
15.5.3	<code>agent_start</code> , <code>agent_end</code> , and <code>agent_settled</code>	157
15.5.4	<code>turn_start</code> , <code>turn_end</code> , and messages	158
15.6	Provider events	158
15.6.1	<code>before_provider_headers</code>	158
15.6.2	<code>before_provider_request</code>	158
15.6.3	<code>after_provider_response</code>	158
15.7	Tool events and parallelism	158
15.8	Extension context reference	160
15.9	Command context reference	160
15.10	Registration API quick decisions	161
15.11	Model-facing output truncation	161
15.12	Safety rules for extension authors	162
16	Run modes, SDK embedding, and operational boundaries	163
16.1	Start with the host, not the flag	163
16.2	How Pi chooses a mode	163
16.3	Extension mode behavior	164
16.4	Print mode as a single-run host	164

16.5	JSON mode and machine-readable automation	165
16.5.1	Session header on the JSON stream	165
16.6	RPC mode as a persistent process boundary	166
16.6.1	Framing and object shapes	166
16.6.2	Lifecycle in overview	166
16.7	The <code>packages/ai</code> boundary	167
16.8	The public SDK entry point	167
16.9	Why SDK callers should not reimplement <code>packages/coding-agent/src/main.ts</code>	167
16.10	Custom tools supplied by an SDK host	168
16.11	Initial messages and attachments	168
16.12	Operational safety boundaries	168
16.13	Operational observability	169
16.13.1	Behavior provided by Pi	169
16.13.2	Host-level recommendations	169
16.14	Choosing an integration approach	169
16.15	Operational checklist	169
17	Failure and recovery reference	171
17.1	How to use the reference	171
17.2	Failure flow by runtime layer	171
17.3	Startup and configuration failures	172
17.4	Provider failures	172
17.5	Tool failures	173
17.6	Cancellation versus timeout	173
17.7	Recovery after compaction and tree movement	174
17.8	UI and renderer fallback	174
17.9	Incident checklist	174
18	Glossary	175

1 Welcome to the Pi codebase

This guide is for developers who are new to Pi's TypeScript codebase; you only need to be comfortable reading code. Its job is practical: by the end, you should be able to trace a prompt through Pi, find the right layer for a change, and build an extension without guessing at the API.

The book is deliberately staged. First learn enough TypeScript to read the examples, then learn the runtime path, then study the concrete tools and state that make Pi useful. Only after that do the extension, UI, integration, and reference chapters introduce the less common edges of the system. Each chapter opens with its vocabulary and a small mental model before it uses the source.

This is a guide to the repository as checked out, not a replacement for the published user documentation. File paths throughout are repository-relative. They are deliberately specific so that you can open the implementation beside this book. When a section simplifies a control flow, it says so; the named source files are the authority for edge cases.

1.1 Prerequisites and development setup

Before diving into the codebase, make sure your environment is ready.

Requirement	Minimum version	Why
Node.js	<code>>= 22.19.0</code>	Pi runs erasable TypeScript directly with Node's native type stripping and uses modern ESM features. Native stripping does not type-check or transform TypeScript syntax that needs JavaScript emit; <code>tsx</code> remains a development runner and <code>jiti</code> loads extensions. See the <code>engines</code> field in <code>package.json</code> .
npm	Bundled with Node.js	Workspace management and package installs. The repo uses <code>npm workspaces</code> .
git	Any recent version	To clone and navigate repository history.
TypeScript knowledge	Reading level	This guide teaches what you need in chapter 1 . You do not need to be fluent.

1.1.1 Quick start

```
git clone https://github.com/earendil-works/pi.git
cd pi
npm install --ignore-scripts # Install dependencies without running lifecycle scripts
npm run build                # Build all packages
npm run check                # Lint, format, and type check
./pi-test.sh                 # Run pi from source (works from any directory)
```

`npm install --ignore-scripts` is intentional: the project pins exact dependency versions and does not run install lifecycle scripts by default. Use `npm ci --ignore-scripts` for a clean CI-style install only when `package-lock.json` is present and matches `package.json`; it removes the existing installation and installs the lockfile exactly. The final command, `./pi-test.sh`, launches the repository's Pi implementation directly from source rather than a globally installed or published build.

If you only want to write extensions (not modify Pi itself), you do not need to clone this repository. Install pi globally and start writing `.ts` files under `~/.pi/agent/extensions/`. The extension chapter walks through each step after the guide has introduced the agent loop and tool contract.

1.1.2 Editor setup

Any editor with TypeScript support works. VS Code users get inline type hints, autocomplete for `ExtensionAPI` methods, and schema validation on tool parameters out of the box when TypeScript resolves the `@earendil-works/*` package types. If you are writing extensions outside the monorepo, install the published packages as dev dependencies to get editor support:

```
npm install --save-dev @earendil-works/pi-ai @earendil-works/pi-coding-agent
```

1.1.3 TypeScript configuration for extension authors

Extensions are loaded by Pi at runtime with `jiti`, so you do not need a `tsconfig.json` or build step for a single-file extension. If you write a multi-file extension or a Pi package, create a minimal `tsconfig.json` that resolves the Pi package types:

```
{
  "compilerOptions": {
    "target": "ESNext",
    "module": "ESNext",
    "moduleResolution": "bundler",
    "strict": true,
    "skipLibCheck": true
  }
}
```

Type-check your extension with `npx tsc --noEmit`.

1.2 What Pi is

Pi is a TypeScript monorepo for an agent harness and a self-extensible coding agent. The repository is an npm workspaces project: its root `package.json` declares the workspaces. The interactive `pi` command is implemented by `@earendil-works/pi-coding-agent`; it composes a terminal UI, an agent runtime, and a provider-neutral AI API.

1.2.1 How the npm workspace works

The root `package.json` declares five package workspaces through `"packages/*"`: `packages/tui`, `packages/ai`, `packages/agent`, `packages/coding-agent`, and `packages/orchestrator`. It also declares five explicit extension-example workspaces: `with-deps`, `custom-provider-anthropic`, `custom-provider-gitlab-duo`, `sandbox`, and `gondolin` under `packages/coding-agent/examples/extensions/`. Thus, `packages/*` is not the complete workspace list.

`npm install` at the repository root installs dependencies for every declared workspace and links local workspace packages for package-name imports such as `@earendil-works/pi-ai`. Those imports use each workspace's package metadata (the published entries point at `dist`); the root `tsconfig.json` separately maps package names to source entry points for whole-project type checking without compiling.

When you write an extension outside the monorepo, you import from the published npm packages (`@earendil-works/pi-coding-agent`, etc.). Pi bundles these at runtime, so you specify them as `peerDependencies` (not `dependencies`) to avoid bundling a duplicate copy.

1.2.2 Import rules at a glance

Context	Import style	Example
Inside a package, same package	Relative with <code>.ts</code>	<code>import { Agent } from "./agent.ts"</code>
Inside the repo, cross-package	Package name (no <code>.ts</code>)	<code>import { Type } from "@earendil-works/pi-ai"</code>
Extension outside the repo	Published package name	<code>import { defineTool } from "@earendil-works/pi-coding-agent"</code>

The four packages on the normal interactive prompt path are:

Package	Role	Start reading at
<code>packages/tui</code>	Reusable terminal UI primitives and differential rendering	<code>src/index.ts</code> , <code>src/tui.ts</code>
<code>packages/ai</code>	Models, provider dispatch, streaming API adapters, authentication primitives	<code>src/index.ts</code> , <code>src/types.ts</code> , <code>src/compat.ts</code>
<code>packages/agent</code>	Provider-independent agent loop, message state, tool execution	<code>src/agent.ts</code> , <code>src/agent-loop.ts</code> , <code>src/types.ts</code>

Package	Role	Start reading at
<code>packages/coding-agent</code>	CLI, interactive application, sessions, built-in tools, configuration, extensions	<code>src/main.ts</code> , <code>src/core/sdk.ts</code> , <code>src/core/agent-session.ts</code>

`packages/orchestrator` is an experimental package depending on the coding agent. It is useful when you specifically need its process orchestration surface, but it is not on the normal interactive prompt path described here.

1.3 A mental model in one diagram

```

terminal input
  |
  v
coding-agent interactive mode
  |
  v
AgentSession: commands, sessions, resources <---- extension hooks: input,
  |                                                    session, UI, requests
  v
agent-core: context, streaming, tools <----- extension hooks: tools,
  |                                                    turn lifecycle
  |
  |
  v
pi-ai: model dispatch and provider stream
  |
  v
provider API (OpenAI, Anthropic, Google, ...)

```

The vertical arrows show the usual prompt path. Extensions are loaded by the coding agent and hook into defined `AgentSession` and `agent-loop` boundaries—for example to inspect input, observe session events, add or wrap tools, customize UI, or alter requests. These hooks participate in the normal runtime; they do not bypass the agent loop.

1.4 Reading order and outcomes

Read the first six content chapters in order, after this welcome chapter. They form the beginner path from “I can read this file” to “I can make a small, safe extension”:

1. [TypeScript used by Pi](#) teaches the language patterns that appear in the rest of the book.
2. [Architecture: packages, ownership, and entry points](#) gives each package one clear job.
3. [Prompt lifecycle: from Enter to the next render](#) shows the whole journey without requiring source-level detail yet.

4. [Agent core: runs, events, queues, and tool batches](#) explains the reusable loop at the center of that journey.
5. [Built-in tools: filesystem and shell execution](#) turns the loop into concrete, inspectable operations.
6. [Extensions: change Pi without forking it](#) uses those concepts to build a first customization.

The next six content chapters explain the surrounding systems in the order a real feature usually encounters them: [sessions and durable context](#), [resources, settings, and trust](#), [models and authentication](#), [the terminal UI](#), [customization decisions](#), and [testing and debugging](#).

The final four content chapters are practice and reference material: [guided source tours](#), the [extension event reference](#), [run modes and SDK embedding](#), and the [failure and recovery reference](#). Use the [glossary](#) whenever a term becomes dense.

The [glossary](#) is a quick reference for terms that recur across the codebase.

1.5 Source map used for this guide

The guide was checked against the package manifests, the extension examples, the coding-agent extension documentation, and these implementation seams:

- CLI composition: `packages/coding-agent/src/main.ts` and `src/core/agent-session-services.ts`.
- Session construction and prompt preprocessing: `packages/coding-agent/src/core/sdk.ts` and `src/core/agent-session.ts`.
- Low-level loop and tool execution: `packages/agent/src/agent.ts`, `src/agent-loop.ts`, and `src/types.ts`.
- Provider dispatch: `packages/ai/src/compat.ts`, `packages/ai/src/models.ts`, and `packages/ai/src/types.ts`.
- Extension contract and loading: `packages/coding-agent/src/core/extensions/` plus `packages/coding-agent/docs/extensions.md`.

The repository evolves. If a symbol named in this book moves, use `rg "symbolName" packages` from the repository root, then update your mental model from the current implementation rather than preserving an old path.

2 Reading and changing Pi's TypeScript

Pi is written as native ECMAScript modules in TypeScript. This chapter is for a developer who knows basic JavaScript but has little or no TypeScript experience. Its goal is not to teach the entire language. Its goal is to help you open a Pi `.ts` file, understand the runtime values and type descriptions, trace the important branches, and make a small safe change.

The chapter follows one small status-reporting tool as it grows. Each step adds one idea that appears in Pi's source. After the guided path, the reference sections collect errors, utility types, and a glossary.

2.1 How to use this chapter

Read the first five parts in order. Every new TypeScript idea starts with a small JavaScript-shaped example, then adds the type information Pi uses. In this chapter, a **value** is something the running program can use, a **type** is a description checked before the program runs, and a **module** is one file that can import and export names. Those three words explain most of the language-specific syntax that follows.

2.2 What you should know first

You should be comfortable with:

- variables, strings, numbers, booleans, objects, and arrays
- `if`, `for`, `return`, and basic function calls
- the idea that a Promise represents a result that arrives later

You do **not** need prior TypeScript experience. You also do not need to learn every advanced TypeScript feature before contributing. Conditional-type metaprogramming, decorators, declaration merging, namespaces, and elaborate mapped types are outside this chapter's main path.

2.3 The five questions to ask when reading Pi code

When a TypeScript file looks dense, ask these questions in order:

1. **What is the runtime value?** Types, interfaces, and `import type` vanish after checking; objects, functions, arrays, and classes remain.
2. **What shape does the value have?** Look for an interface, type alias, schema, or generic that describes it.
3. **Which check selects the shape?** A `type` or `role` tag often narrows a union to one variant.

4. **Where did the value cross a boundary?** JSON, model output, config files, and network responses need runtime validation.
5. **Who owns the work?** Identify the callback, `AbortSignal`, array snapshot, class, or cleanup method responsible for the operation.

These questions are more useful than memorizing isolated syntax rules.

2.3.1 0. Set up the workspace and run TypeScript

Understanding an example is useful only if you can reproduce it. Pi's setup has three separate actions:

Action	Meaning	Typical command
Install	download the packages named by the repository	<code>npm install --ignore-scripts</code>
Run	execute JavaScript or erasable TypeScript	<code>node --experimental-strip-types file.ts</code> or the project launcher
Check	ask the type checker to find mistakes without running the program	<code>npx tsgo --noEmit</code>

The commands do different jobs. A file can run successfully while the type checker rejects it, and a type checker cannot help if the required dependencies are not installed.

2.3.1.1 Install the workspace dependencies

From the repository root, check the required Node version first:

```
node --version
```

Then install the workspace dependencies:

```
npm install --ignore-scripts
```

`--ignore-scripts` prevents package lifecycle scripts from running during this basic setup. For a clean checkout where the lockfile must be followed exactly, use:

```
npm ci --ignore-scripts
```

The repository contains several workspace packages. A package import such as `@earendil-works/pi-ai` can resolve to another package in this same checkout rather than to a separate copy from the network.

To add a dependency, distinguish code needed at runtime from a tool needed only during development:

```
npm install example-runtime-package
npm install --save-dev example-development-tool
```

Dependency changes update `package.json` and usually the lockfile. Review both files before sharing the change; installing a package is not a TypeScript operation.

2.3.1.2 Run a TypeScript file

A small erasable `.ts` file can be run by a supported Node version with type stripping:

```
node --experimental-strip-types /path/to/example.ts
```

Pi's source and extension workflows may instead use `tsx`, `jiti`, or the repository launcher. These tools make source execution convenient, but they do not replace type checking. Use the project's documented launcher when running the agent rather than inventing a new one-off command.

2.3.1.3 Check the code

A focused check asks `tsgo` to inspect the project without emitting JavaScript:

```
npx tsgo --noEmit
```

The repository-wide gate does more than type checking:

```
npm run check
```

The normal beginner loop is:

```
install dependencies → edit → run a focused example → type-check → run the gate
```

2.3.1.4 Setup troubleshooting

Symptom	First check
command not found: <code>tsgo</code>	dependencies are not installed; run the workspace install
Cannot find module	check the package install, import path, <code>.ts</code> suffix, and <code>tsconfig</code> mapping
file runs but the check fails	stripping/transpiling ran code but did not type-check it
a dependency works but changes the lockfile	inspect <code>package.json</code> and the lockfile before sharing

Checkpoint: Which command installs packages? Which runs a file? Which checks types? What can succeed while the others fail?

2.4 Part 1: First contact with a TypeScript file

2.4.1 1. Read a .ts file in 60 seconds

Vocabulary: a *value* is data available while the program runs. An *annotation* is a type written by the author. The *type checker* reads those annotations and reports inconsistent values before the program runs. The *runtime* is the JavaScript execution itself.

Start with ordinary JavaScript. Nothing in this function tells JavaScript what `user` should look like:

```
function greeting(user) {  
  return `Hello, ${user.name}`;  
}  
  
greeting({ name: "Ada" });
```

TypeScript adds a description of the value flow:

```
interface User {  
  name: string;  
}  
  
function greeting(user: User): string {  
  return `Hello, ${user.name}`;  
}
```

The `interface User` and the `: User` and `: string` annotations help the editor and type checker. They do not insert checks into the running function. After Node strips the annotations, the runtime operation is still property access and string interpolation.

Compare the visible TypeScript with what remains at runtime:

```
// Written and checked  
function greeting(user: User): string {  
  return `Hello, ${user.name}`;  
}  
  
// Runtime shape after type syntax is removed  
function greeting(user) {  
  return `Hello, ${user.name}`;  
}
```

A type checker can reject this call before execution:

```
// greeting({ name: 42 }); // number is not assignable to string
```

Compare three annotation choices:

```
// JavaScript: no declared contract
function show(value) { return value.name; }

// TypeScript: input contract, inferred return type
function showTyped(value: { name: string }) {
  return value.name; // string
}

// TypeScript: input and output contracts are both explicit
function showExplicit(value: { name: string }): string {
  return value.name;
}
```

The middle and final versions are checked the same way for callers. The final version makes the public return contract easier to see.

But a value arriving from JSON can still be wrong at runtime. We will return to that boundary in Section 8.

2.4.1.1 Small JavaScript features that appear everywhere

These are JavaScript features, not special TypeScript features:

```
const name = user.name;
const label = `User: ${name}`; // template string
const first = users[0]; // array indexing
const copy = [...users]; // shallow array copy
const { name: displayName } = user; // destructuring

function joinLabels(separator: string, ...labels: string[]): string {
  return labels.join(separator); // rest parameters collect remaining args
}
```

Optional chaining avoids a property access when the left side is missing:

```
const firstName = users[0]?.name;
```

Nullish coalescing supplies a fallback only for `null` or `undefined`:

```
const timeout = options.timeoutMs ?? 5_000;
```

That differs from `||`, which also treats `0`, `false`, and `""` as falsey:

```
const zero = 0 || 10; // 10 - often surprising
const zero2 = 0 ?? 10; // 0 - nullish coalescing preserves zero
```

These operators are used in Pi before any TypeScript-specific feature is involved.

2.4.1.2 What is type-only?

The checker understands these declarations, but the runtime does not receive them:

```
interface User { name: string }
type UserId = string;

function readUser(id: UserId): User {
  return { name: id };
}
```

The function and returned object remain. `User` and `UserId` do not become runtime objects. A type annotation is not a conversion and not validation.

2.4.1.3 Checkpoint

You should now be able to:

- identify which lines perform runtime work
- recognize annotations as descriptions rather than checks
- explain what `?.` and `??` do
- distinguish a shallow spread copy from a deep clone

If those are clear, continue. Every later section adds one layer to this same mental model.

2.4.2 2. Files, exports, and imports

A **module** is one file with an explicit set of names it exports and imports. **ESM** means ECMAScript modules, the JavaScript module system Pi uses. A *public surface* is the set of names other files are allowed to use.

A module is a file with an explicit public surface. Pi uses ECMAScript modules. Start with a named export:

```
// math.ts
export function add(left: number, right: number): number {
  return left + right;
}

export const version = 1;
```

Another file imports the names:

```
// main.ts
import { add, version } from "./math.ts";

console.log(add(2, 3), version);
```

Pi's source imports use the `.ts` suffix for relative monorepo imports:

```
import { Agent } from "./agent.ts";
import type { AgentEvent } from "./types.ts";
```

Cross-package imports use the package's public name instead:

```
import { Type } from "@earendil-works/pi-ai";
import type { ExtensionAPI } from "@earendil-works/pi-coding-agent";
```

`Type` is a runtime value used to build a schema. `ExtensionAPI` is only a type, so `import type` makes that intent explicit and removes the import from runtime output.

Compare the two kinds of imports:

```
import { Type } from "@earendil-works/pi-ai";           // needed at runtime
import type { ExtensionAPI } from "@earendil-works/pi-coding-agent"; // erased

function register(pi: ExtensionAPI): void {
  // TypeScript checks the parameter; no ExtensionAPI object is imported here.
  void pi;
}
```

2.4.2.1 Named and default exports

Both forms are valid JavaScript modules:

```
// Named
export function run(): void {}
import { run } from "./run.ts";

// Default
export default function run(): void {}
import run from "./run.ts";
```

Follow the style of the file you are editing. Pi packages generally favor explicit named exports; extension entry files may use a default export.

2.4.2.2 Imports you should not use

Do not deep-import another package's internal source file:

```
// Avoid
// import { Type } from "@earendil-works/pi-ai/src/internal/helper.ts";

// Prefer the public entry point
import { Type } from "@earendil-works/pi-ai";
```

Do not use a dynamic import merely to express an ordinary type dependency:

```
// Avoid
// type Agent = import("./agent.ts").Agent;

// Prefer a top-level type-only import
import type { Agent } from "./agent.ts";
```

A cycle involving only types can often be broken with `import type` or by moving shared types into a third file. A cycle involving runtime values deserves more investigation; it may indicate a weak module boundary.

2.4.2.3 Checkpoint

For a new import, ask:

- Is it relative or cross-package?
- Does a relative import have `.ts`?
- Is the imported name a runtime value or a type?
- Am I importing through the package's public surface?

Pi examples of these conventions appear in `packages/coding-agent/examples/extensions/` and throughout the package source.

2.4.3 3. Describe values: annotations, inference, and shapes

Vocabulary: a *shape* is the set of fields and field types an object has. A *literal type* is one exact value such as `"running"`. An *intersection* (`&`) combines requirements. *Widening* is TypeScript choosing a broader type such as `string` instead of the literal `"running"`.

You can annotate values explicitly or let TypeScript infer their types:

```
const count: number = 3;
const countInferred = 3; // number

function add(left: number, right: number): number {
  return left + right;
}

function addInferred(left: number, right: number) {
  return left + right; // return type inferred as number
}
```

Inference follows rules. A mutable string variable usually widens to `string`:

```
let role = "user";           // string
const fixedRole = "user";    // "user"
```

The narrow literal is useful when a value must belong to a closed set:

```
type Role = "user" | "assistant";
const role: Role = fixedRole;
// const badRole: Role = role; // role may be either allowed literal
```

2.4.3.1 Interfaces and type aliases

Both can describe an object shape:

```
interface RunOptions {
  timeoutMs?: number;
  signal?: AbortSignal;
}

type RunOptionsAlias = {
  timeoutMs?: number;
  signal?: AbortSignal;
};
```

The practical convention in Pi is:

- use **interface** for public object APIs that may be extended
- use **type** for unions, intersections, and utility-based aliases

A type alias also names a union naturally:

```
type RunState = "idle" | "streaming" | "error";
```

Interfaces extend object shapes with **extends**; type aliases compose with **&**:

```
interface EventBase { timestamp: number }
interface ClickEvent extends EventBase { x: number; y: number }

type Label = { label: string };
type LabeledClick = ClickEvent & Label;
```

These are type relationships. They do not create runtime inheritance.

2.4.3.2 Optional, required, and undefined

These declarations express different contracts:

```
interface MaybeAbsent {
  timeoutMs?: number;           // key may be absent
}

interface PresentButUndefined {
  timeoutMs: number | undefined; // key must exist
}

interface RequiredNumber {
  timeoutMs: number;           // key and number are required
}
```

Reading an optional value requires a fallback or a check:

```
function effectiveTimeout(options: MaybeAbsent): number {
  return options.timeoutMs ?? 5_000;
}
```

2.4.3.3 Readonly and literal contracts

`readonly` prevents assignment through that type. It is not a runtime freeze:

```
interface Point {
  readonly x: number;
  readonly y: number;
}

const point: Point = { x: 1, y: 2 };
// point.x = 3; // error
```

as `const` preserves literal values and makes the object deeply `readonly` at the type level:

```
const wide = { mode: "fast", retries: 1 };
// wide.mode is string

const precise = { mode: "fast", retries: 1 } as const;
// precise.mode is "fast"; precise.retries is 1
```

The `satisfies` operator checks a value against a type without replacing the value's inferred type:

```
type ModeConfig = { mode: "fast" | "careful"; retries: number };

const config = {
  mode: "fast",
  retries: 2,
} satisfies ModeConfig;
```

Use `satisfies` when you want validation of an object literal while retaining useful literal information for later code.

Compare the three tools on one value:

```
type Settings = { mode: "fast" | "careful"; retries: number };

const annotated: Settings = { mode: "fast", retries: 2 };
// The variable is viewed as Settings; mode is the union, not necessarily "fast".

const asserted = { mode: "typo", retries: 2 } as unknown as Settings;
// The assertion can silence the checker; it is not a validation strategy.

const checked = { mode: "fast", retries: 2 } satisfies Settings;
// The object is checked, while its useful inferred details are retained.
```

`as` is a promise to the checker. `satisfies` asks the checker to verify the value. Neither one validates data that arrived from JSON; runtime validation is covered in Section 8.

2.4.3.4 Structural typing and excess properties

TypeScript compares object shapes rather than requiring a named relationship:

```
interface Named { name: string }

const person = { name: "Ada", age: 36 };
const named: Named = person; // okay; person has at least name
```

Fresh object literals receive an additional excess-property check:

```
// const bad: Named = { name: "Ada", age: 36 };
// error: age does not exist in type Named
```

This catches misspelled option names, but it does not mean TypeScript rejects every extra property in every situation.

2.4.3.5 The running example

The chapter's status tool starts with a closed status type:

```
type Status = "idle" | "running" | "failed";

interface StatusReport {
  status: Status;
  message: string;
  updatedAt: number;
```

```

}

function formatReport(report: StatusReport): string {
    return `${report.status}: ${report.message}`;
}

```

The type checker now catches misspelled statuses and missing report fields. It still cannot validate a report loaded from JSON; that is a later boundary.

Build the report in small steps:

```

const status: Status = "running";
const message: string = "working";
const updatedAt: number = Date.now();
const report: StatusReport = { status, message, updatedAt };

```

A typo in `status` is caught while constructing the object. A parsed JSON object requires a runtime check even if you write `as StatusReport` afterward.

This is the chapter's running example. Later sections will turn the report into an event, store a short history, emit updates through a callback, parse it from untrusted input, validate it with `TypeBox`, and make the work cancellable.

2.4.3.6 Checkpoint

You should be able to:

- distinguish inference from an explicit annotation
- choose `interface` or `type` for a new declaration
- explain `?` versus `| undefined`
- explain the difference between `readonly`, `as const`, and `satisfies`
- read the `StatusReport` example without treating types as runtime objects

2.5 Part 2: Choose and protect a value's shape

2.5.1 4. Unions, narrowing, and exhaustive branches

Vocabulary: a *union* says a value may be one of several alternatives. *Narrowing* is evidence that removes alternatives. A *discriminant* (or tag) is a shared literal field such as `type`. A *type predicate* is a function result such as `value is User` that tells the checker what a successful test proved.

A union means a value is one of several alternatives. Narrowing is the evidence that tells TypeScript which alternative is active.

2.5.1.1 Primitive narrowing

```
type StringOrNumber = string | number;

function double(value: StringOrNumber): string | number {
  if (typeof value === "string") {
    return value + value;
  }
  return value * 2;
}
```

The `typeof` check changes the type inside each branch. `instanceof` performs a similar job for class instances:

```
function errorMessage(value: Error | string): string {
  if (value instanceof Error) return value.message;
  return value;
}
```

Other narrowing tools include equality, `in`, truthiness, and reusable type predicates. Use the narrowest check that reflects the actual contract.

Choose the check based on what you actually know:

Check	What it proves	Typical use
<code>typeof value === "string"</code>	primitive kind	string/number unions
<code>value instanceof Error</code>	class instance	caught errors
<code>value.type === "tool"</code>	tagged variant	Pi events/messages
<code>"name" in value</code>	property exists	foreign or ad-hoc objects
<code>value is User</code>	reusable guard contract	shared validation helper

2.5.1.2 Discriminated unions

A discriminated union gives every variant a common tag with a different literal:

```
type Content =
  | { type: "text"; text: string }
  | { type: "image"; mediaType: string; data: string };

function describe(content: Content): string {
  switch (content.type) {
    case "text":
      return content.text;
    case "image":
      return `image: ${content.mediaType}`;
  }
}
```

The checker knows that `text` exists in the first branch and `mediaType` exists in the second. Outside a branch, only fields common to all variants are safe.

Compare a real union with an optional-field bag:

```
// Weak: the tag and fields do not describe a reliable shape
type WeakContent = {
  type: string;
  text?: string;
  mediaType?: string;
  data?: string;
};

// Strong: each tag selects a complete variant
type StrongContent =
  | { type: "text"; text: string }
  | { type: "image"; mediaType: string; data: string };
```

Prefer the strong form when you control the data model.

2.5.1.3 Exhaustiveness with `never`

A closed union can require every case to be handled:

```
function describeAll(content: Content): string {
  switch (content.type) {
    case "text":
      return content.text;
    case "image":
      return content.mediaType;
    default: {
      const exhaustive: never = content;
      return exhaustive;
    }
  }
}
```

If someone adds an `audio` variant without adding a case, `content` is no longer `never` and the checker reports the missing branch. Without the check, incomplete logic may compile silently.

In plain language, `never` means “there is no possible value left here.” The `default` branch is expected to be unreachable. The assignment makes that expectation executable for the type checker.

2.5.1.4 Type predicates

A boolean tells the caller only that a function returned true. A type predicate also tells TypeScript what was proven:

```

function isTextBad(value: Content): boolean {
  return value.type === "text";
}

function isText(value: Content): value is { type: "text"; text: string } {
  return value.type === "text";
}

function uppercase(value: Content): string {
  if (isText(value)) return value.text.toUpperCase();
  return value.mediaType;
}

```

Use a predicate for a reusable guard. Use a direct `if` or `switch` for a one-off branch.

2.5.1.5 Nested unions

Pi messages often contain one union inside another:

```

type Message =
  | { role: "user"; content: string }
  | { role: "assistant"; content: Content[] };

function preview(message: Message): string {
  if (message.role === "user") {
    return message.content.slice(0, 40);
  }

  return message.content
    .map((block) => {
      switch (block.type) {
        case "text": return block.text;
        case "image": return `[image ${block.mediaType}]`;
      }
    })
    .join(" ");
}

```

Read from the outside in: narrow `role`, then narrow each content block.

2.5.1.6 Literal preservation

A widened `string` is not a discriminant:

```
const wideEvent = { type: "ping" }; // type is string
// const event: { type: "ping" } = wideEvent; // error

const narrowEvent = { type: "ping" } as const;
const event: { type: "ping" } = narrowEvent; // okay
```

The `satisfies` operator is another option when you want to check an object while retaining useful inferred details.

2.5.1.7 Pi source map

The same pattern appears in:

- AgentEvent in `packages/agent/src/types.ts`
- event switches in `packages/agent/src/agent-loop.ts`
- assistant content blocks in `packages/ai/src/types.ts`
- session entries in `packages/coding-agent/src/core/session-manager.ts`
- provider status handling in `packages/ai/src/api/openai-responses-shared.ts`

When opening one of these files, find the union definition first. List its tag values before reading the switch.

2.5.1.8 The running example: report events

The status tool can now notify a consumer about lifecycle changes:

```
type StatusEvent =
  | { type: "started"; report: StatusReport }
  | { type: "updated"; report: StatusReport }
  | { type: "failed"; message: string };

function eventLabel(event: StatusEvent): string {
  switch (event.type) {
    case "started":
    case "updated":
      return formatReport(event.report);
    case "failed":
      return `failed: ${event.message}`;
  }
}
```

The tag chooses the fields available to each branch. The same reading method applies to Pi's AgentEvent union.

2.5.1.9 Checkpoint

Write a union with two variants and answer:

1. Which check selects each variant?
2. Which fields are safe only after narrowing?
3. What happens if a third variant is added?
4. Would a predicate or a direct branch be clearer here?

Answers: the tag or primitive check selects the branch; variant-specific fields are unsafe before that check; an exhaustive switch reports the new variant; a predicate is useful when the same test is reused, while a direct branch is simpler once.

2.5.2 5. Mutability, snapshots, and copies

Vocabulary: an *object reference* is a value pointing to an object. An *alias* is a second reference to the same object. A *shallow copy* creates a new outer container but keeps nested references. A *deep copy* creates independent nested data. A *snapshot* is a copy retained to represent an earlier state.

TypeScript does not change JavaScript reference behavior. A spread creates a new outer object but does not clone nested objects:

```
type State = { level: string; meta: { count: number } };

const previous: State = { level: "low", meta: { count: 1 } };
const next = { ...previous, level: "high" };

next.meta.count = 99;
// previous.meta.count is also 99 - the nested object is shared
console.log(next === previous);           // false: new outer object
console.log(next.meta === previous.meta); // true: shared nested object
```

Arrays behave similarly:

```
type Message = { text: string };
const messages: Message[] = [{ text: "a" }, { text: "b" }];
const snapshot = messages.slice();

messages.push({ text: "c" }); // snapshot length remains 2
snapshot[0].text = "changed"; // messages[0].text also changes
console.log(snapshot === messages); // false: new array
console.log(snapshot[0] === messages[0]); // true: shared element
```

Choose the operation for the ownership you need:

Operation	New container	New nested objects	Typical use
<code>{ ...object }</code>	yes	no	shallow object update

Operation	New container	New nested objects	Typical use
<code>array.slice()</code> / <code>[...array]</code>	yes	no	snapshot order/length
<code>array.map(...)</code>	yes	depends on mapper	transform
<code>structuredClone(value)</code>	yes	yes for cloneable data	deep copy

`structuredClone` cannot clone functions, DOM nodes, or every class instance. Do not deep-clone large trees merely because a shallow snapshot is unfamiliar.

A `readonly` parameter protects the container through that reference:

```
function publish(items: readonly Message[]): number {
  // items.push({ text: "x" }); // error
  return items.length;
}

const live: Message[] = [{ text: "a" }];
publish(live); // mutable arrays can be viewed as readonly
```

It does not deeply freeze the message objects. `as const` is stronger for a literal value, but neither changes another alias that holds the same object.

Prefer explicit new containers for state transitions:

```
type RunState = {
  status: "idle" | "running";
  messages: readonly Message[];
};

function appendMessage(state: RunState, message: Message): RunState {
  return {
    ...state,
    status: "running",
    messages: [...state.messages, message],
  };
}
```

Pi snapshots message and tool lists around agent runs. The event contract still determines whether payload objects may be mutated.

For the running example, a history can expose a `readonly` view while updates create a new array:

```
function addEvent(
  history: readonly StatusEvent[],
  event: StatusEvent,
): readonly StatusEvent[] {
  return [...history, event];
}
```

The caller can retain the old history safely; the event objects themselves are still only as immutable as their declared fields.

2.5.2.1 Checkpoint

Predict the result of each operation before running it:

- nested property after object spread
- array length after `.slice()` and a later `push`
- element mutation through a readonly array
- whether `structuredClone` is appropriate for a class instance

Answers: a nested property remains shared after spread; a sliced array keeps its old length; `readonly` blocks array operations through that reference but does not deeply freeze elements; `structuredClone` is not a general solution for class instances or functions.

2.6 Part 3: Functions, reusable types, and data boundaries

2.6.1 6. Functions, callbacks, and closures

Vocabulary: a *function type* describes the parameter and return types of a function. A *callback* is a function passed to another function to be called now or later. A *closure* is a function that remembers variables from its defining scope. `this` is the receiver selected when an ordinary method is called.

Functions are values. They can be stored, passed to another function, or returned from a factory. This is the basis of Pi extension handlers and tool execution.

2.6.1.1 Function types

```
type Callback = () => void;
type Mapper = (value: number) => string;

const callback: Callback = () => console.log("ready");
const mapper: Mapper = (value) => String(value);
```

Compare a direct call, an immediate callback, and a stored callback:

```
function announce(): void { console.log("ready") }

announce(); // direct call: runs now

function callNow(handler: Callback): void {
  handler(); // callback runs during this function
}

function storeForLater(handler: Callback): () => void {
  return () => handler(); // callback runs when the returned function is called
}
```

A callback is not called when it is registered:

```
let onReady: Callback | undefined;

function register(handler: Callback): void {
  onReady = handler; // stores it
}

function fire(): void {
  onReady?.(); // calls it later
}

register(() => console.log("go"));
fire();
```

The common mistake is `register(handler())`, which calls the function now and passes its result instead of passing the function.

The running status tool uses the same relationship:

```
type StatusListener = (event: StatusEvent) => void;

function createReporter(listener: StatusListener): (event: StatusEvent) => void {
  return (event) => listener(event);
}

const report = createReporter((event) => console.log(eventLabel(event)));
// report({ type: "failed", message: "timeout" }); // invoked later
```

2.6.1.2 Closures

A closure remembers variables from the scope where it was created:

```
function makeCounter(): () => number {
  let count = 0;
  return () => {
    count += 1;
    return count;
  };
}

const next = makeCounter();
next(); // 1
next(); // 2
```

Closures are useful for small private state. A long-lived registry can also keep large objects alive, so unsubscribe or dispose when the API provides it.

The same feature can retain more than intended:

```
function registerLargeObject(registry: Array<() => void>, object: object): void {
    registry.push(() => console.log(object)); // closure retains object
}
```

Capture only what the callback needs when a registry may live for a long time.

2.6.1.3 Arrow functions and this

JavaScript resolves `this` at call time for ordinary methods:

```
class Service {
    private label = "service";

    dumpMethod(): string {
        return this.label;
    }

    dumpArrow = (): string => this.label;
}

const service = new Service();
const method = service.dumpMethod;
// method(); // detached method loses the instance receiver

const arrow = service.dumpArrow;
arrow(); // still works; arrow captured the instance
```

Arrow fields capture the surrounding `this`; methods do not. Use methods for normal class APIs and arrow fields when a handler will be detached and must keep its instance context.

2.6.1.4 Parameters and callback context

Optional and default parameters both allow omission, but defaults add a runtime fallback:

```
function connect(host: string, port?: number): string {
    return `${host}:${port ?? 443}`;
}

function connectWithDefault(host: string, port = 443): string {
    return `${host}:${port}`;
}
```

Extension APIs may require a full callback signature even when one parameter is unused. Name intentionally unused parameters with `_`:

```
// Simplified local types for the shape of a Pi-style callback
interface SessionStartEvent { type: "session_start" }
interface SessionContext { log(message: string): void }

// Illustrative registration API
function onSessionStart(
  handler: (event: SessionStartEvent, context: SessionContext) => void,
): void {}

onSessionStart((_event, context) => {
  context.log("started");
});
```

2.6.1.5 Checkpoint

- Type a callback that accepts a string and returns `void`.
- Show the difference between registering and calling a handler.
- Explain why a detached ordinary method can lose `this`.
- Identify what a closure keeps alive.

Answers: use `(value: string) => void`; registration passes the function without parentheses; a detached method no longer has its original receiver; a closure retains every outer variable it references.

2.6.2 7. Generics and the utility types worth recognizing

Vocabulary: a *generic* is a type parameter chosen when code is used. `T` usually means “some caller-selected type.” A *constraint* limits that choice. `keyof T` means the union of keys on `T`; `T[K]` means the value type at key `K`. A *utility type* transforms an existing type into a related shape.

Generics preserve a relationship between inputs and outputs. Without them, a helper either duplicates implementations or falls back to `any`:

```
function firstAny(items: any[]): any {
  return items[0];
}

function first<T>(items: readonly T[]): T | undefined {
  return items[0];
}

const text = first(["a", "b"]); // string | undefined
const number = first([1, 2]); // number | undefined
```

Read `T` as “some type selected by the caller.” It is not a runtime value:

```
function identity<T>(value: T): T {
    return value;
}

const message = identity("hello"); // T inferred as string
const count = identity(3);         // T inferred as number
```

2.6.2.1 Constraints

A constraint gives the implementation a safe property to use:

```
function nameOf<T extends { name: string }>(value: T): string {
    return value.name;
}

nameOf({ name: "Ada", age: 36 });
// nameOf({ age: 36 }); // error: name is missing
```

`keyof` creates a union of an object's keys. Indexed access returns the value type for a selected key:

```
function getField<T, K extends keyof T>(object: T, key: K): T[K] {
    return object[key];
}

const user = { name: "Ada", age: 36 };
// keyof typeof user is "name" | "age"
const name = getField(user, "name"); // T is user; K is "name"; result is string
// getField(user, "email"); // error: "email" is not a key of user
```

Generic object types are common for results:

```
interface Result<T> {
    ok: boolean;
    data: T;
}

const users: Result<string[]> = { ok: true, data: ["Ada"] };
```

`readonly T[]` accepts both mutable and readonly arrays when the helper only reads them. It communicates ownership without a runtime freeze.

2.6.2.2 Utility types

Recognize these when reading Pi code:

```

interface Retry {
  maxAttempts: number;
  backoffMs: number;
}

type RetryPatch = Partial<Retry>;          // both properties optional
type RetryRequired = Required<RetryPatch>;
type RetryReadonly = Readonly<Retry>;     // shallow top-level readonly

type RetryIdentity = Pick<Retry, "maxAttempts">;
type RetryWithoutBackoff = Omit<Retry, "backoffMs">;
type Labels = Record<"idle" | "running", string>;

```

See the transformations directly:

```

interface Retry { maxAttempts: number; backoffMs: number }

type Patch = Partial<Retry>;              // both keys optional
type Identity = Pick<Retry, "maxAttempts">; // keep one key
type WithoutBackoff = Omit<Retry, "backoffMs">;
type RequiredRetry = Required<Patch>;     // both keys required again

```

Other useful readers:

```

function connect(host: string, port: number): boolean { return true; }
type Args = Parameters<typeof connect>; // [string, number]
type Output = ReturnType<typeof connect>; // boolean

type MaybeName = string | null | undefined;
type Name = NonNullable<MaybeName>;      // string

```

With `Record<string, boolean>`, any string key is accepted. With `Record<"idle" | "running", string>`, every key in the closed union is required. Prefer an explicit type when a utility expression hides the actual contract.

2.6.2.3 Pi bridge: schema inference

Pi's `defineTool` uses a generic schema parameter to connect the runtime `parameters` value to the static `params` argument. You do not need to reproduce that type machinery. When reading it, ask which relationship the generic is preserving.

The status event helper can use the same relationship without knowing a particular event type:

```

function send<E>(event: E, listener: (event: E) => void): void {
  listener(event);
}

```

```

send<StatusEvent>(
  { type: "failed", message: "timeout" },
  (event) => console.log(eventLabel(event)),
);

```

E is not a runtime constructor. It preserves the type relationship between what `send` receives and what its listener receives.

2.6.2.4 Checkpoint

- Explain what type `first(["a"])` returns and why it includes `undefined`.
- Read `K extends keyof T` in plain English.
- Choose between `Pick`, `Omit`, and an explicit interface.
- State what a generic cannot do at runtime.

Answers: the result is `string | undefined` because the array may be empty; `K` must be one of `T`'s keys and `T[K]` is that key's value type; use `Pick` or `Omit` when the relationship should follow a source type, otherwise write an explicit interface; generics are erased and cannot inspect a runtime value.

2.6.3 8. Untrusted data: `unknown`, `any`, and casts

Vocabulary: a *boundary* is where data enters from outside the code that constructed it, such as JSON or a network response. *Untrusted* means the checker has no proof that the value has the expected shape. A *type assertion* (`as`) changes the checker's belief; it does not convert or validate the value.

Types describe checked code. They do not validate a value that arrived through a string, network response, config file, model response, or extension boundary. Use `unknown` until you have evidence:

```

function processAny(value: any): string {
  return value.name.toUpperCase(); // checker is disabled
}

processAny({ name: 42 }); // runtime failure
processAny(null);        // runtime failure

```

`any` can silently flow into typed variables:

```

const raw: any = JSON.parse("null");
const user: { name: string } = raw; // compiles, but runtime value is null

```

`unknown` forces a check:

```

function processUnknown(value: unknown): string {
  if (!value || typeof value !== "object") return "unknown";
  const name = (value as { name?: unknown }).name;
  return typeof name === "string" ? name.toUpperCase() : "unknown";
}

```

The `as` assertion did not validate `name`; the later `typeof` check did.

2.6.3.1 Four treatments of JSON

```
interface User {
  name: string;
  age: number;
}

const text = '{"name":"Ada","age":"unknown"}';
const raw: unknown = JSON.parse(text);

// Unsafe: changes only the checker's belief
const unsafe = raw as User;

// Partial: checks one field but misses age
if (raw && typeof raw === "object" && "name" in raw) {
  const partial = raw as { name: string };
}

// Manual guard: returns a value built from checked fields
function parseUser(value: unknown): User | undefined {
  if (!value || typeof value !== "object") return undefined;
  const candidate = value as { name?: unknown; age?: unknown };
  if (typeof candidate.name !== "string") return undefined;
  if (typeof candidate.age !== "number") return undefined;
  return { name: candidate.name, age: candidate.age };
}
```

A type predicate can make a reusable guard narrow its argument:

```
function isUser(value: unknown): value is User {
  return parseUser(value) !== undefined;
}
```

For a caught error, use the same discipline:

```
function messageOf(value: unknown): string {
  return value instanceof Error ? value.message : String(value);
}

function riskyOperation(): void {
  throw new Error("example failure");
}

try {
```

```

    riskyOperation();
} catch (error) {
    console.error(messageOf(error));
}

```

2.6.3.2 Boundary checklist

Boundary	Start with	Then
JSON.parse	unknown	guard or schema
config file	parsed unknown	validate, default, migrate
provider HTTP JSON	unknown	adapter/schema
model/tool arguments	external data	validate before trust
catch (error)	unknown	instanceof or helper

2.6.3.3 Checkpoint

For each value in the table, identify the first runtime check you would make. Then explain why an interface alone cannot replace that check.

Answers: start JSON and network data as **unknown**; narrow or validate before property access; an interface is erased and cannot inspect the bytes or object that arrived at runtime.

The running report needs the same boundary treatment:

```

function parseStatusReport(value: unknown): StatusReport | undefined {
    if (!value || typeof value !== "object") return undefined;
    const candidate = value as {
        status?: unknown;
        message?: unknown;
        updatedAt?: unknown;
    };
    if (candidate.status !== "idle" &&
        candidate.status !== "running" &&
        candidate.status !== "failed") return undefined;
    if (typeof candidate.message !== "string") return undefined;
    if (typeof candidate.updatedAt !== "number") return undefined;
    return {
        status: candidate.status,
        message: candidate.message,
        updatedAt: candidate.updatedAt,
    };
}

const text = '{"status":"running","message":"working","updatedAt":0}';
const report = parseStatusReport(JSON.parse(text));

```

The return value is safe to use because it was built from checked fields.

2.6.4 9. Runtime schemas with TypeBox

Vocabulary: a *schema* is a runtime description of allowed data. A *static type* is what the TypeScript checker derives or verifies before execution. `Static<typeof Schema>` means “the TypeScript type derived from the schema value named `Schema`.” `Value.Check` is a runtime boolean test. JSON Schema is a portable data-description format that tool providers can consume.

TypeBox gives Pi a runtime schema and a matching static type. The schema is a runtime value; `Static<typeof Schema>` is a compile-time type derived from it.

```
import { Type, type Static } from "@earendil-works/pi-ai";
import { Value } from "typebox/value";

const UserSchema = Type.Object({
  name: Type.String(),
  age: Type.Number(),
});

type User = Static<typeof UserSchema>;

function readUser(raw: unknown): User | undefined {
  if (!Value.Check(UserSchema, raw)) return undefined;
  return raw;
}

const valid = readUser({ name: "Ada", age: 36 }); // User
const invalid = readUser({ name: "Ada", age: "old" }); // undefined
```

If the schema changes, the derived type changes with it. This avoids maintaining an interface and validator that drift apart.

2.6.4.1 Builders

Builder	Runtime meaning	Static result
<code>Type.String()</code>	string	<code>string</code>
<code>Type.Number()</code>	number	<code>number</code>
<code>Type.Boolean()</code>	boolean	<code>boolean</code>
<code>Type.Object({...})</code>	object with fields	object type
<code>Type.Array(X)</code>	array of X	<code>X[]</code>
<code>Type.Optional(X)</code>	key may be omitted	<code>X undefined</code>
<code>Type.Union([A, B])</code>	either schema	<code>A B</code>
<code>Type.Literal("x")</code>	exact value	<code>"x"</code>

For example:

```
const SearchSchema = Type.Object({
  pattern: Type.String({ description: "Text to find" }),
  path: Type.Optional(Type.String()),
  caseSensitive: Type.Boolean(),
});

type SearchParams = Static<typeof SearchSchema>;
// { pattern: string; path?: string; caseSensitive: boolean }
```

See each builder in isolation:

```
const title = Type.String();
const count = Type.Number();
const enabled = Type.Boolean();
const tags = Type.Array(Type.String());
const mode = Type.Union([Type.Literal("read"), Type.Literal("write")]);
const optionalPath = Type.Optional(Type.String());
```

Each expression is a runtime schema value. `Static<typeof title>` would be `string`; `Static<typeof tags>` would be `string[]`.

At runtime, an optional key may be absent. In the static type, it may be `undefined`. Validation does not automatically fill defaults or migrate older config versions.

2.6.4.2 Tool schemas

A Pi extension tool uses a schema as the parameter contract. Conceptually, first focus only on the relationship:

```
// Conceptual shape - schema validation happens before execute
const parameters = Type.Object({ message: Type.String() });

type Params = Static<typeof parameters>; // { message: string }

async function execute(params: Params): Promise<string> {
  return params.message;
}
```

The real Pi API adds metadata and lifecycle arguments:

```
import { Type } from "@earendil-works/pi-ai";
import { defineTool } from "@earendil-works/pi-coding-agent";

export default defineTool({
  name: "status",
  label: "Status",
  description: "Report the current status.",
```

```

parameters: Type.Object({
  message: Type.String({ description: "Status message" }),
}),
execute: async (_toolCallId, params, signal, _onUpdate, _ctx) => {
  if (signal?.aborted) {
    return { content: [{ type: "text", text: "cancelled" }] };
  }
  return {
    content: [{ type: "text", text: params.message }],
  };
},
});

```

`defineTool` connects the `parameters` schema to the inferred `params` type. The agent validates raw tool arguments before execution; the extension still owns its own assumptions about downstream values and side effects.

The running example has now crossed the full boundary:

```

const StatusSchema = Type.Object({
  status: Type.Union([
    Type.Literal("idle"),
    Type.Literal("running"),
    Type.Literal("failed"),
  ]),
  message: Type.String(),
  updatedAt: Type.Number(),
});

type ValidatedStatusReport = Static<typeof StatusSchema>;

function validateStatus(value: unknown): ValidatedStatusReport | undefined {
  return Value.Check(StatusSchema, value) ? value : undefined;
}

```

The schema replaces the repeated field checks from Section 8 when this shape is shared or exposed as a tool contract.

2.6.4.3 StringEnum

Some providers expect a JSON Schema string enum:

```

import { StringEnum } from "@earendil-works/pi-ai";

const Operation = StringEnum(["read", "write"] as const);
// { type: "string", enum: ["read", "write"] }

```

A union of `Type.Literal` values can emit an `anyOf/const` shape instead. Use the helper when the target provider expects the simpler enum representation. Its implementation is in `packages/ai/src/utils/typebox-helpers.ts`.

2.6.4.4 Versioned configuration

Validation answers “does this value match this schema?” Migration may still be needed:

```
interface Config { version: number; newField?: string }
const DEFAULT_VALUE = "default";
const config: Config = { version: 1 };

if (config.version < 2) {
  config.newField = config.newField ?? DEFAULT_VALUE;
  // Rename or remove fields from old versions here.
}
```

2.6.4.5 Pi source map

- `TypeBox` is re-exported by `packages/ai/src/index.ts`.
- Tool argument validation is part of the agent loop.
- `defineTool` and its schema-derived `Static<TParams>` parameter are in `packages/coding-agent/src/core/e`
- Extension examples show the public authoring API.

2.6.4.6 Checkpoint

Read a schema and answer:

1. What is its `Static` type?
2. Which keys may be absent?
3. Where does `Value.Check` run?
4. Which assumptions still require application code?

Answers: derive the object fields and their static types; `Type.Optional` keys may be absent; `Value.Check` inspects the actual runtime value; defaults, migrations, provider-specific schema formats, and side effects still require application code.

2.7 Part 4: Async work and object lifecycle

2.7.1 10. Promises, async functions, and streams

Vocabulary: a `Promise<T>` is an object representing one future result. A Promise begins *pending* and becomes *fulfilled* with a value or *rejected* with an error. An `async` function returns a Promise automatically. An *async iterable* can produce many values over time; an async generator uses `yield` to produce them one at a time.

An `async` function always returns a Promise. Its annotation describes the resolved value:

```
async function loadCount(): Promise<number> {  
    return 3;  
}  
  
const count: number = await loadCount();
```

Compare a then-chain with `async/await`:

```
function loadThen(): Promise<string> {  
    return fetch("/status")  
        .then((response) => response.text());  
}  
  
async function loadAwait(): Promise<string> {  
    const response = await fetch("/status");  
    return response.text();  
}
```

Use `try/catch` around awaited work when you can recover or report the failure:

```
async function run(): Promise<void> {  
    try {  
        const text = await loadAwait();  
        console.log(text);  
    } catch (error) {  
        console.error(messageOf(error));  
    }  
}
```

For independent work:

```
const [one, two] = await Promise.all([loadCount(), loadCount()]);  
const results = await Promise.allSettled([loadCount(), loadCount()]);
```

`Promise.all` rejects when one operation rejects. `allSettled` reports every operation, whether fulfilled or rejected.

The result shapes make the difference visible:

```
// Promise.all: values are available only if every operation succeeds
const values: number[] = await Promise.all([loadCount(), loadCount()]);

// allSettled: inspect each outcome separately
for (const result of await Promise.allSettled([loadCount(), loadCount()])) {
  if (result.status === "fulfilled") console.log(result.value);
  else console.error(result.reason);
}
```

2.7.1.1 Async iterables

A Promise produces one eventual value. An async iterable produces values over time. `for await...of` waits for each next value, and `yield` supplies one value from an async generator:

```
async function* tokens(): AsyncIterable<string> {
  yield "Hel";
  yield "lo";
}

for await (const token of tokens()) {
  process.stdout.write(token);
}
```

Pi provider and agent streams use this mental model. Look for `for await` and async generator functions when following streaming code.

A status reporter can stream updates without changing the event shape:

```
async function* statusUpdates(
  events: AsyncIterable<StatusEvent>,
): AsyncIterable<string> {
  for await (const event of events) {
    yield eventLabel(event);
  }
}
```

The event values arrive over time, but each value is still narrowed by its `type` tag.

2.7.1.2 Checkpoint

- Predict whether a function returns `Promise<T>` or `AsyncIterable<T>`.
- Choose `Promise.all` or `Promise.allSettled` for a stated requirement.
- Rewrite one then-chain using `await` and `try/catch`.

Answers: one eventual result uses `Promise<T>`; a stream uses `AsyncIterable<T>`; use `all` when all work is required and `allSettled` when partial results matter; `await` moves the result into ordinary sequential code where `try/catch` can handle rejection.

2.7.2 11. Cancellation with AbortSignal

Vocabulary: an `AbortController` owns cancellation and can trigger it. An `AbortSignal` is the notification passed to work that may be cancelled. A *parent signal* belongs to the caller; a child operation should usually forward it. An abort error means cancellation happened; it is not automatically a programming bug.

Cancellation is a signal, not an unrelated boolean:

```
const controller = new AbortController();
const { signal } = controller;
const url = "https://example.invalid/status";

const response = fetch(url, { signal });
controller.abort(); // the fetch rejects with an abort error
```

The lifecycle is:

1. receive or create a signal
2. pass it to cancellable work
3. abort the owning controller from the cancel path
4. distinguish expected cancellation from unexpected failure

The important distinction is between checking a flag and cancelling the operation itself:

```
async function expensiveOperation(): Promise<void> {}

// Checking after the work does not cancel the work
if (signal.aborted) return;
await expensiveOperation();

// Passing the signal lets the API stop while it is running
await fetch(url, { signal });
```

A helper should forward the signal it receives:

```
async function fetchBad(url: string, signal: AbortSignal): Promise<string> {
  const response = await fetch(url); // parent cancellation is ignored
  return response.text();
}

async function fetchGood(url: string, signal: AbortSignal): Promise<string> {
  const response = await fetch(url, { signal });
  return response.text();
}
```

Handle an abort without swallowing unrelated failures:

```

async function load(url: string, signal: AbortSignal): Promise<string> {
  try {
    const response = await fetch(url, { signal });
    return response.text();
  } catch (error) {
    if (signal.aborted) return "";
    throw error;
  }
}

```

Use platform helpers when a child operation also needs a timeout:

```

async function loadWithTimeout(
  url: string,
  parent: AbortSignal,
): Promise<string> {
  const signal = AbortSignal.any([parent, AbortSignal.timeout(5_000)]);
  const response = await fetch(url, { signal });
  return response.text();
}

```

Do not create an unrelated child controller and expect the parent to cancel it. Check `signal.aborted` before expensive setup when work may start late.

Pi supplies signals to agent/provider work and tool execution. Extensions that perform network or subprocess I/O should forward the signal from their tool context.

The running reporter can stop waiting for updates when its parent cancels:

```

async function consumeStatuses(
  events: AsyncIterable<StatusEvent>,
  signal: AbortSignal,
): Promise<void> {
  for await (const event of events) {
    if (signal.aborted) return;
    console.log(eventLabel(event));
  }
}

```

For network or subprocess work inside the loop, pass the same signal to that operation rather than checking the flag only after the work finishes.

2.7.2.1 Checkpoint

Find the cancellation bug in each example:

- a helper accepts no signal
- a helper accepts a signal but omits it from `fetch`

- a helper catches every error and returns a normal result
- a child creates a controller without linking it to its parent

Answers: the first cannot receive cancellation; the second drops the signal; the third hides both cancellation and unexpected failures; the fourth listens to a controller the parent cannot abort. The corrected pattern passes the parent signal into the cancellable API and handles abort separately.

2.7.3 12. Classes, inheritance, and cleanup

Vocabulary: a *class* is a runtime object template. A *field* stores state on an instance. A *method* is a function on the class. A *getter* looks like a property when called. **private**, **protected**, and **public** are visibility contracts. *Inheritance* lets a child class reuse a parent class's behavior.

Use a class when state survives several calls, multiple methods share that state, or the object owns a lifetime. Use a function for a pure transformation:

```
export function clamp(value: number, minimum: number, maximum: number): number {
    return Math.max(minimum, Math.min(maximum, value));
}
```

A plain JavaScript class already has runtime behavior:

```
class CounterJS {
    constructor(initial) { this.value = initial; }
    increment() { return ++this.value; }
}
```

TypeScript adds checked field and method contracts. A class can expose private state through a small public API:

```
class Counter {
    private value: number;
    private readonly handlers: Array<() => void> = [];

    constructor(initial: number) {
        this.value = initial;
    }

    increment(): number {
        this.value += 1;
        return this.value;
    }

    get current(): number {
        return this.value;
    }
}
```

private is checked by TypeScript; it is not a runtime security boundary. **protected** allows subclasses to use a member. Public members form the API.

2.7.3.1 extends and implements

`extends` is runtime JavaScript inheritance. The child gets the parent's methods through the prototype chain; `super()` runs the parent constructor:

```
class CancellableError extends Error {
  readonly code: string;

  constructor(code: string, message: string) {
    super(message);
    this.code = code;
    this.name = "CancellableError";
  }
}
```

The constructor must call `super()` before accessing `this`. `implements` is a type-checking checklist and does not create runtime behavior:

```
interface Startable {
  start(): void;
  stop(): void;
}

class Worker implements Startable {
  start(): void {}
  stop(): void {}
}

class WorkerWithName implements Startable {
  protected readonly name: string;

  constructor(name: string) {
    this.name = name;
  }

  start(): void { console.log(`${this.name} started`); }
  stop(): void { console.log(`${this.name} stopped`); }
}
```

`protected` allows a subclass to use `name`, while callers outside the class cannot. `implements` only checks the class shape; it does not create methods.

2.7.3.2 Erasable syntax in Pi

A *parameter property* declares and assigns a class field inside a constructor parameter, such as `constructor(private value: number)`. Pi's checked TypeScript uses `erasableSyntaxOnly`. Avoid syntax that requires a compiler to generate runtime code:

```
// Avoid in Pi-checked source
class Bad {
  constructor(private value: number) {}
}

enum Mode {
  Idle,
  Running,
}
```

Use explicit fields and literal unions instead:

```
class Good {
  private value: number;

  constructor(value: number) {
    this.value = value;
  }
}

type Mode = "idle" | "running";
```

Pi also avoids namespaces in favor of ESM modules.

2.7.3.3 Cleanup and ownership

A class that owns a controller or subscription owns the cleanup:

```
class SessionRunner implements Startable {
  private controller = new AbortController();

  start(): void {
    this.controller = new AbortController();
  }

  stop(): void {
    this.controller.abort();
  }
}
```

Pi's `Agent` and `TUI` classes own mutable state, listeners, buffers, and lifecycle. Read their public methods before reaching for private fields.

2.7.3.4 Checkpoint

- Choose a function or class for a given responsibility.
- Explain **extends** versus **implements**.
- Rewrite a parameter property using an explicit field.
- Identify which object owns cleanup for a signal or subscription.

Answers: use a function for stateless transformation and a class for shared state or lifetime; **extends** creates runtime inheritance while **implements** checks a contract; declare the field and assign it in the constructor; the class that creates the controller or subscription owns stopping and disposing it.

2.8 Part 5: Apply the skills to Pi

2.8.1 13. Pi's runtime and developer workflow

Vocabulary: *type stripping* removes erasable annotations so Node can run a `.ts` file; it does not check types. `tsx` runs TypeScript during development. `jiti` loads extension files on demand. `tsgo` is Pi's type-checking/build tool. `erasableSyntaxOnly` is a TypeScript option that rejects syntax Node cannot strip without generating JavaScript.

Pi runs TypeScript through several mechanisms. They serve different purposes:

Situation	Mechanism	Type-checks?
Source execution on supported Node	type stripping	no
Development launcher	<code>tsx</code>	no
Extension loading	<code>jiti</code>	no
Package build	<code>tsgo</code> emit	yes during build
Quality gate	<code>npm run check</code>	yes

Runtime success does not prove that the type checker accepts the code. The repository's root configuration enables strict checking and erasable syntax.

A normal edit loop is:

```
# from the monorepo root
npx tsgo --noEmit
npm run check
./pi-test.sh
```

Use a focused command while iterating, then the repository gate before sharing a change. Do not use `tsx` or `jiti` as substitutes for type checking.

A successful run means the program started and executed. A successful type check means the checker found no reported type errors. You need both kinds of evidence.

Checkpoint: Which command installs packages, which command runs code, and which command checks types? What does it mean if a file runs but `tsgo` fails?

2.8.2 14. Guided source tour: a custom tool

Vocabulary: a *tool* is a named operation the model can request. A *tool call ID* identifies one request. *execute* is the callback the host invokes later. A *tool result* is the runtime object returned to the host. The update callback and context are optional channels for progress and host services.

Before the full Pi signature, reduce the idea to its core:

```
import { Type, type Static } from "@earendil-works/pi-ai";

const parameters = Type.Object({ message: Type.String() });
type Params = Static<typeof parameters>;

async function execute(params: Params): Promise<string> {
  return params.message;
}
```

The real Pi API adds metadata and lifecycle arguments:

Start with the public shape of a tool:

```
import { Type } from "@earendil-works/pi-ai";
import { defineTool } from "@earendil-works/pi-coding-agent";

export default defineTool({
  name: "status",
  label: "Status",
  description: "Report a status message.",
  parameters: Type.Object({
    message: Type.String(),
  }),
  execute: async (_toolCallId, params, signal, _onUpdate, _ctx) => {
    if (signal?.aborted) {
      return { content: [{ type: "text", text: "cancelled" }] };
    }
    return { content: [{ type: "text", text: params.message }] };
  },
});
```

Read it in this order:

1. **Imports:** `Type` and `defineTool` are runtime values.
2. **Schema:** `parameters` is a runtime validation description.
3. **Callback:** `execute` is stored now and invoked later.
4. **Arguments:** `params` is statically inferred but originated as external input.
5. **Cancellation:** `signal` can stop I/O owned by the tool.
6. **Result:** `content` is a runtime object passed back to the host.

The numbered reading order above maps directly to the code block. The real helper is defined in `packages/coding-agent/src/core/extensions/types.ts`. Its `execute` callback receives the tool call ID, schema-derived parameters, optional signal, update callback, and extension context.

2.8.3 15. Guided source tour: an agent event

Vocabulary: an *event* is a runtime record announcing that something happened. An *emitter* or producer creates and publishes the event. A *consumer* or handler receives it. An event *lifecycle* is the ordered sequence such as start, update, and end.

The following is a simplified excerpt of the agent event union. The real union is larger and should be read in the source file named below:

```
type AgentMessage = { role: string; content: unknown };
type AgentEvent =
  | { type: "agent_start" }
  | { type: "message_update"; message: AgentMessage }
  | { type: "tool_execution_end"; toolName: string; isError: boolean };
```

The actual union is larger. The reading method is the same:

1. Find the `type` literals.
2. Locate the switch or callback consuming them.
3. Read only fields on the selected variant.
4. Look for `never` if all variants must be handled.
5. Check whether the event is emitted immediately or stored for later.

The agent loop in `packages/agent/src/agent-loop.ts` emits and consumes these lifecycle events. In other words, the loop is a producer of records and a handler/stream consumer decides what to do with each record. Assistant content blocks and session entries use similar tagged shapes.

Checkpoint: If a new `tool_execution_cancelled` variant is added, search for the union definition and every switch or handler that consumes `AgentEvent`. Then decide whether the new payload crossed an untrusted boundary.

2.8.4 16. Guided source tour: state and lifecycle

Vocabulary: *ownership* asks which object is responsible for creating, changing, and releasing a resource. An *alias* is another reference to the same object. A *lifecycle* is the sequence from creation through active work to shutdown. `dispose` means release listeners, timers, processes, or other resources.

When reading an agent run, trace ownership instead of every field:

- Which function receives the `AbortSignal`?
- Which helper forwards it to I/O?
- Which arrays are copied before a run?
- Which class owns listeners or buffers?
- Which method stops or disposes the work?

This connects the earlier concepts. Read this miniature from top to bottom:

```

class Runner {
  private readonly listeners: Array<(message: string) => void> = [];
  private controller = new AbortController();
  private messages: readonly string[] = [];

  onMessage(listener: (message: string) => void): void {
    this.listeners.push(listener);
  }

  run(message: string): void {
    this.messages = [...this.messages, message];
    for (const listener of this.listeners) listener(message);
  }

  stop(): void {
    this.controller.abort();
    this.listeners.length = 0;
  }
}

```

Answers to the ownership questions: `listeners` belongs to `Runner`; the array copy prevents replacement of the old message list; the controller owns cancellation; `onMessage` stores callbacks; and `stop` releases both work and listeners.

Source pattern	Question to ask
<code>readonly Message[]</code>	Who may replace the container?
<code>[...messages]</code> or <code>.slice()</code>	Which alias is being isolated?
<code>signal</code> parameter	Is cancellation forwarded?
<code>type</code> field	Which union variant is active?
<code>private</code> field	Which class owns the state?
<code>stop()</code> / <code>dispose()</code>	Who releases resources?

2.8.5 17. Make a small safe change

Vocabulary: a *declaration* describes a type or function. A *construction site* creates a value of that type. A *caller* invokes a function. A *contract* is the set of assumptions callers and implementations share. A *focused check* is a narrow type-check or test run used while iterating.

Use this workflow when changing a Pi TypeScript file:

1. Find the declaration of the type or function.
2. Find construction sites and callers.
3. Identify discriminants and optional fields.
4. Mark any JSON, model, network, or extension boundary.
5. Check ownership of arrays, callbacks, signals, and cleanup.
6. Make the smallest change that preserves the existing contract.
7. Add a case to exhaustive switches when adding a union variant.

8. Run a focused check, then `npm run check`.
9. Review the diff for accidental API or runtime changes.

2.8.5.1 Practice change: add a new event variant

Suppose this union gains:

```
type Event =  
  | { type: "started" }  
  | { type: "finished"; durationMs: number }  
  | { type: "failed"; message: string };
```

Before editing a consumer, locate every switch over `event.type`. Add the new case, update the **never** check if needed, and decide whether any runtime payload validation is required. This is the same workflow used for larger Pi unions.

A typical before/after change looks like this:

```
type Event =  
  | { type: "started" }  
  | { type: "finished"; durationMs: number }  
+ | { type: "failed"; message: string };
```

The declaration is only the first step. Search for construction sites that need to create **failed** events, consumers whose exhaustive switches now fail, and tests that describe the old event set. This is why a small type change can touch several files.

2.9 Reference: use when you get stuck

2.9.1 R1. Common TypeScript errors

Each entry follows the same pattern: representative compiler message, plain meaning, smallest fix, and the main section to revisit. Error wording can vary slightly by TypeScript version.

2.9.1.1 Type is not assignable

```
const count: number = "3"; // Type 'string' is not assignable to type 'number'  
const fixed: number = 3;
```

The value and destination types disagree. Inspect inference and annotations. **Return to:** Section 3.

2.9.1.2 Property does not exist

```
const user = { name: "Ada" };  
// user.age; // Property 'age' does not exist on type '{ name: string; }'
```

You may have a typo, or you may need to narrow a union first. **Return to:** Sections 3 and 4.

2.9.1.3 Object is possibly undefined

```
function lengthOf(value?: string): number {  
    // return value.length; // Object is possibly 'undefined'  
    return value?.length ?? 0;  
}
```

Use a guard, optional chaining, or a fallback. **Return to:** Sections 1 and 3.

2.9.1.4 Cannot find module

Typical message: Cannot find module './neighbor.ts' or its corresponding type declarations. Check the relative path, .ts suffix, package installation, and tsconfig path mapping. Do not deep-import another package's source internals. **Return to:** Sections 0 and 2.

2.9.1.5 Excess property check

```
interface Options { timeoutMs: number }  
// const options: Options = { timeoutMs: 1, extra: true }; // error
```

Fresh object literals receive an extra-field check. Assigned variables behave according to structural typing. **Return to:** Section 3.

2.9.1.6 never assignment fails

```
type Shape = { kind: "circle" } | { kind: "square" };  
  
function area(shape: Shape): number {  
    switch (shape.kind) {  
        case "circle": return 1;  
        case "square": return 4;  
        default: {  
            const exhaustive: never = shape;  
            return exhaustive;  
        }  
    }  
}
```

```

    }
  }
}

```

A new union variant requires a new case. **Return to:** Section 4.

2.9.1.7 Generic inference fails

Empty arrays provide little evidence:

```

function first<T>(values: readonly T[]): T | undefined { return values[0] }
const values = first<string>([]);

```

Supply a type argument only when inference cannot determine one. **Return to:** Section 7.

2.9.1.8 Import type misuse

A type-only import cannot be used as a runtime value. Remove `type` for a value import, or change the code so the name is used only in a type position. **Return to:** Section 2.

2.9.1.9 Private member access

```

class Store { private value = 1 }
// new Store().value; // error

```

Use the class's public method or getter. Do not cast around ownership boundaries. **Return to:** Section 12.

2.9.1.10 Erasable syntax

```

// Avoid under Pi's erasableSyntaxOnly
// class Bad { constructor(private value: number) {} }

class Good {
  private value: number;
  constructor(value: number) { this.value = value; }
}

```

A typical message is that the syntax is not allowed when `erasableSyntaxOnly` is enabled. Declare the field explicitly instead. **Return to:** Sections 12 and 13.

2.9.2 R2. Utility type quick reference

These are compile-time transformations. They do not make runtime objects partial, readonly, or validated. **T** means a source type, **K** means a selected key or union of keys, and **V** means a value type.

Type	Meaning
<code>Partial<T></code>	all properties optional
<code>Required<T></code>	all properties required
<code>Readonly<T></code>	top-level properties non-reassignable
<code>Pick<T, K></code>	keep selected keys
<code>Omit<T, K></code>	remove selected keys
<code>Record<K, V></code>	map keys to values
<code>ReturnType<typeof fn></code>	function result type
<code>Parameters<typeof fn></code>	function argument tuple
<code>NonNullable<T></code>	remove <code>null</code> and <code>undefined</code>

Examples:

```
interface Config { port: number; host: string }
type Patch = Partial<Config>;           // { port?: number; host?: string }
type HostOnly = Pick<Config, "host">; // { host: string }
type Flags = Record<"debug" | "verbose", boolean>;
```

Return to: Section 7. For runtime behavior, add validation or copying code separately.

2.9.3 R3. Glossary

Term	Meaning
ESM / module	A JavaScript file that exports and imports names
Runtime	The JavaScript execution where values and side effects occur
Type checker	A tool that reports type inconsistencies before execution
Annotation	An explicit type written by the author
Inference	The checker deriving a type from context
Union	A value that may have one of several types
Discriminant	A literal tag selecting a union variant
Narrowing	Refining a union after a successful check
Exhaustiveness	Handling every variant of a closed union
<code>never</code>	A type with no possible values
Type predicate	A return type such as <code>value is User</code> that narrows for callers
Generic	A type parameter chosen by the caller
Constraint	A limit such as <code>T extends HasName</code>

Term	Meaning
<code>unknown</code>	A safe type requiring narrowing before use
<code>any</code>	An escape hatch that disables checking
Type assertion	An <code>as</code> claim; it is not validation
Type-only import	An import removed from runtime output
TypeBox schema	A runtime schema that can also produce a static type
<code>Static<typeof S></code>	The static type derived from schema <code>S</code>
<code>AbortSignal</code>	A standard cancellation signal
<code>AbortController</code>	The object that triggers cancellation for its signal
<code>Promise</code>	One future fulfilled value or rejected error
<code>Callback</code>	A function passed to be called now or later
<code>Closure</code>	A function retaining variables from its defining scope
<code>Boundary</code>	A point where untrusted external data enters
<code>Schema</code>	A runtime description of allowed data
<code>satisfies</code>	Checks a value against a type while preserving inference
<code>as const</code>	Preserves literals and adds type-level readonly
<code>keyof</code>	The union of keys in an object type
Indexed access	A type expression such as <code>T[K]</code> selecting a property value type
Parameter property	Constructor syntax that declares and assigns a field
<code>protected</code>	A class member available to the class and its subclasses
<code>tsgo</code>	Pi's type-checking/build command
<code>tsx</code>	A development runner for TypeScript source
<code>jiti</code>	A runtime loader used for extension files
Erasable syntax	TypeScript syntax Node can strip without code generation

Common confusions:

- `as` is not conversion; it changes only the checker's belief.
- `readonly` is not `Object.freeze`; it is normally a static restriction.
- `extends` creates runtime inheritance; `implements` only checks a shape.
- `Static<typeof Schema>` is a compile-time type; the schema value is runtime.

2.9.4 R4. Cumulative self-check

Read this small Pi-shaped function from outside in. It combines a union, `unknown`, a callback, a `Promise`, and cancellation only after each concept has been introduced in the main path:

```
type Input =
  | { type: "message"; text: string }
  | { type: "tool"; name: string; args: unknown };
```

```

async function runTool(
  name: string,
  args: unknown,
  signal: AbortSignal,
): Promise<void> {
  // A real implementation validates args before using them.
  void name;
  void args;
  void signal;
}

async function handle(
  input: Input,
  signal: AbortSignal,
  emit: (event: { type: "log"; message: string }) => void,
): Promise<void> {
  if (input.type === "message") {
    emit({ type: "log", message: input.text });
    return;
  }

  if (signal.aborted) return;
  // Validate input.args before passing it to a tool.
  await runTool(input.name, input.args, signal);
}

```

Questions:

1. Which field narrows `input`?
2. Why is `input.text` unavailable in the tool branch?
3. Which value is untrusted?
4. What does the `AbortSignal` protect?
5. Is `emit` called now or stored for later?
6. What return type does `handle` have?

Answers:

1. `input.type` narrows the union.
2. Only the message variant has `text`.
3. `input.args` is `unknown` and needs validation before `runTool` trusts it.
4. It lets the parent cancel before or during tool work; checking the flag alone does not cancel an operation that is already running.
5. `emit` is called immediately in this function, though a callback can also be stored.
6. `Promise<void>` because it is an async function with no returned value.

2.9.5 R5. Type-checking a standalone extension

Use this only when an extension lives outside the monorepo. An extension inside the monorepo uses the root workspace configuration; a standalone extension must install the published Pi packages and provide its own `tsconfig.json`.

Option	Meaning
<code>module</code>	how imports and exports are represented
<code>moduleResolution</code>	how the checker finds imported files
<code>strict</code>	enable the strongest useful checking defaults
<code>skipLibCheck</code>	skip checking dependency declaration files
<code>noEmit</code>	check without writing JavaScript
<code>include</code>	which files belong to this project

Use this only when an extension lives outside the monorepo:

```
npm install --save-dev @earendil-works/pi-ai \
  @earendil-works/pi-coding-agent typescript
```

```
{
  "compilerOptions": {
    "target": "ESNext",
    "module": "ESNext",
    "moduleResolution": "bundler",
    "strict": true,
    "skipLibCheck": true,
    "noEmit": true
  },
  "include": ["src/**/*.ts"]
}
```

Adjust `include` to your project layout, then run:

```
npx tsc --noEmit
```

Success means the command exits without reported errors; failure points to a file and line that needs attention. Runtime loading through `jiti` is separate from this check. A runtime that loads an extension successfully does not prove that the extension type-checks. If a package cannot be resolved, check the published package installation, module-resolution setting, and import spelling.

2.10 Closing

You are ready for the architecture and extension chapters when you can:

- read a module and identify runtime versus type-only imports

- follow object shapes, unions, and exhaustive switches
- recognize generics without treating `T` as a runtime value
- treat JSON, model output, and tool arguments as untrusted until validated
- follow `Promise`, `stream`, and `AbortSignal` ownership
- recognize Pi's `defineTool`, `AgentEvent`, snapshots, and class lifecycle
- make a small change and run the repository's type-checking workflow

When a later chapter cites a type-heavy file, return to the relevant guided tour rather than trying to memorize every TypeScript feature at once.

Your next practice step is:

1. open one source-tour file
2. trace one union, tool schema, callback, or signal
3. make a documentation-only or test-only change and run the focused check

The reference sections are for revisiting a term when it appears, not for memorizing every utility type before you start contributing.

3 Architecture: packages, ownership, and entry points

Pi deliberately separates generic agent machinery from the coding-agent application. A useful rule is: move a feature down only when it is genuinely useful outside the coding agent. The coding-agent package owns command-line behavior, local filesystem tools, project resources, session UX, and the extension API; the **agent** package should not learn those policies.

3.1 What you will learn first

Start with the package map, then follow the startup diagram, and only then read the package walk-through. A **package** is a deployable unit with its own public exports. A **dependency** is a package another package imports. An **entry point** is the file where a feature or application begins. A **boundary** is a place where one representation becomes another, such as a provider message becoming an agent event.

For a small example, the terminal-facing **coding-agent** package depends on the generic **agent** package, while **agent** depends on the provider-neutral **ai** package. The direction tells you where a fix belongs: a terminal component does not belong in **packages/agent**, and provider JSON does not belong in the TUI.

3.2 Dependency direction

Dependent package	-> Direct runtime dependency
packages/orchestrator (experimental)	-> packages/coding-agent
packages/coding-agent	-> packages/tui
packages/coding-agent	-> packages/agent
packages/coding-agent	-> packages/ai
packages/agent	-> packages/ai

Each arrow points from the package that imports and depends on another package to that dependency. The list shows the normal architectural direction, not every development-tool dependency. **coding-agent** integrates all lower layers and depends directly on each of them; **agent** also depends on the AI message/model contracts. **tui** has no dependencies on other packages in this monorepo, allowing it to be reused by other terminal applications.

3.3 Repository-level conventions

The root `package.json` defines npm workspaces for `packages/*` and selected extension examples. All packages are ESM. The root `tsconfig.json` maps public package names back to source files for whole-repository checking. Each package has its own `package.json` with public exports and build script. Release versioning is lockstep: all packages share one version and release together.

When developing inside this checkout, import the public package name if you are crossing a package boundary. Use a relative `.ts` import only within a package. This preserves the boundary that published consumers see.

The normal quality gate after code changes is `npm run check`. Per repository rules, do not run `npm run build` or the full test suite merely as a habit; use the requested or relevant targeted validation. To validate a change confined to `developer-guide/`, run `quarto render developer-guide` from the repository root; `npm run check` does not render this book.

3.4 Component walkthrough

3.4.1 `packages/tui`: terminal rendering primitives

`packages/tui/src/index.ts` is the public barrel. It exports the TUI class, the `Component` interface, containers, text/editor/list components, terminal abstractions, key parsing, keybindings, autocomplete, and ANSI-aware text utilities.

`packages/tui/src/tui.ts` owns the component tree, focus, layout, overlays, input routing, and differential rendering. `src/terminal.ts` provides the process-terminal boundary. The components under `src/components/` render specific UI pieces; `src/keybindings.ts`, `src/keys.ts`, and the editor components handle keyboard semantics. This package does not know what a model, tool call, session, or extension is.

Use this layer when a change is a general terminal interaction primitive. For a Pi-specific chat row, command, or tool renderer, start in `coding-agent` instead.

3.4.2 `packages/ai`: a provider-neutral streaming contract

The small public entry point `packages/ai/src/index.ts` exports core types, models, authentication utilities, event-stream helpers, `TypeBox`, and selected API option types. It intentionally avoids eagerly importing provider catalogs and provider factories. Provider-specific modules live under `src/providers/` and streaming implementations under `src/api/`.

The core concepts are defined in `src/types.ts`:

- A `Model` carries provider/API metadata and capabilities.
- A `Context` combines the system prompt, normalized messages, and tool definitions.
- An `AssistantMessageEventStream` yields provider-independent start, text, thinking, tool-call, completion, and error events.

`src/compat.ts` supplies the compatibility `streamSimple()` dispatch used on the coding-agent path. It chooses a built-in model registry or resolves an API provider from `model.api`, then passes a `Context` plus simple stream options to that provider. `src/models.ts` exposes a `Models` implementation that owns provider registration and routes model requests. Individual files such as `src/api/openai-responses.ts` and `src/api/anthropic-messages.ts` serialize the normalized context into a provider request and translate that provider’s stream back into Pi events.

Provider adapters are the right place for protocol-specific JSON, streaming parsing, or provider quirks. Do not put a UI decision or a built-in filesystem tool in this package.

3.4.3 packages/agent: generic conversation and tool loop

`packages/agent/src/types.ts` declares the generic agent contract: agent messages, tools, events, loop configuration, hooks, queue modes, and tool execution mode. `src/agent.ts` wraps that loop in the stateful `Agent` class. It owns current model, system prompt, tools, messages, active runs, abort controllers, and steering/follow-up queues.

`src/agent-loop.ts` is the important execution core. It:

1. Emits agent/turn/message events.
2. Lets a caller transform agent messages, then converts them to LLM messages.
3. Builds a provider `Context` and consumes an async response stream.
4. Validates model-produced tool arguments.
5. Runs tools sequentially or with sequential preflight plus parallel execution.
6. Appends tool results and repeats until there are no tool calls or queued messages.

The generic loop deliberately receives callbacks such as `convertToLlm`, `transformContext`, `beforeToolCall`, and `afterToolCall`. It therefore has no dependency on `SessionManager`, local disk tools, or the interactive TUI.

3.4.4 packages/coding-agent: the Pi application

This is where the generic pieces become the `pi` command. Its package manifest defines the `pi` binary as `dist/cli.js`. In source, `src/cli.ts` sets process-level behavior and calls `main(process.argv.slice(2))` from `src/main.ts`.

`src/main.ts` parses command-line arguments, determines interactive/RPC/print mode, resolves trust and settings, creates the runtime, selects the model, and starts `InteractiveMode`, RPC mode, or print mode. It is composition and application policy, not the agent loop.

The major coding-agent subsystems are listed below. Every path in the **Important files** column is relative to `packages/coding-agent/src/`.

Subsystem	Responsibility	Important files
Session composition	Create settings, auth, model registry, resources, agent session	<code>core/sdk.ts</code> , <code>core/agent-session-services.ts</code>
Runtime replacement	Own a session plus cwd-bound services across new/resume/fork	<code>core/agent-session-runtime.ts</code>

Subsystem	Responsibility	Important files
Session policy	Prompt preprocessing, resource/system prompt assembly, retries, compaction, extension event bridge	<code>core/agent-session.ts</code>
Durable transcript	JSONL session entries, tree branches, context reconstruction	<code>core/session-manager.ts</code> , <code>core/messages.ts</code>
Resources	Load extensions, skills, prompts, themes, and context files	<code>core/resource-loader.ts</code> , <code>core/extensions/loader.ts</code>
Configuration	Global/project settings and defaults	<code>core/settings-manager.ts</code> , <code>core/defaults.ts</code>
Model/auth resolution	Registered models, dynamic providers, credential lookup	<code>core/model-registry.ts</code> , <code>core/auth-storage.ts</code>
Built-in tools	Read, bash, edit, write, search, file mutation coordination	<code>core/tools/</code> , <code>utils/tools-manager.ts</code>
Interactive app	Chat transcript, editor, dialogs, commands, renderer adapters	<code>modes/interactive/interactive-mode.ts</code> , <code>modes/interactive/components/</code>
Extensions	Types, discovery/loading, event dispatch, context wrappers	<code>core/extensions/types.ts</code> , <code>core/extensions/loader.ts</code> , <code>core/extensions/runner.ts</code> , <code>core/extensions/wrapper.ts</code>

The table is an orientation map, not an exhaustive API list. To navigate it, start with the subsystem matching the behavior you are changing, then follow its named entry file before reading neighboring implementation files. Public coding-agent exports come from `src/index.ts`; extension authors should import from the package root rather than internal paths unless an API is explicitly documented for that purpose.

3.4.5 packages/orchestrator: experimental process orchestration

`packages/orchestrator` depends on coding-agent and exports configuration, handlers, IPC client/server/protocol, process supervision, storage, and serve functions from `src/index.ts`. Its manifest describes it as experimental. Treat it as a separate surface for multi-process orchestration, not as a hidden step of normal prompt processing.

3.5 Constructing a running coding-agent session

The startup path is intentionally split so the runtime can be rebuilt when a session changes its working directory.

```
main.ts
-> createAgentSessionRuntime(...)
   createAgentSessionServices(...)
```

```

    SettingsManager + AuthStorage + ModelRegistry + DefaultResourceLoader
-> createAgentSessionFromServices(...)
    -> createAgentSession(...) in core/sdk.ts
        -> Agent + AgentSession + built-in/custom tool registry
-> InteractiveMode | runRpcMode | runPrintMode

```

`createAgentSessionServices()` reloads the resource loader and applies provider registrations that extensions made during their factories. `createAgentSession()` then resolves/restores a model and thinking level, creates an `Agent`, and wraps it in `AgentSession`. The `AgentSessionRuntime` class tears down the old extension/session runtime before applying a newly created one for operations such as session resume or fork.

3.6 Persistence and context are different things

Pi sessions are JSONL files managed by `SessionManager`. They are tree-shaped: entries have `id` and `parentId`, so a branch can exist in one file. The current branch is converted into LLM context by `buildContextEntries()` and related helpers in `core/session-manager.ts`.

The extension API exposes two methods relevant to this boundary. `pi.appendEntry(customType, data)` appends a custom session entry for durable extension state. `pi.sendMessage(message, options)` inserts a custom conversation message and can optionally trigger or queue model work.

Not every durable entry reaches the provider. An entry written with `pi.appendEntry()` does not participate in LLM context, while a message sent with `pi.sendMessage()` does. This distinction is central to extension design: keep private UI/state metadata out of model context unless the model needs it.

Compaction is another context transformation, not deletion. Pi writes a summary entry and reconstructs a compact context from the summary and newer entries. The generic algorithms are exported from `packages/agent/src/harness/compaction/`; coding-agent coordinates the user-facing policy and session entries.

3.7 Where to make a change

Use the narrowest layer that truthfully owns the behavior:

Desired change	First place to investigate
Add a provider/API serializer	<code>packages/ai/src/api/</code> and <code>src/providers/</code>
Change generic tool scheduling or streaming semantics	<code>packages/agent/src/agent-loop.ts</code>
Add keyboard/editor/component capability reusable by TUIs	<code>packages/tui/src/</code>
Change Pi commands, session behavior, built-in tool policy, settings	<code>packages/coding-agent/src/</code>
Add user/team behavior without forking Pi	Extension, skill, theme, prompt, settings, or package

Continue to [Prompt lifecycle](#) for a source-level trace of how these package boundaries cooperate from interactive input through model streaming, tool execution, persistence, and rendering.

3.8 Beginner checkpoint

Before continuing, name the owner for each change: a reusable terminal primitive, a provider serializer, a generic tool-loop rule, a Pi session rule, and a user-specific behavior. If you can point each one to `tui`, `ai`, `agent`, `coding-agent`, and an extension respectively, the package map is working.

4 Prompt lifecycle: from Enter to agent settlement

This chapter traces the normal interactive path through the agent lifecycle. It explains how event-driven transcript updates are requested, not terminal renderer layout or paint internals; for those, follow the `InteractiveMode` handlers into `packages/coding-agent/src/modes/interactive/components/` and `packages/tui`.

It is source-grounded in `packages/coding-agent/src/modes/interactive/interactive-mode.ts`, `packages/coding-agent/src/core/agent-session.ts`, `packages/agent/src/agent.ts`, `packages/agent/src/a` and the `packages/ai` dispatch layer.

The exact call graph also has guards for commands, compaction, retries, queued messages, and other modes. The numbered path summarizes the current source; its diagrams omit helper calls that do not alter the data/control boundary being explained.

4.1 A small vocabulary for the trace

An **event** is a record saying that something happened. A **stream** is a sequence of partial results delivered over time. A **turn** is one model request and the tool work that follows it. A **settled** run is one with no retry, compaction, or queued continuation left to perform.

Use two prompts to keep the distinction concrete: "What files changed?" usually creates one model turn, while "Read the config and fix the failing test" can create several turns because the model may call `read`, then `bash`, then answer. The path below explains what stays the same in both examples.

4.2 The complete path

```
Editor Enter
-> InteractiveMode.getUserInput()
-> AgentSession.prompt(text)
  -> extension command / input preprocessing / prompt expansion
  -> user + extension custom messages
  -> Agent.prompt(messages)
    -> agentLoop
      -> transform context -> convert messages -> provider stream
      -> assistant stream events -> tool validation/execution
      -> tool-result messages -> repeat or finish
    -> session persistence + extension events + transcript rendering
```

4.3 Stage 1: Terminal input enters the TUI

`InteractiveMode` owns the coding-agent's interactive screen. Its editor submission handler recognizes built-in slash commands and `!/**` user-shell commands before normal agent prompting. A normal idle submission is queued or resolved through `getUserInput()`. The `run()` loop awaits that method, then calls `await this.session.prompt(userInput)`.

When Pi is already streaming, regular Enter calls `session.prompt(text, { streamingBehavior: "steer" })`; Alt+Enter uses `"followUp"`. Neither starts a second concurrent agent run; the call queues a message for a defined loop boundary. During compaction, non-command input is held by interactive mode until the compaction flow completes.

The initial prompt supplied by command-line arguments travels through the same `session.prompt()` API. Print and RPC modes do not instantiate `InteractiveMode`, but they converge on `AgentSession` rather than reimplementing the provider path.

4.4 Stage 2: Commands, keybindings, and input interception

There are two distinct command systems:

- Built-in interactive commands such as `/model`, `/settings`, `/new`, and `/compact` are handled in `InteractiveMode` before a normal prompt reaches the session.
- Extension commands registered with `pi.registerCommand()` are detected first inside `AgentSession.prompt()`. They execute immediately, including while an agent is streaming, and do not enter the model context unless their handler explicitly sends a message.

For ordinary text, `AgentSession.prompt()` asks the extension runner to emit an `input` event. The event sees raw text after extension-command lookup and before skill/template expansion. An extension can continue, transform, or handle it. If it continues, Pi expands `/skill:name ...` and prompt templates, then continues with the expanded text.

Hook selection comes first: built-in commands have already been handled by `InteractiveMode`, and a matching registered extension command has already returned before `input` is emitted. Therefore `input` is the interception point for ordinary text and unclaimed slash text. An extension that wants to intercept a skill invocation must use `input`; one that wants a durable custom command should use `registerCommand`, not parse slash text manually.

4.5 Stage 3: Agent-session orchestration constructs a turn

`AgentMessage` is the agent-core union of provider-compatible LLM `Message` values and application-defined custom message types. That lets the session retain rich application messages even when its later conversion step chooses a different LLM representation.

For a fresh run, `AgentSession.prompt()` validates the selected model and credentials, checks whether compaction is needed, and creates a user message. It then appends pending `nextTurn` messages: custom messages buffered by `sendCustomMessage(..., { deliverAs: "nextTurn" })` for inclusion with the *next* fresh user prompt, rather than delivery to an active run. Next it emits

`before_agent_start`, whose handlers can add custom messages and chain a system-prompt override, then calls `_runAgentPrompt(messages)`.

`_runAgentPrompt()` calls `Agent.prompt()`. It can later call `Agent.continue()` for automatic retry, automatic compaction retry, or work queued by an `agent_end` handler. On settlement it clears the per-run prompt override, flushes pending user-bash messages, and emits `agent_settled`.

The `Agent` owns the abort controller, streaming state, accumulated messages, active tools, and queues. `Agent.prompt()` snapshots its state and starts `runAgentLoop()`.

4.6 Stage 4: Context and system prompt cross three construction boundaries

Model choice is not part of this stage. The initial model is chosen earlier, during `createAgentSession()` in `core/sdk.ts` (session restore, configured default, or `findInitialModel()`), before `new Agent(...)`. The session can later change `Agent.state.model` (for example through `/model`). When a run starts, `Agent.prompt()` snapshots the then-current model together with the system prompt, tools, and transcript. Each assistant request in that run uses that already-chosen model plus messages accumulated by the loop. The three boundaries below therefore transform context *for* a chosen model; they do not select one.

First, the coding agent builds a base system prompt in `AgentSession._rebuildSystemPrompt()`. That method combines the working directory, loaded `AGENTS.md/context` files, loaded skills, custom and appended system prompts, and selected tools plus their snippets and guidelines. `buildSystemPrompt()` in `core/system-prompt.ts` converts the structured data to text. Changing active tools rebuilds the base prompt, so tool prompt metadata can affect the next model turn.

Immediately before *each* model request, `streamAssistantResponse()` in `agent-loop.ts` crosses the other two boundaries in sequence. It calls the optional `transformContext` callback on `AgentMessage[]`, then calls `convertToLlm()` to produce provider-compatible `Message[]`. Coding-agent supplies both callbacks when creating `Agent` in `core/sdk.ts`:

- `transformContext` dispatches the extension `context` event.
- `convertToLlm` filters and normalizes coding-agent-specific message types. When `blockImages` is enabled, this conversion-time step replaces image content in converted user and tool-result messages with a text placeholder, immediately before provider dispatch. It is dynamic, so a setting change affects subsequent conversions.

The resulting `Context` has exactly `systemPrompt`, `messages`, and `tools`. That shape lets the session persist rich UI-only entries while only a selected representation reaches the LLM.

When `streamAssistantResponse()` then invokes the installed stream function, it passes the *already* snapshotted model together with that normalized context. Provider dispatch (Stage 5) resolves credentials and an adapter for that model; context construction only prepares the adapter input.

4.7 Stage 5: Provider dispatch for the already-chosen model

This stage charts the coding-agent stream path from the snapshotted `Model` to a provider adapter. Model *catalog* selection and runtime changes belong to session creation and `/model` (see Stage 4’s opening and `developer-guide/09-models-providers-and-auth.qmd`); they are not re-decided here.

Coding-agent installs a custom `streamFn` on `Agent` in `core/sdk.ts`. For each assistant request the path is:

1. `streamAssistantResponse()` (agent-loop) calls `streamFn(model, context, options)` with the snapshotted model and the Stage 4 Context.
2. That `streamFn` resolves credentials with `modelRegistry.getApiKeyAndHeaders(model)`, merges provider-attribution and request headers (`mergeProviderAttributionHeaders`), then emits extension `before_provider_headers` so handlers can mutate the assembled headers.
3. `streamFn` calls `streamSimple(model, context, options)` from `packages/ai/src/compat.ts`, passing `apiKey`, merged headers, timeouts, and retries. Agent core also forwards `onPayload` / `onResponse` from the `Agent` instance into those options (wired in `createLoopConfig()`).
4. `compat.streamSimple` either delegates to built-in model support (`compatModels.streamSimple`) when the model matches a registered builtin API instance, or looks up the registered API provider by `model.api` and calls that provider’s `streamSimple`.
5. The adapter serializes the normalized context and returns a provider-independent `AssistantMessageEventStream`. During that path:

- `onPayload` becomes extension `before_provider_request` and may replace the provider-specific payload after serialization.
- `onResponse` becomes extension `after_provider_response` and observes HTTP status/headers before body consumption when the adapter exposes them.

`ModelRegistry` still owns the catalog, configured provider overrides, credential lookup, OAuth state, and header sources used by step 2; its role in this stage is auth/header resolution for the chosen model, not model selection.

Scope boundary for this chapter: the numbered path stops at adapter `streamSimple` and the three provider-extension insertion points. Vendor wire formats, transport negotiation, custom provider registration, and auth-storage details live in `developer-guide/09-models-providers-and-auth.qmd` and the provider-hook reference in `packages/coding-agent/docs/extensions.md` (`before_provider_headers`, `before_provider_request`, `after_provider_response`). Those headers/payload hooks are not substitutes for a custom provider; use `pi.registerProvider()` when you must declare a model catalog, endpoint, auth, or streaming implementation.

4.8 Stage 6: Streaming response becomes agent events and TUI updates

`streamAssistantResponse()` consumes the `AssistantMessageEventStream` with `for await`. It creates a partial assistant message on `start`, replaces that partial message as `text/thinking/tool-call` deltas arrive, and emits `message_start`, `message_update`, and `message_end` events.

`Agent.processEvents()` reduces those events into state and then awaits all subscribers. `AgentSession` bridges generic events to persistence, extensions, and application behavior. `InteractiveMode` updates the visible transcript, current streaming row, tool rows, footer, and render requests. The TUI package performs the actual terminal rendering.

The extension lifecycle maps directly onto this path: `agent_start`, `turn_start`, `message_start`, many `message_update` events, `message_end`, then later `turn_end` and `agent_end`.

4.8.1 Compact event reference

Event family	Meaning here	Detail
<code>agent_start</code> / <code>agent_end</code> / <code>agent_settled</code>	Low-level run starts, ends, then has no automatic continuation left	Stage 10; extension timeline
<code>turn_start</code> / <code>turn_end</code>	One assistant response and any resulting tool work	Stages 6–8
<code>message_start</code> / <code>message_update</code> / <code>message_end</code>	Transcript message begins, streams (assistant only), and finalizes	Stages 6 and 9
<code>tool_execution_start</code> / <code>tool_execution_update</code> / <code>tool_execution_end</code>	A tool row begins, receives partial output, and finalizes	Stage 7

For event payloads and extension ordering guarantees, see the lifecycle reference in `packages/coding-agent/docs/extension-lifecycle`.

4.9 Stage 7: Tool calls are parsed, validated, gated, and executed

After a final assistant message, `agent-loop.ts` filters content blocks whose `type` is `"toolCall"`. A `"length"` stop means the token limit cut off the assistant output, so any tool call may contain truncated JSON arguments even if its parsed block appears complete. The loop therefore fails every tool call in that message rather than risk side effects from incomplete arguments.

For each normal tool call, the loop:

1. Emit `tool_execution_start`.
2. Look up the active tool by name in the current context.
3. Apply its optional `prepareArguments` compatibility transform.
4. Validate arguments with `validateToolArguments()` against the `TypeBox` schema.
5. Invoke `beforeToolCall`, which `coding-agent` wires to extension `tool_call`. A handler may mutate validated input or return `{ block: true }`.
6. Execute `tool.execute(id, args, signal, onUpdate)`.
7. Convert incremental `onUpdate` values into `tool_execution_update` events.
8. Invoke `afterToolCall`, wired to extension `tool_result`, and apply its partial result patch.
9. Emit `tool_execution_end` and append a `toolResult` message.

Agent core defaults `toolExecution` to `"parallel"` (`options.toolExecution ?? "parallel"` in `packages/agent/src/agent.ts`). `Coding-agent's new Agent({...})` in `core/sdk.ts` does not pass `toolExecution`, so that default remains. A caller of agent core can select sequential execution, and a tool declaring `executionMode: "sequential"` makes its containing batch sequential. Otherwise, preflight happens in assistant source order, successful tool executions may run concurrently, and final

`toolResult` messages are emitted in assistant source order. Start/update/end event ordering differs in parallel mode; see the extension documentation before using sibling tool results as state.

Built-in tools are supplied by `coding-agent`. They are ordinary tools at this boundary, which is why an extension can add a tool or override a built-in one. Custom filesystem-mutating tools must participate in `withFileMutationQueue()` to avoid lost writes when sibling tools run in parallel.

4.10 Stage 8: Results create the next agent action

Each final tool result is appended to the in-memory agent context and becomes the model-facing material for the next turn. If at least one tool call was executed and the batch was not explicitly terminating, the loop starts another assistant response. If the model instead returns ordinary final text, the loop checks steering messages first and follow-up messages when it would otherwise stop.

```
user/custom context -> model -> assistant tool calls
                        |
                        v
                    validated tool results
                        |
                        +----> model again
```

A tool can return `terminate: true` to hint that the next model call should be skipped, but the batch terminates only when every finalized tool result sets that flag. A tool signals failure by throwing; merely returning a field named `isError` does not mark it failed.

4.11 Stage 9: Persistence and transcript updates settle the turn

As message events complete, `AgentSession` records them through its `SessionManager`. The session is a JSONL tree, so the persisted form can later rebuild a branch and compact context. Model and thinking-level changes are also session entries. Extension state added with `appendEntry()` is persisted as a custom entry but remains outside LLM context.

`InteractiveMode.handleEvent()` responds to the same event stream. Assistant updates request transcript updates for the streaming message; tool lifecycle events manage pending tool rows; finalized messages become permanent transcript components. Renderer registration lets an extension replace the display of its custom messages, entries, or tool calls/results without changing how the model loop works. This chapter stops at that event-to-component boundary; the title intentionally does not promise terminal rendering internals.

4.12 Stage 10: Errors, cancellation, retries, compaction, and shutdown

There are several deliberately different failure paths:

Situation	What happens
Provider/model run throws	<code>Agent.runWithLifecycle()</code> turns it into a final assistant message with <code>stopReason: "error"</code> or <code>"aborted"</code> , then emits normal terminal events.
Provider stream reports error/aborted	<code>agent-loop.ts</code> ends the turn without executing tools.
Tool schema invalid, unknown tool, hook blocks, or execute throws	The loop creates an error tool result and gives that result to the model.
User presses Esc / abort is requested	The active <code>AbortController</code> is aborted; abort-aware provider/tool/extension work should receive the signal.
Retryable assistant error	<code>AgentSession</code> may prepare an automatic retry after the low-level run ends.
Context pressure	<code>AgentSession</code> may compact, write a summary entry, rebuild context, and continue/retry.
Session replacement or process exit	The extension runtime receives <code>session_shutdown</code> ; session-bound resources must be cleaned up.

`agent_end` means the current low-level loop stopped. `agent_settled` is later: it means Pi has no automatic retry, compaction retry, or queued continuation left. Status integrations and cleanup that require true idleness should use `agent_settled` or `ctx.waitForIdle()` in a command context.

4.13 Trace this yourself

All source paths in this chapter are relative to the repository root. For a live trace, start at these symbols in order:

1. `InteractiveMode.run()` and `getUserInput()`.
2. `AgentSession.prompt()` and `_runAgentPrompt()`.
3. `Agent.prompt()`, `createLoopConfig()`, and `processEvents()`.
4. `runAgentLoop()`, `streamAssistantResponse()`, and `executeToolCalls()`.
5. `streamSimple()` in `packages/ai/src/compat.ts`.
6. The `streamSimple` function in the selected provider API adapter.

Use the extension event timeline in `packages/coding-agent/docs/extensions.md` as a second view of the same lifecycle. It documents what extension authors can rely on; the source symbols above show which layer makes each guarantee.

5 Agent core: runs, events, queues, and tool batches

`packages/agent` is the reusable runtime beneath the coding agent. It knows about a model, a system prompt, messages, tools, streaming, queues, and cancellation. It deliberately does not know about JSONL sessions, themes, project trust, extensions, or built-in filesystem tools.

The central source files are `packages/agent/src/agent.ts`, `src/agent-loop.ts`, and `src/types.ts`.

5.1 The smallest useful mental model

An **agent run** repeats this cycle: send context to a model, receive streamed events, execute any requested tools, append the results, and either stop or send another turn. The **Agent** class owns changing state around that cycle; `agent-loop.ts` performs the cycle; `types.ts` defines the records exchanged.

Keep two cases in mind. A text-only answer finishes after one provider stream. A tool-using answer returns to the provider after each tool batch. The rest of the chapter explains the events and queues that make both cases observable.

5.2 The three layers inside the package

Layer	Main file	Responsibility
Data contracts	<code>types.ts</code>	Tool/message/event/loop configuration types
Stateful controller	<code>agent.ts</code>	Current state, queues, aborting, listeners, lifecycle
Pure-ish loop orchestration	<code>agent-loop.ts</code>	Provider turn, event emission, tool batches, continuation

This split is useful when debugging. If state appears stale, start at **Agent**. If a provider turn or tool batch sequence is wrong, start at `agent-loop`. If the caller cannot supply a valid hook/tool/message shape, start in `types.ts`.

5.3 Agent state and snapshots

The `Agent` holds state including:

- current system prompt, model, thinking level, and active tools
- durable messages accumulated in this runtime
- streaming message and pending tool IDs for UI observers
- active run/abort controller
- steering and follow-up message queues
- event listeners and optional caller hooks

Before a prompt starts, `createContextSnapshot()` copies the `messages` and `tools` arrays with `slice()` and captures the current system-prompt string. This is an array snapshot, not a deep clone: the message objects, their nested content blocks, and the tool definition objects remain shared with agent state.

Array-level changes on either side stay isolated. After the run starts, replacing, pushing into, or reordering the agent state arrays does not alter the current loop context, and the same is true in reverse for the snapshot arrays the loop holds. Mutating an object already inside either array can still affect both views, because those objects are shared by reference. Treat messages and tool definitions as immutable once installed if a run may be active.

The model and thinking level are captured separately in the loop config. The thinking level is copied as a primitive; the model object, like message and tool objects, is not cloned. The loop owns its current context object and appends new messages to its private message array as it emits events; `Agent.processEvents()` appends completed messages to the state transcript. A later `prepareNextTurn` update can explicitly replace the context, model, or thinking level between provider turns.

5.4 Starting a prompt versus continuing a context

When idle, `Agent.prompt()` normalizes input to one or more `AgentMessage` values, then calls `runAgentLoop()` with the snapshot. It does not start a second run when one is active. Because `prompt()` is async, the call returns a rejected promise:

`Agent is already processing a prompt. Use steer() or followUp() to queue messages, or wait for completion.`

The rejection occurs before normalization, event emission, or transcript mutation, and it does not abort or otherwise change the existing run. `Agent.continue()` rejects concurrently in the same way with `Agent is already processing. Wait for completion before continuing.` Callers must await the call (or otherwise handle its promise), not expect a synchronous throw. Queue with `steer()` or `followUp()`, or await `waitForIdle()`, instead of retrying concurrently.

When idle, `Agent.continue()` resumes from the current transcript without appending a new caller message. Its default rule is that the last transcript message must not be an assistant message: continuation expects a user/tool-result tail that still needs a model response. There is one exception for queued work that arrived while the previous run was finishing:

1. If the transcript ends in an assistant message, `continue()` first drains the steering queue. Any drained steering messages are run as prompt messages (without an extra initial steering poll).
2. If steering is empty, it drains the follow-up queue and runs those messages the same way.
3. If both queues are empty, it rejects with `Cannot continue from message role: assistant`.

An empty transcript rejects with `No messages to continue from`. A non-assistant tail proceeds through `runContinuation()` and asks the model again from the existing context snapshot.

`prompt()` versus `continue()` exists to prevent accidental replay of a finished turn. Calling `continue()` after a normal assistant stop would re-issue the same trailing context to the provider without a new user or tool-result message. That is usually a bug (double-send / empty re-prompt). Callers that truly need another model call must first change the transcript tail or deliver queued steering or follow-up content.

<code>prompt(messages)</code>	adds new user/custom messages and begins a run
<code>continue()</code>	asks the model again from existing context
<code>steer(message)</code>	queues content between assistant turns
<code>followUp(message)</code>	queues content after the agent would otherwise stop

In the interactive coding-agent UI, Enter while a run is active maps to steer and Alt+Enter maps to follow-up; details are under Termination, steering, and follow-up below.

Coding-agent uses `continue()` deliberately. It starts with a user message, then may call `continue()` after automatic retry/compaction preparation or after messages were queued by a post-run extension event.

5.5 Active-run lifecycle and cancellation

`runWithLifecycle()` owns one `AbortController` per run. It marks the state as streaming, calls the loop, converts an unexpected thrown failure into an assistant error/aborted message sequence, then clears the active run.

`Agent.abort()` aborts that controller. The signal is passed to the provider stream and tool execution. A tool/extension that starts child work must forward the signal itself; abort does not magically cancel a detached fetch/process whose API was never given the signal.

`waitForIdle()` resolves after the active low-level run and its awaited event listeners finish. In coding-agent, `AgentSession` may then automatically retry, compact and retry, or start a queued continuation. `AgentSession` emits `agent_settled` once after that session-level chain has ended and no such automatic work remains. `agent_settled` is a coding-agent extension/session event, not an event in the `packages/agent AgentEvent` union. Extension authors who need Pi as a whole to be idle should observe `agent_settled`; see [Extension event reference: `agent\_start`, `agent\_end`, and `agent\_settled`](#).

5.6 Event stream as the observable protocol

The loop emits events rather than updating a UI directly. Important event families are:

Events	Meaning
<code>agent_start, agent_end</code>	One low-level run begins/ends
<code>turn_start, turn_end</code>	A provider response plus any tool batch
<code>message_start, message_update, message_end</code>	User, assistant, and tool-result message lifetime
<code>tool_execution_start,</code> <code>tool_execution_update, tool_execution_end</code>	Tool row/process lifetime

`Agent.processEvents()` first updates internal state, then awaits registered listeners. This ordering means a listener sees a coherent agent state through the event it is handling. Coding-agent relies on that property when syncing session entries and calling extension hooks.

5.7 One provider turn in detail

`streamAssistantResponse()` is the provider boundary in `agent-loop.ts`.

```
AgentMessage[] context
-> optional transformContext
-> convertToLlm
-> provider Context { systemPrompt, messages, tools }
-> streamFn(model, context, options)
-> for await normalized provider events
-> partial/final AssistantMessage plus message events
```

The low-level loop does not assume a provider has the same message shape as an agent message. `convertToLlm` belongs to the caller precisely because a coding agent may have session-only messages while another embedding has a different message representation.

On a provider `start` event, the loop pushes a partial assistant message into its current context and emits `message_start`. Text, thinking, and tool-call deltas replace that partial state and emit `message_update`. A final `done` or `error` resolves the stream result and emits `message_end`.

5.8 Tool argument preparation and validation

After a final assistant message, the loop finds content blocks with `type: "toolCall"`. It resolves the requested tool from active context, calls optional `prepareArguments`, then validates the result against the tool schema.

`prepareArguments` exists for session compatibility. An extension/tool can adapt legacy stored call shapes into its current strict schema before validation without weakening the public parameters declaration. It is not a general way to accept arbitrary invalid input.

An unknown tool, invalid schema, hook failure, or explicit block becomes an immediate error result rather than a crash. The model receives that error in a tool-result message and can decide how to recover.

5.9 Sequential and parallel tool execution

`ToolExecutionMode` has two choices:

Mode	Behavior
<code>sequential</code>	Prepare, execute, finalize, and persist each tool before the next
<code>parallel</code>	Emit each start event and preflight each call in assistant-content order; after all preflights, run prepared tools concurrently; emit updates and end events as work progresses; after all calls finish, emit tool-result messages in assistant-content order

The `Agent` default is `parallel`. The ordering contract is direct: in a parallel batch, `tool_execution_start` and argument preparation occur in source order; prepared `execute()` functions start only after the preflight pass; `tool_execution_update` and `tool_execution_end` can interleave and end in completion order; `message_start/message_end` for the final `toolResult` messages wait for the whole batch and then occur in source order. An immediate preflight error can emit its end event during the ordered preflight pass, before prepared siblings begin execution.

The effective mode is batch-wide. The batch is sequential if `agent.toolExecution === "sequential"` or any tool called by that assistant message has `executionMode: "sequential"` in the active context. It is parallel only when the agent mode is `"parallel"` and none of those called tools requires sequential execution. A tool marked `"parallel"` does not override a global or sibling sequential requirement.

To inspect this behavior, first record `agent.toolExecution` and the `executionMode` fields of the active tool definitions at run start and after any `prepareNextTurn` refresh, then match those definitions to the tool calls in the assistant message. Subscribe to `tool_execution_start`, `_update`, and `_end` to observe lifecycle ordering; `agent.state.pendingToolCalls` contains the IDs between start and end. A start event and pending ID include preflight, so they do not prove that the tool's `execute()` body ran—instrument the tool wrapper or `execute()` entry when that distinction matters.

Parallel execution means a tool hook cannot assume a sibling's result has finished or been persisted. Use explicit coordination or `turn_end` state when tools depend on one another.

5.10 Before and after tool hooks

The generic loop accepts two caller callbacks:

- `beforeToolCall` receives assistant message, tool call, validated args, and current context. It may block with a reason.
- `afterToolCall` receives the executed result/error state and may replace content, details, error flag, or terminate hint.

Coding-agent connects these callbacks to `ExtensionRunner.emitToolCall` and `emitToolResult`. That bridge keeps the reusable agent package free of extension types while allowing extension hooks to participate in execution.

5.11 Termination, steering, and follow-up

When a tool batch completes, the loop creates final `toolResult` messages and appends them to context. Unless every result has `terminate: true`, it performs another provider turn. The all-results rule avoids one tool silently stopping a batch whose other tools expect a model follow-up.

After each completed turn, the loop asks for steering messages. Steering messages enter before the next provider request. When there are no more tool calls and no steering messages, it asks for follow-up messages. Follow-ups restart the inner loop only after the agent would otherwise stop.

```
assistant/tool batch complete
-> steering queue? yes: inject before next model call
-> no tools/steering: follow-up queue? yes: begin another turn
-> neither: agent_end
```

In interactive mode, these queues are selected by the submit key while Pi is working:

- **Enter** submits with `streamingBehavior: "steer"`. The message waits for the current assistant response and its entire tool batch to finish, then enters before the next provider request. It does not interrupt a tool midway.
- **Alt+Enter** submits with `streamingBehavior: "followUp"`. The message is not considered until the inner tool/steering loop would otherwise stop, so already queued steering and the work it causes run first.

When Pi is idle, both keys perform a normal submission; Alt+Enter does not create a deferred message. Queue drain settings can deliver either the oldest message only ("`one-at-a-time`", the default) or all messages at a drain point ("`all`"). During compaction the interactive layer preserves the same steering-versus-follow-up classification and delivers the queued input when compaction permits.

5.12 Next-turn refresh hook

The loop can call `prepareNextTurn` after `turn_end`. It may replace context, model, or thinking level for the next provider call. Coding-agent uses this to refresh the current system prompt and active tool list from session state.

This matters when an extension dynamically changes tools or model state during a run. The current turn remains internally consistent; the next provider call sees the refreshed session snapshot rather than a stale configuration.

5.13 Error semantics are intentionally normalized

The `StreamFn` contract says ordinary request/model/runtime failures should be represented by a returned assistant event stream whose final message has `stopReason: "error"` or "`aborted`", rather than a rejected promise. The agent also protects itself by converting unexpected throws into a final assistant error message.

Tool failures are different: `execute` throws become an error `toolResult`, so a model can read output/error and choose a recovery. This difference is important for tests and UI: a provider failure ends the run; a tool failure usually lets the model continue.

5.14 Embedding the agent package

If you use `packages/agent` outside `coding-agent`, you must provide policy that `coding-agent` normally supplies:

- initial `AgentState` with a valid model/system prompt/tools
- `convertToLlm` for your message representation
- provider `streamFn` and credentials or key resolver
- listeners that persist/render state if needed
- context transformation/queue behavior appropriate to your application
- tool definitions and any permission gate

The package gives a rigorous loop, not a complete coding application. That is the reason Pi can support the interactive CLI, SDK examples, and other hosts without coupling all of them to terminal/session decisions.

5.15 Agent-core debugging checklist

When a turn misbehaves, capture these facts before editing code:

1. What did `convertToLlm` produce?
2. What system prompt and active tools were in the context snapshot?
3. Which normalized provider events arrived, and was a final message emitted?
4. Did the assistant end with tool calls, error, aborted, length, or stop?
5. Which tool arguments passed validation/preparation?
6. Was the batch sequential or parallel, and what event ordering occurred?
7. Did steering/follow-up queues inject another message?
8. Did a `prepareNextTurn` refresh change model/tools/context?

This reduces a complex agent issue to a small number of observable protocol boundaries.

6 Built-in tools: filesystem, shell, and discovery

Pi ships seven named tools: `read`, `bash`, `edit`, `write`, `grep`, `find`, and `ls`. Together they cover filesystem access and mutation, shell execution, and bounded repository discovery. Each built-in is a tool definition with a TypeBox parameter schema that Pi wraps as an `AgentTool`, placing built-ins and extension-provided tools in the same lifecycle.

Implementation lives in `packages/coding-agent/src/core/tools/`. The public construction helpers are re-exported from the `@earendil-works/pi-coding-agent` package. The chapter focuses on behavior that matters when using, extending, or debugging the tools rather than teaching shell commands.

6.1 Start with the tool contract

Before reading `read` or `bash`, learn the shared shape: a **schema** describes valid inputs, a **description** tells the model when to use the tool, an **execute** function performs the work, and optional **renderers** show the call and result in the TUI. For example, both `read` and a custom `listTodos` tool need validated input and an actionable result, even though one uses the local filesystem and the other may use a database.

The chapter teaches the contract first, then compares read-only work with mutating work and short commands with streaming commands. That order makes the queue, truncation, and cancellation rules easier to recognize in each tool.

6.2 Tool registry and default selection

`core/tools/index.ts` is the tool factory map. It defines the `ToolName` union and dispatches each name to a `create*ToolDefinition` or `create*Tool` helper.

```
tool definition: schema + description + execute + optional renderers
    |
    v
wrapToolDefinition(...)
    |
    v
@earendil-works/pi-agent-core AgentTool: validated arguments + callback
```

In prose, Pi combines a schema, description, execution function, and optional renderers, then wraps that definition as the validated `AgentTool` consumed by the agent loop.

Pi's SDK creates all definitions so extensions can inspect their metadata, but the initial active set contains the four coding tools: `read`, `bash`, `edit`, and `write`. `grep`, `find`, and `ls` remain available

definitions outside that default active list. Explicit tool selection, `--no-builtin-tools`, extensions, or `pi.setActiveTools()` can change what the model receives.

6.3 Shared design principles

Every built-in tool follows the same broad contract:

1. A `TypeBox` schema validates the model's arguments in `agent-loop.ts`.
2. `execute` receives an ID, parsed params, an optional abort signal, and an optional incremental-update callback.
3. Success returns model-facing content plus optional structured `details`.
4. Failure throws; the agent loop converts it into an error tool result.
5. Optional renderers turn call/result data into a compact TUI row.

Tool descriptions and prompt metadata directly shape model behavior. `promptGuidelines` is an optional list of instruction bullets on a tool definition. When rebuilding the system prompt, Pi includes the snippets and `promptGuidelines` from active tools. For example, `read` tells the model to inspect files with `read` rather than `cat` or `sed`; `write` reserves full-file output for new files or complete rewrites.

6.4 Output limits protect the next provider turn

Large tool output can consume the same context window needed for reasoning and response. `truncate.ts` defines a shared 50 KB / 2,000 line limit, along with head/tail/line helpers and metadata describing why output was truncated.

The correct truncation direction depends on the tool:

Tool use	Useful part	Typical policy
File content or search result	Beginning/ordered matches	Keep head
Long-running command log	Latest state/error	Keep tail
Individual match line	Prefix is usually enough	Truncate line

Built-in tools report truncation to the model and provide an actionable next step. Custom tools should pair every truncated result with the same explicit status and recovery guidance so the model can continue without mistaking partial output for a complete result.

6.5 `read`: text ranges and image attachments

`read.ts` accepts a required `path` plus optional 1-indexed `offset` and `limit`. It resolves the path relative to the session cwd, checks readability, then branches on MIME detection.

For text files, `read` splits lines, applies the requested range, and then applies shared truncation. The continuation messages are deliberately specific:

```
Showing lines 1-2000 of 6500. Use offset=2001 to continue.
```

If a single first line exceeds the byte limit, the `read` result reports that condition and suggests a bounded shell fallback. The explicit notice accurately marks the line as partial.

For supported images (JPEG, PNG, GIF, WebP, BMP), `read` loads the binary data, processes and resizes the image subject to settings, and returns an image content block alongside a text note. The tool checks the current model’s input capabilities and adds a note when the model cannot receive images. The global `images.blockImages` setting adds another later defense in the conversion path.

6.5.1 Read operations are pluggable

`ReadOperations` has `readFile`, `access`, and optional MIME detection. The default uses the local filesystem. An extension can build a read tool backed by SSH, a container, or another remote system by providing those operations. The schema and TUI behavior stay the same; only the I/O boundary changes.

Providing `ReadOperations` preserves the existing schema and TUI behavior while redirecting filesystem access, avoiding a duplicate read implementation.

6.6 write: create or replace a complete file

`write.ts` accepts `path` and full `content`. It creates parent directories recursively, overwrites existing content, and returns a byte count. Its prompt guideline explicitly reserves it for new files or complete rewrites.

The implementation wraps the complete operation in `withFileMutationQueue`:

```
resolve target path
-> wait for same-file queue
-> check abort
-> mkdir parent
-> check abort
-> write file
-> check abort
-> release queue
```

The queue is keyed by a resolved, canonical path. Existing paths are passed through `realpath`, so symlink aliases share a queue; missing target paths use the resolved absolute path. Different files still execute in parallel.

The sequence above checks abort three times—before `mkdir`, after `mkdir`, and after `writeFile`—and holds the same-file queue until each in-flight filesystem operation settles. A custom mutating tool should preserve this ordering so the next writer starts from settled filesystem state.

6.7 edit: exact replacement and queued mutation

`edit.ts` is for targeted changes to an existing file. Its schema is built around a path and edit blocks with exact old/new text. The exact-match approach lets Pi detect ambiguous or stale replacements rather than applying a fuzzy patch to the wrong location.

Like `write`, `edit` resolves the target and wraps the full read-modify-write window in `withFileMutationQueue`. If a custom read-modify-write tool queues only its final write, two parallel calls can both read old content, derive conflicting replacements, and lose one change. The lock must cover reading, validation, transformation, and writing.

Use `edit` rather than `write` when preserving unrelated content is important. Use `write` when creating a file or deliberately replacing its entirety. The model prompt guidelines encode this distinction to reduce destructive rewrites.

6.8 bash: process execution with streamed tail output

`bash.ts` accepts a command and optional timeout in seconds. It uses `createLocalBashOperations()` by default, which resolves the configured shell, verifies the cwd, and starts a child process with `stdout/stderr` pipes. On non-Windows systems the child starts as a detached process group. Cancellation and timeout call `killProcessTree`, which invokes `taskkill /F /T` on Windows; on non-Windows systems it sends `SIGKILL` to the process group and falls back to killing the child process.

The tool can apply a configured `shellCommandPrefix` and an extension-provided `spawnHook` that changes the command, cwd, or environment. These are command customization and routing hooks, not security boundaries. For enforced containment, provide `BashOperations` whose `exec` implementation enters the sandbox or remote environment, and replace the filesystem tools' operations so `read`, `write`, `edit`, `grep`, `find`, and `ls` use the same boundary. Apply the boundary to `user_bash` as well. A `tool_call` hook can add policy checks, but operating-system or container controls must enforce isolation.

6.8.1 Streaming output

`OutputAccumulator` collects `stdout` and `stderr`. While a process runs, `bash` throttles partial `onUpdate` events to roughly one every 100 ms. The TUI can show a live preview while the model waits for final output.

Final command output keeps the tail. If output was truncated, the tool saves the full output to a temporary file and includes that path in `details` and a model-facing notice. Nonzero exit status, timeout, and abort become thrown errors with accumulated output appended where available. The agent loop then returns that failure as an error tool result to the model.

A failed command therefore remains actionable: the model receives the log and exit status along with the error instead of only an exception string.

6.9 grep, find, and ls: bounded repository discovery

These search/listing tools use the cwd and respect ignore policy. Their schemas make the intended scope explicit:

Tool	Important inputs	Default limit	Result shape
grep	pattern, optional path/glob, literal/case/context options	100 matches	File path and line-numbered matching lines
find	glob pattern, optional path	1,000 results	Matching paths relative to search directory
ls	optional directory path	500 entries	Alphabetical listing, directory suffix /, includes dotfiles

All three also use the shared byte cap. **grep** additionally limits long output lines. The tools are better suited to agent context than raw **rg/find** shell output because their limits and formatting are predictable.

Pi exposes all three as built-in names that an extension can register and activate. When overriding one, retain its result details and renderer contract unless the extension also supplies every affected display and session behavior.

6.10 Paths, cwd, and remote operations

Tools must distinguish what the model typed from the actual target path. The built-ins resolve relative paths against the session cwd, not an arbitrary process cwd, and format paths for display relative to the session when useful.

Remote execution can preserve structured tool calls. The exported operations interfaces make each tool's I/O replaceable:

```
ReadOperations    readFile + access + optional MIME detection
WriteOperations   mkdir + writeFile
EditOperations    readFile + writeFile + access
BashOperations    exec with data callback, cwd, signal, timeout, env
Grep/Find/Ls      operation interfaces for filesystem/search behavior
```

Each line names the boundary an extension replaces: file tools receive focused filesystem methods, while **bash** receives an execution method with streaming and cancellation inputs.

An extension can create a tool with remote operations, register it under the same name, and preserve Pi's validation and rendering behavior. The extension documentation's SSH and sandbox examples demonstrate this pattern.

6.11 User shell commands are separate from model bash

In interactive mode, input beginning with `!` runs a user shell command; `!!` marks its result excluded from LLM context. The extension `user_bash` event can provide custom operations or a complete result for the user-command path.

User shell commands and assistant `bash` calls follow distinct paths. The user directly requests a `!` command, while the model generates a tool call that Pi records in the assistant/tool-result protocol. A sandbox extension should cover both paths by supplying sandboxed operations for built-in tools and `user_bash`.

6.12 Tool rendering is a first-class interface

Built-in definitions include `renderCall` and `renderResult` functions. The interactive tool row receives current arguments, partial/final state, expanded flag, prior component, execution time, and an `invalidate()` callback. Those inputs support syntax-highlighted read/write previews, compact default rendering with an expand hint, streamed shell output, and error-specific presentation.

An extension that overrides built-in execution can omit a renderer and inherit the built-in renderer per slot. Renderer inheritance requires the expected call argument and result-detail shapes. When an override changes either shape, provide compatible renderers or a self-contained rendering shell.

6.13 Tool lifecycle and extension interception

The agent loop emits this sequence for each tool call:

```
tool_execution_start
-> schema validation and tool_call extension gate
-> execute / tool_execution_update ...
-> tool_result extension patch
-> tool_execution_end
-> final toolResult message events
```

In prose, Pi validates and gates a call before execution, emits zero or more updates while it runs, allows extensions to patch the result, then emits the end event and final protocol messages.

In default parallel mode, start/preflight is source-ordered, execution/update events can interleave, and final tool-result messages are emitted in assistant source order. Custom tools should coordinate through explicit shared state or lifecycle events instead of expecting an earlier sibling tool result to already appear in the extension context's `ctx.sessionManager` (`ReadOnlySessionManager` over the durable session tree) during a concurrent `tool_call` handler.

6.14 A safe custom file-mutation skeleton

The following focused example shows the required same-file mutation ordering. A production tool also needs the schema, result, rendering, and test concerns listed after the code.

```
import { withFileMutationQueue } from "@earendil-works/pi-coding-agent";
import { readFile, writeFile } from "node:fs/promises";
import { resolve } from "node:path";

async function replaceOnce(
  cwd: string,
  path: string,
  oldText: string,
  newText: string,
  signal?: AbortSignal,
) {
  const target = resolve(cwd, path);

  // Keep read -> validate -> transform -> write inside the same-file queue.
  return withFileMutationQueue(target, async () => {
    if (signal?.aborted) throw new Error("Operation aborted");

    const current = await readFile(target, "utf8");
    if (signal?.aborted) throw new Error("Operation aborted");

    if (oldText.length === 0) throw new Error("Expected text must not be empty");
    const match = current.indexOf(oldText);
    if (match === -1) throw new Error("Expected text was not found");
    if (current.indexOf(oldText, match + oldText.length) !== -1) {
      throw new Error("Expected text must occur exactly once");
    }

    const updated =
      current.slice(0, match) + newText + current.slice(match + oldText.length);
    if (signal?.aborted) throw new Error("Operation aborted");

    await writeFile(target, updated, "utf8");
    if (signal?.aborted) throw new Error("Operation aborted");
  });
}
```

In a real tool, add a TypeBox schema, precise description, result content, truncation where relevant, and tests for no-match, abort, and concurrent calls.

6.15 Tool-author review questions

Before exposing a custom tool to a model, answer:

1. Does the TypeBox schema allow only safe inputs?
2. Is the tool active only when its prompt guidance is appropriate?
3. Does every large output have a bounded, actionable representation?
4. Does cancellation reach all long-running work?
5. Are writes serialized across aliases of the same target?
6. Does a thrown error give the model enough detail to recover?
7. Are **details** durable and renderer-compatible?
8. Does the behavior work in parallel with built-in tools?
9. Is a policy gate better implemented as `tool_call` interception than inside the tool itself?

Treat tools as a public protocol connecting the model, session, UI, and extension ecosystem.

7 Extensions: change Pi without forking it

An extension is a TypeScript or JavaScript module that Pi loads at runtime with `jiti`. Jiti resolves the module and transpiles TypeScript syntax on demand, so an extension does not need a separate build step; it does not bundle or type-check the code. The module's default export is a factory receiving `ExtensionAPI`. The factory registers hooks, tools, commands, shortcuts, flags, renderers, and providers, which Pi invokes at defined lifecycle boundaries.

Extensions execute with the permissions of the Pi process. They are trusted code, not a sandbox. Review a package before installing it and treat an extension's filesystem, process, network, and credential access as you would a normal local program.

7.1 Learning path and first vocabulary

An **extension** is code Pi loads at runtime. A **hook** is a named event where that code can observe or change a supported part of the lifecycle. A **tool** is a model-callable function with a validated input shape. A **context** is the object passed to a handler, including the session, cancellation signal, and possible UI.

Begin with the smallest useful extension: one command that displays a notice. Then compare it with a tool, a hook, and durable state. The examples later in the chapter are larger because each adds one of those responsibilities; do not copy the package example until the one-file version is clear.

7.2 Choose the right customization mechanism

Before writing code, use the smallest mechanism that owns the behavior:

Need	Prefer
Persist an instruction for an AI model or reusable task procedure	AGENTS.md , a skill, or a prompt template
Change colors and visual style	Theme
Change AI model/provider selection or static behavior	Settings or a custom provider
Add a command, tool, lifecycle gate, state, custom UI, or request hook	Extension
Reuse Pi from another Node program	Coding-agent SDK (<code>createAgentSession</code>)
Change generic loop/provider/TUI behavior for every consumer	Core contribution in the owning package

An extension is powerful, but do not use one to recreate or patch Pi internals as an unreviewed fork. Favor public `ExtensionAPI` methods over importing coding-agent internals.

7.3 Discovery, trust, and loading

Pi automatically discovers a single-file extension or directory `index.ts` in these places:

Scope	Files discovered
Global	<code>~/.pi/agent/extensions/*.ts</code> , <code>~/.pi/agent/extensions/*/index.ts</code>
Project	<code>.pi/extensions/*.ts</code> , <code>.pi/extensions/*/index.ts</code>

Project-local resources are loaded only after the project is trusted. You can also add local paths in `settings.json`, or try a path for one invocation with `pi -e ./path/to/extension.ts`. The latter is ideal for a quick test, but `/reload` hot reload is intended for auto-discovered locations.

At startup, the resource loader loads extension factories before session start. An async factory is awaited before startup continues. That makes it suitable for one-time initialization such as discovering available AI models and calling `pi.registerProvider()`. Do not start timers, watchers, processes, or sockets in the factory. For example, this command loads extension factories to discover available AI models from providers but does not start a session:

```
pi --list-models
```

Start long-lived session resources in `session_start` and close them idempotently in `session_shutdown`.

7.4 The lifecycle you can rely on

```
factory runs
-> project_trust (eligible global/CLI extensions only)
-> session_start
-> resources_discover

prompt
-> extension commands, then input
-> before_agent_start -> agent_start
-> for each AI model/tool turn:
    turn_start -> context -> provider hooks -> streamed messages
    -> tool execution hooks -> turn_end
-> agent_end -> agent_settled

reload/switch/fork/quit -> session_shutdown
```

Figure 4.1 — Representative extension event flow (not exhaustive).

The diagram omits message-level events and detailed branches for session operations; the table below includes some representative events omitted from the diagram. For the complete event sequence, return values, and parallel-tool ordering, see `packages/coding-agent/docs/extensions.md`; use `packages/coding-agent/src/core/extensions/types.ts` for the exact types.

7.4.1 Event groups and common use

Event group	Representative events	Use it for
Trust/resources	<code>project_trust</code> , <code>resources_discover</code>	Decide project trust or contribute paths for skills/prompts/themes
Session	<code>session_start</code> , <code>session_shutdown</code> , <code>session_before_switch</code> , <code>session_before_compact</code>	Initialize/clean resources, persist/rebuild state, gate a session operation
Input/agent	<code>input</code> , <code>before_agent_start</code> , <code>agent_start</code> , <code>agent_end</code> , <code>agent_settled</code>	Transform input, add prompt context, track the true idle state
Turn/messages	<code>turn_start</code> , <code>context</code> , <code>message_update</code> , <code>turn_end</code>	Adjust context or render/measure streaming activity
Provider	<code>before_provider_headers</code> , <code>before_provider_request</code> , <code>after_provider_response</code>	Tracing, a request rewrite, response metadata
Tool	<code>tool_execution_start</code> , <code>tool_call</code> , <code>tool_execution_update</code> , <code>tool_result</code> , <code>tool_execution_end</code>	Observe progress, block/mutate calls, patch results
AI model	<code>model_select</code> , <code>thinking_level_select</code>	Update UI/state for the selected AI model
User shell	<code>user_bash</code>	Route ! and !! through a different execution backend

`tool_call` is a policy enforcement point: it runs after schema validation and can block a tool.

Its input is mutable. A change affects execution and is visible to later `tool_call` handlers. **Pi does not revalidate input after a handler mutates it.** Keep mutations narrow and validate every invariant your change could violate.

`context` receives a deep copy of messages that an extension may replace. It is a safer place to prune/inject LLM context than modifying session storage. `before_provider_request` is lower-level still: it sees the provider-specific payload after serialization, so changes there are not reflected by `ctx.getSystemPrompt()`.

7.5 A complete first extension

This extension adds a `hello` tool callable by an AI model, a `/hello-pi` command, and a startup notification. It intentionally has no dependencies beyond Pi's bundled extension APIs.

1. Create the file in an auto-discovered global location:

```
mkdir -p ~/.pi/agent/extensions
```

2. Save this as ~/.pi/agent/extensions/hello-pi.ts:

```
import { Type } from "@earendil-works/pi-ai";
import { defineTool, type ExtensionAPI } from "@earendil-works/pi-coding-agent";

const helloTool = defineTool({
  name: "hello",
  label: "Hello",
  description: "Greet a person by name",
  parameters: Type.Object({
    name: Type.String({ description: "The person to greet" }),
  }),
  async execute(_toolCallId, params, _signal, _onUpdate, _ctx) {
    return {
      content: [{ type: "text", text: `Hello, ${params.name}!` }],
      details: { greeted: params.name },
    };
  },
});

export default function register(pi: ExtensionAPI) {
  pi.registerTool(helloTool);

  pi.registerCommand("hello-pi", {
    description: "Show a greeting notification",
    handler: async (args, ctx) => {
      ctx.ui.notify(`Hello, ${args || "world"}!`, "info");
    },
  });

  pi.on("session_start", async (_event, ctx) => {
    ctx.ui.notify("hello-pi extension is ready", "info");
  });
}
```

The tool definition uses `defineTool`, as does the repository's verified `packages/coding-agent/examples/extension`. `Type.Object` is a runtime schema. `defineTool` derives the static type of `params`, and the agent loop validates AI-model-supplied arguments against that schema before calling `execute`.

3. Start Pi in any project. The startup notification confirms discovery. Run `/hello-pi Ada` to exercise the command. Ask Pi to greet Ada with the `hello` tool, or inspect its active tool list, to exercise the interface exposed to the AI model.
4. Test as a user: edit the discovered file and run `/reload` in Pi. That path emits `session_shutdown`, reloads resources, then emits `session_start` with `reason: "reload"`.

Separate rule for extension authors: if a *command handler* itself calls `ctx.reload()`, end the handler there. Reloading tears down the current runtime and spins up a new one, but the old `await` resumes in the pre-reload call frame with the old `ctx` and closed-over state:

```
// Author responsibility: do not resume work on the pre-reload instance.
await ctx.reload();
return;
```

Put any post-reload behavior in the newly loaded factory (for example a `session_start` handler), not after `await ctx.reload()`.

For a one-off test outside discovery, use `pi -e ~/.pi/agent/extensions/hello-pi.ts`. The global file is preferable when testing `/reload`.

7.6 What ExtensionAPI and context provide

ExtensionAPI is the registration/control surface. Its important methods are:

API	Purpose
<code>pi.on()</code>	Subscribe to a typed lifecycle event
<code>pi.registerTool()</code>	Add or override a model-callable tool
<code>pi.registerCommand()</code>	Add <code>/name</code> command with optional argument completion
<code>pi.registerShortcut()</code> / <code>pi.registerFlag()</code>	Add a keybinding action or CLI option
<code>pi.sendMessage()</code> / <code>pi.sendUserMessage()</code>	Put a custom or user message into agent flow
<code>pi.appendEntry()</code>	Persist extension-only session data outside LLM context
<code>pi.registerMessageRenderer()</code> / <code>pi.registerEntryRenderer()</code>	Render custom message/entry types in the transcript
<code>pi.getActiveTools()</code> / <code>pi.setActiveTools()</code>	Inspect or change the currently exposed tool set
<code>pi.setModel()</code> / <code>pi.setThinkingLevel()</code>	Change live session AI-model settings
<code>pi.registerProvider()</code> / <code>pi.unregisterProvider()</code>	Register an endpoint/provider for AI models at runtime
<code>pi.exec()</code>	Run a process with Pi's helper interface

Handlers receive a context. The most useful fields are `ctx.cwd`, `ctx.mode`, `ctx.hasUI`, `ctx.sessionManager`, `ctx.modelRegistry`, `ctx.model`, `ctx.signal`, and `ctx.ui`. Pass `ctx.signal` to cancelable nested work. Check `ctx.mode === "tui"` for terminal-only behavior and `ctx.hasUI` before prompting.

`ExtensionCommandContext` adds session-changing methods such as `newSession`, `fork`, `navigateTree`, `switchSession`, `waitForIdle`, and `reload`. They are available only in command handlers because changing a session from an ordinary event handler can deadlock or leave stale state.

7.7 Tools: make the contract for the AI model explicit

A custom tool needs a unique `name`, human-facing `label`, description for the AI model, `TypeBox` `parameters`, and `execute`. The return `content` goes to the LLM; `details` is structured metadata for renderers and state recovery.

Add `promptSnippet` when the tool should have a one-line entry in Pi's default system prompt. Add `promptGuidelines` as instructions for the AI model while the tool is active. Each guideline must name its tool because Pi flattens all active tool guidelines into one section: write `Use hello` when the user asks for a greeting, not `Use this tool` when the user asks for a greeting.

For robust tools:

- Throw from `execute` to report an error to the model.
- Forward `signal` into `fetch`, process helpers, and long computations.
- Use `onUpdate` for incremental result rendering.
- Truncate large output with the exported truncation helpers; built-in limits are 50 KB or 2,000 lines.
- For string enums exposed to Google-compatible APIs, see `StringEnum` vs `Type.Union`.
- Use `withFileMutationQueue(absolutePath, callback)` around the *entire* read-modify-write window for a file-mutating tool.

An extension may override a built-in tool by registering the same name. That is an advanced compatibility commitment: preserve the built-in result `details` shape expected by session and UI code. If the override omits a renderer, Pi can reuse the built-in renderer slot, but prompt metadata is not inherited.

7.8 Understanding TypeBox schemas for tool parameters

A `TypeBox` schema validates model-supplied arguments at runtime and generates the JSON Schema sent to the provider. A runtime schema is therefore required even when TypeScript already knows the parameter types.

7.8.1 Common schema patterns

```
import { Type } from "@earendil-works/pi-ai";

// Required string parameter
Type.String({ description: "A description helps the model use it correctly" })

// Optional number
Type.Optional(Type.Number({ description: "Count in seconds" }))

// Required object with typed properties
Type.Object({
  path: Type.String({ description: "File path" }),
  recursive: Type.Optional(Type.Boolean({ default: false })),
});
```

```

})

// String enum (use StringEnum for Google-compatible APIs)
import { StringEnum } from "@earendil-works/pi-ai";
StringEnum(["small", "medium", "large"], { description: "Size" })

```

7.8.2 How to think about schemas as a JavaScript developer

A `TypeBox Type.Object({...})` resembles a machine-readable JSDoc `@param` block. It answers three questions:

1. **What shape of JSON does the model need to send?** (the schema itself)
2. **What TypeScript type does params have in execute?** (derived from schema)
3. **What does the model see in its tool description?** (serialized to JSON Schema)

If you define a parameter as `Type.String()` and the model sends 42, Pi rejects the call before your `execute` runs. This prevents malformed tool calls from reaching your code.

7.8.3 StringEnum vs Type.Union for string literals

For tools exposed to Google's Gemini API, use `StringEnum` from `@earendil-works/pi-ai` instead of `Type.Union([Type.Literal("a"), Type.Literal("b")])`. Google's API does not support the enum constraint produced by `Type.Union` with literals, so the model might suggest invalid values. `StringEnum` generates the correct JSON Schema form that all providers understand.

7.9 Commands, UI, and rendering

Commands are best for direct user actions and configuration. If multiple extensions register the same name, Pi keeps both and gives them numeric invocation suffixes in load order. Do not assume a command name is globally unique after loading third-party packages.

`ctx.ui` supports notifications, select/confirm/input/editor dialogs, footer status, widgets, editor text, themes, custom autocomplete, tool expansion, and replacement editors. For a multi-key interaction, `ctx.ui.custom()` temporarily replaces the editor with a `@earendil-works/pi-tui` component.

UI behavior depends on the mode. In TUI mode, `notify()` displays a fire-and-forget message, while dialogs wait for the user's response. In RPC mode, notifications emit requests that the client may display or ignore; dialogs emit requests and wait for matching client responses. In JSON and print modes, notifications are no-ops and dialogs return their cancellation defaults (`false` or `undefined`). Check `ctx.hasUI` before opening a dialog, and require `ctx.mode === "tui"` for components that need a terminal.

Use a message renderer for a custom message created with `pi.sendMessage()`; that content participates in LLM context. Use an entry renderer for a custom entry created with `pi.appendEntry()`; that is durable transcript/UI state but does not reach the LLM. This is the key display-versus-context choice.

7.10 State, sessions, and queues

Use this section to choose where extension state lives and how it schedules work around an active agent.

In-memory variables vanish on reload, resume, fork, and restart. Reconstruct state in `session_start` by examining the active branch. For tool-backed state, store a recoverable snapshot in that tool result's `details`; it follows the conversation branch naturally. Use `appendEntry()` for state that should not enter model context.

`pi.sendMessage()` injects a custom message into the session and accepts a `deliverAs` mode. The agent loop drains the queues at different points:

Mode	When it is applied	Effect on the active run
<code>steer</code> (default)	After the current assistant message and its tool batch finish, before the next AI-model request	Can re-enter the active multi-turn tool loop. Later steers and tool work still run before any <code>followUp</code> .
<code>followUp</code>	Only when the agent is about to stop: no remaining tool calls and no pending steers	Starts another turn after that work would otherwise end. Already-queued steers, and the tools they cause, run first.
<code>nextTurn</code>	With the <i>next</i> user-originated prompt	Never interrupts the active run and never starts one by itself.

While the agent is idle, `sendMessage()` with `deliverAs`: `"steer"` or `"followUp"` still only appends unless you also pass `triggerTurn`: `true`, which starts an AI-model turn immediately. `triggerTurn` is ignored for `nextTurn`.

`pi.sendUserMessage()` injects a genuine user message (not a custom one) and always starts or continues a turn:

- Idle: send immediately; `deliverAs` is not required.
- Streaming: you must pass `deliverAs`: `"steer"` or `"followUp"` (same queue timings as above). Omitting it throws. `nextTurn` is not available here.

Do not start a second direct agent run from a hook; use these queues.

7.11 Distribute an extension as a Pi package

For a reusable package, declare explicit resources in `package.json`:

```
{
  "name": "@acme/pi-team-tools",
  "keywords": ["pi-package"],
  "peerDependencies": {
    "@earendil-works/pi-ai": "*",
    "@earendil-works/pi-agent-core": "*"
  }
}
```

```

    "@earendil-works/pi-coding-agent": "*",
    "@earendil-works/pi-tui": "*",
    "typebox": "*"
  },
  "pi": {
    "extensions": ["/extensions/"],
    "skills": ["/skills/"],
    "prompts": ["/prompts/"],
    "themes": ["/themes/"]
  }
}

```

Put third-party runtime packages in **dependencies**, not **devDependencies**: Pi package installation uses production installs by default. Core Pi packages are bundled by Pi, so list each one the package actually imports as a **peerDependency** with the documented "*" range rather than bundling a competing copy. The broad manifest above assumes a full-featured package that imports all five bundled packages. Include `@earendil-works/pi-tui` only when code imports TUI components or TUI types for custom UI/rendering. Include `typebox` only when code imports schemas directly from `typebox` rather than using the `Type` re-export from `@earendil-works/pi-ai`. Package conventions also discover `extensions/`, `skills/`, `prompts/`, and `themes/` when a `pi` manifest is not present, but an explicit manifest is clearer and supports filtering.

Users can install a package from npm or git with `pi install`, or list it under the **packages** array in settings. Project-installed packages are trust-gated just like project-local extensions.

7.11.1 Step-by-step: from zero to a shareable extension package

The following example creates a minimal distributable package outside the Pi monorepo.

Step 1 – Initialize a package

```

mkdir pi-hello-demo
cd pi-hello-demo
npm init -y

```

Step 2 – Add the extension code

```

mkdir extensions

```

Create `extensions/index.ts` with the hello-pi extension from the earlier example in this chapter.

Step 3 – Configure `package.json`

Edit `package.json` to add:

```
{
  "name": "@your-name/pi-hello-demo",
  "keywords": ["pi-package"],
  "peerDependencies": {
    "@earendil-works/pi-ai": "*",
    "@earendil-works/pi-coding-agent": "*"
  },
  "pi": {
    "extensions": ["/./extensions"]
  }
}
```

The "pi-package" keyword makes the package discoverable. The pi manifest declares its resource directories, and peerDependencies identifies the Pi runtime packages supplied by the host. This list is intentionally shorter than the earlier generic manifest: the hello extension imports only @earendil-works/pi-ai and @earendil-works/pi-coding-agent. Add @earendil-works/pi-agent-core, @earendil-works/pi-tui, or typebox only if the package imports them.

Step 4 – Test locally

In a project directory where you use Pi, add your package path to settings:

```
{
  "packages": ["/absolute/path/to/pi-hello-demo"]
}
```

Then start Pi. Your extension should load. Test with /hello-pi and verify the startup notification appears.

Step 5 – Publish to npm

```
npm publish
```

Users can then install it with:

```
pi install @your-name/pi-hello-demo
```

For local testing before publishing, pi install also accepts a git URL or local path.

7.11.2 Package layout conventions

Directory	Contents	Discovered automatically?
extensions/	One or more .ts extension factories	Yes, when no pi manifest
skills/	.md skill files	Yes, when no pi manifest

Directory	Contents	Discovered automatically?
<code>prompts/</code>	Prompt template files	Yes, when no <code>pi</code> manifest
<code>themes/</code>	JSON theme files	Yes, when no <code>pi</code> manifest

With an explicit `pi` manifest, Pi uses only the declared paths. This supports non-standard locations and lets a package exclude conventional directories.

7.12 Debugging extensions

Work from loading toward the failing behavior:

1. **Confirm the module loads.** In TUI mode, add a `session_start` notification. Then load the file directly with `pi -e path/to/extension.ts`; if that works, discovery is the problem. Check the exact filename (`extensions/name.ts` or `extensions/name/index.ts`), the default factory export, and project trust for `.pi/extensions/`. A named export alone is not loaded.
2. **Check the code independently.** Run `npx tsc --noEmit path/to/extension.ts`. Jiti transpiles TypeScript but does not type-check it, so type errors can survive loading and fail later.
3. **Isolate one boundary.** Exercise one command or one simple tool call. Add temporary notifications to the relevant hooks in TUI mode, or concise logging when testing print mode. Check command names, tool names, schemas, and descriptions before debugging a multi-turn interaction.
4. **Inspect model-facing state.** Run `/debug`. It writes rendered TUI lines and the latest LLM messages to `~/.pi/agent/pi-debug.log`, which shows whether tool definitions and prompt metadata reached the model.
5. **Exercise lifecycle edges.** Run `/reload`, then test a new session, resume, fork, and exit when relevant. Verify `session_shutdown` cleanup and reconstruction in `session_start`.
6. **Test without a TUI.** Run `pi -e path/to/extension.ts -p "your prompt"` to catch unguarded UI assumptions. Notifications do nothing in print mode, so do not use their absence there as evidence that loading failed.

7.12.1 Common pitfalls

Symptom	Likely cause	Fix
Extension never loads	Wrong path, missing default export, or untrusted project	Check discovery conventions, export the factory as default, and establish project trust
<code>tool_call</code> handler does not match	Tool name or narrowing predicate is wrong	Check <code>event.toolName</code> and <code>isToolCallEventType()</code>
State disappears after <code>/reload</code>	State exists only in closed-over variables	Rebuild it from session entries in <code>session_start</code>
RPC dialog waits indefinitely	The client does not answer <code>extension_ui_request</code>	Send the matching response or set a dialog timeout
A dialog silently denies in print/JSON mode	<code>ctx.hasUI</code> was not checked	Define a noninteractive policy before calling the dialog

Symptom	Likely cause	Fix
Extension works once but fails after reload	Stale references or leaked resources	Clean up idempotently in <code>session_shutdown</code> ; do not reuse the old context after <code>reload()</code>
Model never calls the tool	Weak description or missing prompt guidance	Tighten <code>description</code> and add tool-named <code>promptGuidelines</code>
Parallel tools overwrite a file	The mutation bypasses Pi's per-file queue	Wrap the entire read-modify-write operation in <code>withFileMutationQueue</code>

Pi logs ordinary handler errors and continues. `tool_call` is fail-safe: if one of its handlers throws, Pi stops running later `tool_call` handlers for that call, does not execute the tool, and does not run `tool_result` handlers for it. Instead, the thrown error message becomes an `isError: true` tool-result message stored in the session and sent to the model; the agent loop and sibling tool calls can continue. Test policy hooks on both allow and deny paths.

8 Sessions, branches, and durable context

Pi does not treat a chat as a flat array that is overwritten on every new turn. The coding agent persists an append-only JSONL session tree and rebuilds the current conversation from a selected leaf. That design supports resume, fork, branch navigation, compaction, labels, and extension state without requiring a database.

This chapter is for extension authors, embedders, and contributors debugging resume, branching, or context selection. If you only use Pi interactively, the branching and compaction sections explain the behavior you will observe.

It is grounded in four files:

- `packages/coding-agent/src/core/session-manager.ts` for persistence and context reconstruction.
- `packages/coding-agent/src/core/messages.ts` for message conversion.
- `packages/coding-agent/src/core/agent-session-runtime.ts` for replacement and cleanup.
- `packages/coding-agent/docs/session-format.md` for the durable file format.

8.1 Vocabulary before the file format

A **session** is Pi's durable record of a conversation. An **entry** is one record in that session. A **branch** is one path through entries that share a common history. The **leaf** is the current last entry on that path. **Context** is the selected representation sent to the model; it is not automatically the whole file on disk.

Start by imagining two answers to the same user question: answer A and answer B share the earlier messages but diverge later. A flat array cannot keep both answers without duplication. The small tree below shows why Pi stores parents and children before the chapter introduces JSONL details.

8.2 Why persistence is a tree

Suppose a user asks for an implementation, then navigates back to an earlier message and explores a different plan. A linear transcript either loses one path or duplicates a large prefix into another file. Pi instead stores entries with an `id` and `parentId`. One file can represent both paths.

Session file:

```
header (metadata only: type=session, version, id, cwd, optional sessionId)
      # not a tree node; has no parentId and is never selected as a leaf
```

Entry tree (linked only by entry parentId values):

```
user: "Investigate auth"
```

```

|
+-- assistant: plan A
    |
    +-- tool result: read auth.ts
        |
        +-- user: "Implement A"                <- old leaf
            |
            +-- branch_summary: "A explored ..."
                |
                +-- user: "Try plan B instead" <- current leaf

```

`parentSession` on the header records fork provenance across session files. It is not a `parentId` edge inside this entry tree.

The active branch is the chain from the current leaf to the root, read in chronological order. Other children are not provider context merely because they exist on disk. When debugging a surprising model response, inspect that active path rather than the whole session file.

8.3 Files, headers, and locations

By default, coding-agent sessions live under the agent directory, arranged by working directory:

```
~/.pi/agent/sessions/--<encoded-cwd>--/<timestamp>_<uuid>.jsonl
```

Each persisted session is a `.jsonl` file. Its first line is a file header, for example:

```
{"type":"session","version":3,"id":"<session-uuid>","timestamp":"<iso-time>","cwd":"/work/project"}
```

The header's `id` identifies the session, not a node. Unlike every subsequent entry, the header has no `parentId` and cannot be selected as the active leaf. `version` controls format migration, `timestamp` and `cwd` describe where the session began, and optional `parentSession` records fork provenance only.

Session storage and other options are ordinary nested keys in settings JSON, not filesystem path conventions. Global settings live in `~/.pi/agent/settings.json`; project settings live in `.pi/settings.json` and override the global file for the same keys. Nested objects merge, so a project can set only the fields it needs.

Override the session directory with:

```
--session-dir > PI_CODING_AGENT_SESSION_DIR > settings.json sessionDir
```

`sessionDir` accepts absolute or relative paths and expands `~`. Example:

```
{ "sessionDir": ".pi/sessions" }
```

Use `SessionManager.inMemory(cwd?)` when embedding Pi where persistence is not desired. An in-memory manager still has branches and context construction; it simply has no session-file path.

8.4 The entry vocabulary

Every non-header entry has a **type**, an ID, a parent ID, and a timestamp. The following table shows why each entry exists and whether it becomes model context through `buildSessionContext()`.

Entry type	Meaning	Becomes LLM context?
<code>message</code>	A standard <code>AgentMessage</code> : user, assistant, tool result, or supported coding-agent message	Yes, subject to branch/compaction selection
<code>model_change</code>	Selected provider/model at that point in history	Used to restore state, not a chat message
<code>thinking_level_change</code>	Selected thinking level	Used to restore state, not a chat message
<code>compaction</code>	Summary plus the first entry retained after summarization	Yes, as a compaction-summary message
<code>branch_summary</code>	Summary of the path left during tree navigation	Yes, as a branch-summary message
<code>custom</code>	Extension-specific durable data	No
<code>custom_message</code>	Extension message with display/details	Yes
<code>label</code>	Named marker attached to an entry	No
<code>session_info</code>	User-visible session name	No

Extension authors should follow one rule from this table: use `custom` for state that Pi or the user needs, and use `custom_message/pi.sendMessage()` only when the model should receive it.

8.5 Message shapes at the provider boundary

The `message` entry wraps an `AgentMessage`. Core roles are:

<code>user</code>	user text and optional images
<code>assistant</code>	streamed text, thinking, and/or <code>toolCall</code> blocks
<code>toolResult</code>	result for a specific assistant tool call

Coding-agent adds message kinds for local interaction and synthesized context, including user bash execution, custom messages, compaction summaries, and branch summaries. `convertToLlm` and the agent loop decide how these richer messages become the provider's normalized message types.

An assistant's content is a list of typed blocks. A tool call is one such block, containing an ID, name, and argument object. The corresponding `toolResult` refers to the call ID. Retaining that link is essential: a model provider expects a tool result to answer a particular tool call, not merely a similarly named tool.

8.6 SessionManager as the persistence boundary

The class exposes both static constructors/listing methods and instance methods. Useful API groupings:

Intent	Methods
Start/open	<code>create</code> , <code>open</code> , <code>continueRecent</code> , <code>inMemory</code> , <code>forkFrom</code>
Discover	<code>list</code> , <code>listAll</code>
Append transcript	<code>appendMessage</code> , <code>appendModelChange</code> , <code>appendThinkingLevelChange</code>
Append summaries/state	<code>appendCompaction</code> , <code>appendCustomEntry</code> , <code>appendCustomMessageEntry</code> , <code>appendLabelChange</code> , <code>appendSessionInfo</code>
Navigate tree	<code>getLeafId</code> , <code>getBranch</code> , <code>getTree</code> , <code>getChildren</code> , <code>branch</code> , <code>resetLeaf</code> , <code>branchWithSummary</code>
Build context	<code>buildContextEntries</code> , <code>buildSessionContext</code>
Inspect metadata	<code>getHeader</code> , <code>getSessionName</code> , <code>getCwd</code> , <code>getSessionFile</code> , <code>isPersisted</code>

The manager owns the JSONL representation. Do not have an extension write a second, unrelated copy of the session file: it will not participate in branch or compaction semantics and may become inconsistent with the transcript.

8.7 Building context from a selected leaf

`buildContextEntries()` is the intermediate view used by the interactive transcript and session context construction. At a high level:

```
all persisted entries
  |
  v
walk parent links from active leaf to root
  |
  v
reverse into chronological active path
  |
  v
apply the latest effective compaction boundary
  |
  v
context entries for rendering and conversion
```

Without compaction, that active path is the selected context. After compaction, Pi substitutes the summary for the older span and keeps entries beginning at `firstKeptEntryId`. Replaced records remain in JSONL for history but are not sent to the LLM.

`buildSessionContext()` also scans the full path to restore the current model and thinking level, then converts selected entries to messages. It omits `custom` entries, so private extension state added with `pi.appendEntry()` does not consume provider context.

8.8 Compaction is a controlled replacement of old context

Context windows are finite. Auto-compaction triggers when:

```
estimated context tokens > model context window - reserveTokens
```

Defaults are `compaction.reserveTokens: 16384` and `compaction.keepRecentTokens: 20000`, configurable under the `compaction` object in global or project settings. Exact token calculation and cut-point behavior live in the agent harness; coding-agent coordinates the session operation, extension events, UI, and retry behavior.

Pi chooses a valid cut near the recent-history target, summarizes the older span (including a prior summary when present), appends a `compaction` entry, and rebuilds context from that summary plus the retained entries. Tool-call and tool-result relationships constrain where the cut may land. Raw history stays on disk; only provider context is replaced.

An extension can intercept `session_before_compact` to cancel compaction or provide a custom summary, then observe the result through `session_compact`.

8.8.1 Split turns and tool continuity

Normally a compaction cut sits at a turn boundary. A turn begins with a user message and includes the assistant's response and resulting tool calls until the next user message. Pi avoids cutting between an assistant tool call and its tool result because that would make the remaining context internally incoherent for the provider.

If one turn exceeds `compaction.keepRecentTokens`, Pi may split it at an assistant message, never at a tool result. For example:

```
summarized turn prefix      retained suffix
user -> assistant(tool A) -> result A | assistant(tool B) -> result B
                                   ^ firstKeptEntryId
                                   ^ isSplitTurn === true

session_before_compact.preparation:
  turnPrefixMessages = [user, assistant(tool A), result A]
  messagesToSummarize = earlier complete turns (if any)
  isSplitTurn         = true
```

Pi separately summarizes any older complete turns and the early prefix of this large turn, then merges those summaries. Here the retained suffix begins at `assistant(tool B)`, so tool call B remains beside result B. Extensions must handle both `turnPrefixMessages` and `messagesToSummarize` rather than assuming that compaction removes only whole user turns.

8.9 Branch navigation and branch summaries

`/tree` changes the leaf to a different entry. Work on the old branch may still matter for the new branch even though it is no longer on the parent chain. Pi can summarize that abandoned path and append a `branch_summary` at the navigation point so the new path keeps a concise account without replaying all old messages.

```
old path:    common ancestor -> A -> B -> C
new target:  common ancestor -> D -> E

optional summary of A/B/C is appended near E
```

Configure this under `branchSummary` (same settings files as above):

```
{
  "branchSummary": {
    "reserveTokens": 16384,
    "skipPrompt": false
  }
}
```

`reserveTokens` reserves response space for the summarizer. With the default `skipPrompt: false`, `/tree` asks whether to summarize; `true` suppresses that question and defaults to no summary. Extensions can provide or cancel a summary with `session_before_tree` and observe the result with `session_tree`.

8.10 Session replacement tears down the old runtime

New, resume, and fork flows go through `AgentSessionRuntime`. That object owns the current `AgentSession` and cwd-bound services such as settings, resources, models, and auth. Replacement deliberately tears the old runtime down and builds a new one for the destination session; it does not mutate the previous session object into a new identity.

```
request /resume, /new, /fork, or the matching extension command
-> session_before_switch or session_before_fork (may cancel)
-> session_shutdown on the old extension runtime
-> old AgentSession disposed
-> services and AgentSession created for the target cwd/session
-> session_start on the new extension instance
-> host rebinds UI/extension runtime to the new session
-> optional withSession callback receives a fresh context
```

Extension commands call `ctx.newSession`, `ctx.switchSession`, or `ctx.fork`. Any `withSession` callback receives a fresh replacement context. Captured pre-switch `pi`, command context, or `SessionManager` objects are stale after replacement and must not be reused. Capture plain strings or IDs before the switch, then use only the callback context for session-bound work.

8.11 Session names, labels, and human navigation

`/name` and `pi.setSessionName()` append `session_info`. The selector then shows the stored name rather than deriving a name from the first user message. Labels are per-entry bookmarks, added by `pi.setLabel(entryId, label)`, and appear in tree navigation. They are lightweight metadata: setting a label does not edit a historic message or change model context.

Use labels for checkpoints, handoff points, and experiment names. Use a session name for the overall task. That division makes a large session tree easier to scan without placing extra prose in provider context.

8.12 Extension persistence patterns

8.12.1 Stateful tool results

When state is the consequence of a tool call, return a serializable snapshot in `details` and reconstruct from current-branch tool results in `session_start`. This keeps state branch-aware.

```
pi.on("session_start", (_event, ctx) => {
  for (const entry of ctx.sessionManager.getBranch()) {
    if (entry.type !== "message" || entry.message.role !== "toolResult") continue;
    if (entry.message.toolName !== "project_notes") continue;
    // Rebuild extension state from entry.message.details after validating it.
  }
});
```

8.12.2 Private durable data

Use `pi.appendEntry("my-extension", data)` when the model does not need the state. That API delegates to the active session's `SessionManager.appendCustomEntry(customType, data)`, which appends a `custom` node as a child of the current leaf and advances the leaf. On the next `session_start`, scan `getEntries()` or the active branch for entries with that `customType`. Register an entry renderer if users should see the data in the transcript.

8.12.3 Persistent model context

Use `pi.sendMessage({ customType, content, display })` when the message should be retained *and* given to the model. Register a message renderer if the default transcript presentation is insufficient. Do not encode secrets or verbose debug data in a custom message because it becomes conversation context.

8.13 Session debugging checklist

When a resumed session behaves unexpectedly, answer these in order:

1. Which JSONL file is active (`getSessionFile()`)?
2. Which leaf is selected (`getLeafId()`)?
3. Which entries are in `buildContextEntries()`?
4. Is a compaction or branch summary affecting context?
5. Which model/thinking-level entry is last on the full path?
6. Is the extension reading the current branch or every historic entry?
7. Did a replacement flow leave a stale object captured in a closure?

This approach is faster than treating session persistence as an opaque feature: the entire model context is explainable as a deterministic view of the active JSONL tree.

9 Resources, settings, and project trust

Before the first prompt, Pi assembles a runtime from configuration, credentials, models, extensions, skills, prompts, themes, and context files. This chapter explains that startup surface so you can change configuration intentionally and know when a project-local change will be ignored.

Unless otherwise noted, paths in this chapter are relative to `packages/coding-agent/`. Primary implementation files are `src/core/settings-manager.ts`, `src/core/resource-loader.ts`, `src/core/trust-manager.ts`, and `src/core/agent-session-services.ts`. The user-facing configuration reference is `docs/settings.md`.

Keep one distinction in view throughout the chapter: an extension is executable code, while a skill, prompt, or context file is instruction supplied to the model. A package is only a delivery format and may contain either category. Choose the least powerful category that solves the problem, and review a package according to what it contains rather than treating “package” as a separate security mode.

9.1 The four words to keep separate

A **resource** is something Pi discovers at startup, such as a skill, prompt, theme, or extension. A **setting** is configuration that changes how Pi runs. **Trust** is the decision that permits project-local code or settings to load. A **scope** is where a value applies, usually global or project-local.

For example, a project prompt file supplies instructions, a project extension can execute code, and a project setting can enable either one. They may arrive from the same package directory, but they do not have the same power or trust requirements.

9.2 The startup services boundary

`createAgentSessionServices()` creates the cwd-bound services needed for a session:

```
cwd + agent directory
-> SettingsManager
-> AuthStorage
-> ModelRegistry
-> DefaultResourceLoader
    -> reload resources subject to trust
-> apply pending provider registrations from extension factories
```

This separation is deliberate. A resumed session can have a different cwd than the process’s initial cwd, so settings, project resources, and models must be created for the effective session cwd. The `AgentSessionRuntime` rebuilds this service bundle on a session replacement rather than pretending that settings from the old project apply to the new one.

9.3 Global and project settings

Pi reads JSON settings from two conventional scopes:

File	Scope	Relative resource paths resolve from
<code>~/.pi/agent/settings.json</code>	All projects for the current user	<code>~/.pi/agent</code>
<code>.pi/settings.json</code>	The current project	<code>.pi</code>

Project settings override global settings. Nested objects are merged, so a project can adjust only one compaction field while retaining the global default for the rest:

```
// global
{ "compaction": { "enabled": true, "reserveTokens": 16384 } }

// project
{ "compaction": { "reserveTokens": 8192 } }
```

The effective configuration has `enabled: true` and `reserveTokens: 8192`. Use this pattern for small project adjustments rather than copying a full settings file that becomes stale as Pi adds defaults.

9.4 Trust is an execution policy, not a cosmetic prompt

Repository-owned settings can select extensions and packages; skills can direct the model to run commands. Pi therefore asks for trust before it loads project configuration, installs project packages, or executes project extensions.

At interactive startup, Pi looks for trust-requiring inputs and for a saved decision covering the current folder or one of its parents. If neither settles the question, Pi can prompt. Approval enables `.pi/settings.json`, local resources under `.pi`, missing package installation, `.agents/skills`, and extension code from those sources. Context instructions such as `AGENTS.md` and `CLAUDE.md` are the exception: they load regardless of trust unless context loading is disabled.

Pi's noninteractive modes are print mode (`-p` or `--print`), which processes the initial prompt and exits; JSON mode (`--mode json`), which writes session events as JSON lines; and RPC mode (`--mode rpc`), which exchanges JSONL commands and events over standard input and output. None can display the startup trust prompt. Without an applicable saved decision, each follows the global `defaultProjectTrust` setting:

Value	Noninteractive behavior without saved decision
"ask"	Ignore project resources because there is no UI prompt
"never"	Ignore project resources
"always"	Trust project resources

The CLI flags `--approve/-a` and `--no-approve/-na` override the decision for one run only.

`/trust` immediately writes the selected decision to `~/.pi/agent/trust.json`. The command changes persisted state for future Pi processes; it does not change the trust decision already resolved by the current process. A live `/reload` also keeps that already-resolved trust state, so a newly saved `/trust` answer does not start loading (or stop ignoring) project resources until Pi itself is restarted. A same-cwd session replacement such as `/new` likewise reuses the process-level decision rather than rereading `trust.json`.

Before loading repository extensions, Pi emits `project_trust` only to user/global and CLI-provided extensions. A handler returns `trusted: "yes" | "no" | "undecided"`; the first yes/no response decides the outcome. Code awaiting approval therefore cannot approve itself.

9.5 Resource categories

The resource loader assembles several independently useful kinds of content:

Resource	What it changes	Typical discovery location
Extension	Runtime behavior, tools, commands, UI, providers	<code>extensions/</code> , configured paths
Skill	Reusable instruction bundle and optional <code>/skill:name</code> command	<code>skills/</code> , <code>.agents/skills</code>
Prompt template	User-invoked prompt expansion	<code>prompts/</code> , configured paths
Theme	Terminal styling data	<code>themes/</code> , configured paths
Context file	Instructions added to system-prompt construction	<code>AGENTS.md</code> and related loader results
Package	Bundle containing the above	<code>npm/git/local</code> source

These categories are not interchangeable. A theme changes presentation, while skills, prompts, and context files shape model behavior. When instructions are enough, do not replace them with an extension that receives the Pi process's permissions.

9.6 Explicit resource paths and patterns

The settings keys `extensions`, `skills`, `prompts`, and `themes` accept local file paths or directories. Their relative paths use the settings scope shown above: the agent directory for global settings and `.pi` for repository settings. `packages` instead names `npm`, `git`, or local package sources.

Resource arrays and package filters support patterns:

Syntax	Meaning
<code>pattern</code>	Include matching paths
<code>!pattern</code>	Exclude matching paths
<code>+path</code>	Force-include an exact path
<code>-path</code>	Force-exclude an exact path

In the object form of a `packages` entry, `pattern` and the exact paths after `+` or `-` are relative to that package's root—not to either settings directory. For example, `+themes/legacy.json` addresses `themes/legacy.json` inside the selected package. Use absolute paths or `~` for top-level local resources only when they truly belong to one machine rather than to the repository.

9.7 Skills and prompt templates on the input path

`AgentSession.prompt()` gives extension commands first chance to handle slash input. It then emits the extension `input` hook. If input is not handled, Pi expands skill commands and prompt templates. That means a raw `/skill:name` is not necessarily the message sent to the model; it can become the skill's content plus user arguments.

`enableSkillCommands` (default `true`) controls interactive registration of skills as `/skill:name` slash commands and autocomplete entries. Setting it to `false` removes that interactive command surface; it does not unload skills from system-prompt construction, and `AgentSession` still expands a submitted `/skill:name` prompt into skill content. When debugging prompt content, inspect both `ctx.getSystemPromptOptions()` and the actual context event rather than searching only the typed text.

9.8 Context files and system-prompt construction

Context discovery first checks the agent directory (normally `~/.pi/agent`) for one context file. It then walks from the session cwd through every ancestor to the filesystem root and checks each directory. In each location it loads only the first existing candidate in this exact order: `AGENTS.md`, `AGENTS.MD`, `CLAUDE.md`, then `CLAUDE.MD`. Ancestor files are ordered from the filesystem root down to the cwd after the global file. Thus a cwd-level file does not stop discovery in parent directories, but an `AGENTS.md` and `CLAUDE.md` in the same directory are not both loaded. `--no-context-files/-nc` disables this discovery.

The loader provides loaded context files and skills to `AgentSession`. `_rebuildSystemPrompt()` creates structured `BuildSystemPromptOptions` from:

- cwd;
- selected tool names, snippets, and guidelines;
- custom/append system-prompt CLI options;
- loaded skills;
- loaded context files.

`buildSystemPrompt()` turns those options into provider-independent system prompt text. The system prompt is rebuilt when the active tool set changes and when session resources reload. Extensions that replace the prompt in `before_agent_start` receive the already-built prompt and can chain a change, but should avoid discarding these inputs unintentionally.

9.9 Settings that alter runtime behavior

Settings are more than UI preferences. The following groups affect runtime semantics and should be considered part of a reproducible project setup.

9.9.1 Model and thinking

`defaultProvider`, `defaultModel`, `defaultThinkingLevel`, `enabledModels`, and `thinkingBudgets` influence startup selection, model cycling, and provider options. A saved session model takes precedence during a resume if it is still configured and authenticated; defaults are a fallback.

9.9.2 Compaction and branch summary

`compaction.enabled`, `reserveTokens`, and `keepRecentTokens` determine when and how much conversation is summarized. `branchSummary.reserveTokens` and `skipPrompt` affect tree navigation. These settings change what reaches a provider, so they are not merely transcript retention preferences.

9.9.3 Retry and delivery

These settings control how long Pi waits after a failure and how queued messages are ordered—both matter when reproducing a session. The `retry` block separates Pi’s agent-level recovery from provider/SDK request retries. Provider retries default to zero so a long SDK delay does not silently take control away from session-level retry logic.

`steeringMode` and `followUpMode` determine whether each queue drains all messages together or one at a time. `transport`, `httpIdleTimeoutMs`, and `websocketConnectTimeoutMs` bound provider connection and idle behavior.

9.9.4 Shell, images, and terminal

`shellPath`, `shellCommandPrefix`, and `npmCommand` affect process execution and package management. Image settings control `resize/block` behavior before a provider request. Terminal image and rendering settings affect display but do not make an image model-capable; model input capabilities still control whether an image can be sent.

9.10 Project configuration that an extension should respect

An extension with its own project-level configuration should:

1. Use `CONFIG_DIR_NAME` from `coding-agent` rather than hardcoding `.pi`.
2. Resolve the path relative to `ctx.cwd`.
3. Call `ctx.isProjectTrusted()` before reading that configuration.
4. Validate parsed JSON as untrusted data.
5. Define which state is global, project-local, and session-local.

For example, a global extension may store user preferences in the agent directory but read a project allowlist only after trust. That matches Pi’s own security model and avoids creating an unreviewed bypass.

9.11 Packages are resource delivery, not a new execution mode

Pi packages distribute extensions, skills, prompts, and themes from npm, git, or a local path. A package can declare a `pi` manifest in `package.json` or use conventional subdirectories. The package manager records sources in the `packages` array of the applicable settings file: global sources in `~/.pi/agent/settings.json`, and sources installed with `-l` in the project’s `.pi/settings.json`. The resource loader then finds the declared resources.

Package scope follows settings scope:

- Global managed installs are user-wide and live under the agent directory.
- Project managed installs are stored under `.pi` and are trust-gated.
- A local path is loaded in place, so edits affect the next load/reload.

The package commands require the package source for install and removal:

```
pi install <source> [-l]
pi remove <source> [-l]
pi list
pi update --extensions
```

Here `<source>` is an npm spec, git source, URL, or local path such as `npm:@foo/pi-tools`, `git:github.com/user/repo`, or `./local-package`.

`pi update --extensions` is the packages-only update target. CLI help defines it as “Update installed packages only”: it updates package content from global settings and, when project trust applies, project settings, without updating the Pi CLI. Despite the historical flag name, it is not limited to extension files inside packages; skills, prompts, and themes shipped by those packages are updated too. Unpinned npm sources move to the version selected by their range (or latest when no version is specified); pinned npm versions are skipped; git clones are reconciled to their configured refs without advancing those refs; local-path sources need no update and are ignored.

These commands operate on the package layer. They do not change the fact that an extension is regular code with the privileges of the Pi process.

9.12 Themes and the TUI boundary

Theme paths load JSON themes. Themes influence colors/formatting used by interactive components and extension renderers; they do not modify keybindings or component logic. An extension can query/change themes through `ctx.ui`, but should use semantic theme keys (`accent`, `warning`, `toolTitle`, and so on) instead of ANSI constants so the user’s theme continues to work.

9.13 Reload semantics

`/reload` replaces extensions, skills, prompts, themes, and context files; it does not preserve arbitrary extension closures. From an extension's point of view, the observable lifecycle events are:

```
old runtime: session_shutdown { reason: "reload" }
new runtime: session_start { reason: "reload" }
              resources_discover { reason: "reload" }
```

Pi reloads settings and resources between the shutdown and startup events while preserving the runtime's already-resolved project trust state. It does not reread a decision just saved by `/trust`; restart Pi for that decision to affect resource loading. Code after `await ctx.reload()` nevertheless continues in the old command call frame. Treat that call as terminal for the handler, and rebuild session-scoped state when the new runtime observes `session_start`.

9.14 Pre-commit security and reproducibility checklist

Use this checklist to review whether a repository configuration has the intended security boundary and will behave consistently for teammates and CI:

1. Does this setting execute code or fetch/install dependencies?
2. Does the team intend to trust the repository for that behavior?
3. Are environment references used for secrets instead of literal credentials?
4. Are relative paths intentional for this scope?
5. Does a nested override preserve the global fields you expect?
6. Will noninteractive CI receive the required trust decision?
7. Can the change be represented as a skill/theme/template instead of code?

These questions make configuration review an architectural step rather than an afterthought.

10 Models, providers, and authentication

Use this chapter when a model is missing, a request fails auth, custom `models.json` overrides do not apply, thinking levels look wrong, cost/footer numbers surprise you, or you need to understand how Pi turns a selected model into a provider request.

Pi keeps four provider concerns separate:

```
model metadata    -> what a model can do and how much it costs
provider config   -> where/how requests are sent
credentials       -> how a provider is authenticated
API adapter       -> how normalized Pi context becomes provider protocol
```

The separation lets the coding agent present one model selector and one agent loop while supporting OpenAI-compatible servers, Anthropic, Google, Bedrock, OAuth, proxies, and extension-defined providers. Streaming is the adapter's delivery path for that protocol, not a separate product surface: adapters emit a normalized event stream that the agent consumes.

Key source files are `packages/ai/src/types.ts`, `packages/ai/src/compat.ts`, `packages/ai/src/models.ts`, `packages/coding-agent/src/core/model-registry.ts`, and `packages/coding-agent/src/core/auth-storage`.

10.1 Vocabulary and a concrete comparison

A **model** is the named capability record Pi selects. A **provider** is the service or adapter that can answer requests. A **credential** proves access to that provider. An **API adapter** translates Pi's normalized request into one provider's wire format. In a local setup, `local-model` is a model ID, `local` is a provider name, an environment variable is a credential source, and `openai-completions` is an adapter choice. These are four different values, not four names for the same thing.

Read the chapter in that order: metadata, registry, credentials, dispatch, then custom providers. It prevents an authentication problem from being mistaken for a model-catalog problem.

10.2 The Model contract

A model is a data record consumed by the agent and provider layer. Its important fields include provider identifier, model ID, API type, input capabilities, reasoning/thinking support, context window, maximum output tokens, cost, base URL, headers, and compatibility flags.

The model's `api` field chooses an adapter such as `openai-completions`, `openai-responses`, `anthropic-messages`, `google-generative-ai`, or `bedrock-converse-stream`. The provider name is not enough: two provider names can use the same wire protocol, and one provider can expose models with different API requirements.

Use `input: ["text", "image"]` only when a model can actually receive image content. That meta-data drives UI/tool behavior and protects against sending an image to a text-only model.

10.3 Model registry assembly

`ModelRegistry` begins with generated built-in provider/model data, applies `models.json` provider and per-model overrides, merges custom models, applies OAuth-provider model modifications where required, and finally applies dynamic extension provider registrations.

```
built-in provider catalogs
-> models.json provider overrides
-> models.json per-model overrides
-> custom models merged/upserted
-> OAuth model adjustment
-> extension registerProvider configuration
-> current ModelRegistry.models
```

Failure isolation is deliberate. If `models.json` cannot be read, parsed, or validated, the registry records the error for the interactive application but still loads the generated built-in catalog. The failed load contributes no custom models or catalog overrides. A typo in a local override can therefore disable that file's customizations without making every known built-in model disappear.

`refresh()` resets dynamic API/OAuth registrations, reloads disk config, then reapplies registered extension providers. This prevents stale registrations from surviving a model configuration reload.

10.4 Selecting an initial model

At session construction, `createAgentSession()` considers a session non-empty when its reconstructed context contains at least one message. Header and settings entries alone do not count. For such a session, it first tries the provider/model recorded on the current branch; if that model exists and has configured authentication, Pi restores it. Otherwise `findInitialModel()` selects a configured/default available model and Pi reports the fallback. The session similarly restores or chooses a thinking level, then clamps it to the selected model's capabilities.

The selection is persisted with `model_change` and `thinking_level_change` entries. Resume is therefore not simply “use today's global default”; it attempts to continue the model choice that created historical context.

10.5 Thinking levels are Pi vocabulary

Pi's model-facing thinking levels are `off`, `minimal`, `low`, `medium`, `high`, `xhigh`, and `max`. Providers use different APIs and vocabulary. A model's `thinkingLevelMap` is a tristate map from those Pi levels to provider-specific values:

Value	Support	Selector / clamp	Wire value
omitted key	Standard levels through high remain available and use the adapter/provider default; xhigh and max are unsupported until mapped	Hidden for xhigh / max ; otherwise kept	Adapter default for that Pi level
string	Level is supported	Shown; not clamped away	That string is what adapters send when the level is selected
null	Level is unsupported	Hidden/skipped; session clamps requests away from it	Never serialized as a provider thinking value

Maps may contain holes. For example, a model can expose **high** and **max** without **xhigh**:

```
{
  "id": "custom-reasoner",
  "reasoning": true,
  "thinkingLevelMap": {
    "minimal": null,
    "low": null,
    "medium": null,
    "high": "default",
    "xhigh": null,
    "max": "maximum"
  }
}
```

Setting **"off": null** means thinking cannot be disabled. This is more precise than a boolean **reasoning** flag: the selector and clamping path agree on the supported set before any provider-specific serialization runs.

10.6 Custom models in `models.json`

The conventional file is `~/.pi/agent/models.json`. A minimal local OpenAI-compatible provider looks like this:

```
{
  "providers": {
    "local": {
      "baseUrl": "http://localhost:11434/v1",
      "api": "openai-completions",
      "apiKey": "local-placeholder",

```

```

    "models": [{ "id": "local-coder" }]
  }
}

```

The placeholder key is deliberate for local services that ignore it. Pi uses configured-auth status when deciding whether to offer a model, so a provider with no auth source would otherwise be unavailable even if its server accepts unauthenticated requests.

A full model definition can set human name, **reasoning**, inputs, context window, maximum tokens, cost rates, and compatibility behavior. Cost rates are per million tokens. Optional **cost.tiers** describe request-wide alternate rate sets, not progressive billing.

```

{
  "cost": {
    "input": 5,
    "output": 30,
    "cacheRead": 0.5,
    "cacheWrite": 6.25,
    "tiers": [
      {
        "inputTokensAbove": 272000,
        "input": 10,
        "output": 45,
        "cacheRead": 1,
        "cacheWrite": 12.5
      }
    ]
  }
}

```

`calculateCost()` in `packages/ai/src/models.ts` picks rates as follows:

1. Start from the base **cost** rates.
2. Sum input-side usage as **input** + **cacheRead** + **cacheWrite**.
3. Among tiers with **inputTokensAbove** strictly below that sum, choose the highest threshold.
4. Apply that complete rate set to every token category on the request: input, output, cache read, and cache write. Tokens are not split across base and tiered rates.

With the rates above, 272,000 input-side tokens still use \$5/\$30; 272,001 switches the whole request to \$10/\$45. Multiple tiers work the same way: only the highest matching threshold is used. This metadata drives Pi's footer and session estimates; it does not determine the provider's invoice.

10.7 Provider overrides and model overrides

Configuration source determines whether a model list merges or replaces:

Goal	Mechanism	Catalog result
Route built-ins through a proxy	<code>models.json</code> provider <code>baseUrl</code> or <code>headers</code> , without <code>models</code>	Keeps the built-in catalog and changes request configuration.
Add or replace individual models	<code>models.json</code> provider <code>models</code> array	Upserts those models into the built-in catalog.
Adjust one known model	<code>models.json</code> <code>modelOverrides</code> , keyed by model ID	Keeps the catalog and changes only that model's metadata.
Define an extension provider catalog	<code>pi.registerProvider(name, { models })</code>	Replaces all current models for that provider.

In particular, `models.json` model definitions merge by model ID, whereas an extension registration with `models` replaces the provider catalog. Choose the source whose catalog semantics match the intended change.

10.8 Config values and secrets

Provider `apiKey` and header values can be literals, `$ENV_VAR/ ${ENV_VAR}` interpolation, or a leading `!command` whose stdout becomes the value. `$$` escapes a dollar and `$!` escapes a leading bang.

```
{
  "apiKey": "$TEAM_PROVIDER_KEY",
  "headers": {
    "x-tenant": "$TENANT_ID",
    "x-token": "!security find-generic-password -ws team-provider"
  }
}
```

Resolution lives in `packages/coding-agent/src/core/resolve-config-value.ts`. Command handling depends on the credential source:

Source	When resolved	Caching
<code>models.json</code> / extension provider <code>apiKey</code> and headers	Request time via uncached resolution	No TTL, no stale reuse, no automatic recovery if the command fails
<code>auth.json</code> API-key key	When that credential is read	Successful or failed command stdout/result is cached for the process lifetime

Availability and auth-status checks deliberately do not execute command-backed values; they report configuration presence rather than proving that the command will succeed. Pi will not infer cache warming, lock contention, or 1Password rate limits for you. If the secret fetch is slow, interactive, rate-limited, or must reuse a previous token for a few minutes, wrap it:

```
{
  "apiKey": "!~/.config/pi/bin/team-provider-token"
}
```

```
#!/usr/bin/env bash
# ~/.config/pi/bin/team-provider-token
set -euo pipefail
cache="${XDG_CACHE_HOME:-$HOME/.cache}/pi/team-provider-token"
max_age_seconds=300
if [[ -f "$cache" ]] && [[ $((($(date +%s) - $(stat -f %m "$cache"))) -lt $max_age_seconds ]]; then
  cat "$cache"
  exit 0
fi
mkdir -p "$(dirname "$cache")"
security find-generic-password -ws team-provider | tee "$cache"
```

Use a path/script you control. Do not put secrets in the script body or logs.

10.9 Credential precedence and storage

AuthStorage persists API-key and OAuth credentials in **auth.json** under the agent directory. Its file backend creates parent directories with restrictive permissions and writes the file mode as 0600. It uses a lock around reads and writes so multiple Pi processes do not race during token refresh.

For the coding agent's **getApiKeyAndHeaders()** request path, credential precedence is:

1. The in-memory runtime override set by **--api-key**.
2. The provider's **auth.json** credential: either an API key or an OAuth token. Expired OAuth credentials are refreshed while holding the storage lock.
3. The provider **apiKey** configured by **models.json** or an extension, resolved from a literal, environment interpolation, or **!command**.

The registry deliberately asks **AuthStorage** without its broader standard provider-environment fallback before step 3. Provider-scoped **env** values saved with an API-key credential override **process.env** only while expanding that provider's configured key and headers. Runtime overrides are never persisted.

The result is not merely a string. **getApiKeyAndHeaders()** returns either an error or the resolved API key, headers, and provider-scoped environment. With **authHeader: true**, it also creates **Authorization: Bearer <key>** and returns an error if no key resolved.

10.9.1 OAuth lifecycle

Running **/login <provider>** invokes the registered provider's **login()** flow with callbacks that present an authorization URL or device code, collect prompts and selections, and report progress. Pi stores the returned access/refresh data and expiry in **auth.json**. For a request, **getApiKey()** calls the provider's **getApiKey()** to derive the wire credential. After expiry, one process refreshes through

`refreshToken()` under the file lock and persists the replacement credentials; other Pi processes then reread the result. An OAuth provider may also implement `modifyModels()` to derive account-specific endpoints or model metadata from the stored credentials. Thus extension OAuth support requires concrete `login`, `refreshToken`, and `getApiKey` implementations, not just an OAuth label.

10.10 Provider headers and request layering

Header precedence is easiest to read from lowest to highest:

1. Coding-agent session and attribution defaults.
2. Model static headers.
3. Resolved provider configuration headers.
4. Resolved per-model request headers.
5. The generated bearer header when `authHeader: true`.
6. Extension `before_provider_headers` mutations.
7. Adapter defaults and transport handling for headers not already supplied.

Within the registry, steps 2–5 override earlier steps; its merged map then overrides attribution defaults. Extension mutations run last before dispatch. Setting a header to `null` asks adapters that support deletion to suppress that header. Never log the merged map: it may contain credentials as well as tracing IDs.

10.11 The unified streaming API

`packages/ai/src/types.ts` defines a provider-independent `Context` and an `AssistantMessageEventStream`. `StreamOptions` carries abort signal, key, transport preference, cache retention, session ID, headers, timeout, retry, payload/response hooks, metadata, and provider-scoped environment.

`streamSimple(model, context, options)` in `src/compat.ts` is a dispatch point:

```
model selected by coding-agent
-> built-in model dispatch, or API provider lookup by model.api
-> provider.streamSimple(model, Context, options)
-> normalized AssistantMessageEventStream
```

Individual API adapters serialize messages/tools/system prompt into their wire format, parse streaming chunks, and emit normalized partial/final events. The agent loop therefore does not need a branch for every vendor protocol.

10.12 Payload and response hooks

`StreamOptions` has `onPayload` and `onResponse`. Coding-agent connects these to extension events. `before_provider_request` sees the provider-specific payload after an adapter serializes Pi context. `after_provider_response` sees status/headers after HTTP response receipt and before body consumption when the provider exposes them.

Use these for narrow instrumentation or compatibility experiments. A broad payload rewrite is fragile because it takes ownership of a provider protocol. If the endpoint genuinely differs from supported APIs, use a custom provider stream implementation instead.

10.13 Transport, timeout, and retry

Pi's shared `Transport` values are `sse`, `websocket`, `websocket-cached`, and `auto`. Providers that do not support a choice ignore it. HTTP idle timeout, WebSocket connection timeout, provider SDK retry count, and max retry delay are separate settings because they govern different failures.

The coding-agent stream function applies settings and calls `streamSimple` with an effective timeout. It treats a configured zero HTTP idle timeout as an effectively disabled finite timeout for SDKs that interpret zero as immediate expiry. Agent-level retry happens later in `AgentSession`, after it receives a final failed assistant message, and is separate from SDK/provider retries.

10.14 Custom providers from an extension

`pi.registerProvider(name, config)` can override endpoint/headers for an existing provider without model replacement, register a new provider with model definitions, register OAuth callbacks for `/login`, or register a custom `streamSimple` implementation for a nonstandard API.

Provider config validation rejects incoherent registration: models require a base URL and either API-key config or OAuth; each model must resolve an API type. An async extension factory is the correct place for remote model discovery because Pi waits for it before startup/model listing continues.

When unregistering, the registry refreshes and reapplies remaining registered providers so built-in models/protocol behavior previously overridden by that provider are restored.

10.15 Provider debugging workflow

When a model does not appear or a request fails, work in this order:

1. Inspect `ModelRegistry.getError()` for a `models.json` parse/load error.
2. Confirm provider/model ID and API type.
3. Use `getProviderAuthStatus()` as a non-secret configuration diagnostic, not as credential validation: it neither runs `!command` values nor refreshes OAuth tokens, and request-time resolution can still fail.
4. Confirm `baseUrl`, `compat`, input capabilities, and thinking map.
5. Use `before_provider_request` only to inspect a minimal sanitized payload.
6. Check `after_provider_response` status/headers where supported.
7. Read the final assistant message's `stopReason` / `errorMessage`, then separate layers: adapter or HTTP failure ends as `stopReason: "error"` (or `"aborted"`); if the UI then shows another attempt, check session events for `auto_retry_start`/`auto_retry_end` (agent transport retry) versus `compaction_start`/`compaction_end` (context overflow recovery). Provider SDK retries happen earlier inside the transport and will not emit those session events.

This preserves Pi's architecture: the model registry owns configuration and credentials; the adapter owns protocol translation; the agent owns turn flow.

11 Terminal UI and interactive mode

Pi's interactive application is built from a reusable terminal UI package and a coding-agent-specific composition layer. The separation is worth learning: an extension normally customizes the coding-agent UI surface, while a contribution to `packages/tui` changes primitives that could serve another terminal application.

Use this chapter according to the layer you plan to change. Extension authors should focus on editor submission, transcript renderers, and `ctx.ui`; coding-agent contributors should follow `InteractiveMode` and session events; TUI contributors should focus on terminal lifecycle, components, input, and differential rendering.

Relevant files include `packages/tui/src/tui.ts`, `packages/tui/src/terminal.ts`, `packages/tui/src/keybindi`, `packages/tui/src/autocomplete.ts`, and `packages/coding-agent/src/modes/interactive/interactive-mod`

11.1 Vocabulary before rendering details

A **terminal UI (TUI)** is an interface drawn inside a terminal window. A **component** is an object that renders lines and handles input. A **frame** is one rendered screen state. **Differential rendering** updates only lines that changed between frames. `InteractiveMode` composes these primitives into Pi's chat; it does not define the generic terminal primitives itself.

Start with a static line, then a component that changes its text, then a streaming assistant row. That progression explains why components invalidate themselves and why the TUI listens to agent events instead of polling a model.

11.2 The TUI package owns terminal mechanics

The public barrel `packages/tui/src/index.ts` exports the TUI class, component/container interfaces, text/markdown/editor/list components, terminal implementations, key parsing/matching, key-bindings, autocomplete, image support, and ANSI-aware text utilities.

The TUI class is responsible for the component tree, focus, layout, input routing, overlays, terminal lifecycle, and differential screen updates. A component returns lines for an available width and can invalidate itself when its rendered state changes. This is a retained component model, not immediate `console.log` output.

11.3 Differential rendering in practical terms

Terminal rendering is expensive and visually disruptive if every update clears the whole screen. The TUI tracks prior lines and writes only the changes needed to transform the terminal into the newly rendered view.

```
component tree changes
-> render visible lines with ANSI styling
-> compare with previous terminal frame
-> emit minimal cursor movement/text changes
```

That design enables token-by-token assistant updates, tool-output streaming, and autocomplete without a full redraw for every byte. It also explains why components should invalidate only when necessary and reuse component instances for high-frequency partial updates where the API supports it.

11.4 Components are composition, not application policy

The TUI exports general components such as `Text`, `Box`, `Container`, `Markdown`, `Editor`, `Input`, `SelectList`, `SettingsList`, `Loader`, and `Image`. Their job is layout/presentation/input behavior. They do not know what a session, provider, or tool result means.

Coding-agent builds a chat-specific component tree around those primitives:

```
InteractiveMode
-> header/status regions
-> chat transcript container
    -> user/assistant/thinking/tool components
-> pending messages/tool area
-> widgets above/below editor
-> editor or extension replacement editor
-> footer and overlays/dialogs
```

This means a custom transcript renderer should return a **TUI Component**; it does not need to manage raw terminal escape sequences itself.

11.5 Terminal lifecycle and capabilities

`ProcessTerminal` and terminal support code manage stdin/stdout, raw mode, size information, terminal capabilities, and cleanup. Image rendering is capability-dependent: Pi can choose Kitty, iTerm2, or fallback behavior based on what the terminal supports.

Interactive mode initializes terminal/theme behavior before starting its event loop and stops/restores terminal state at shutdown. If you add a feature that opens an external editor, subprocess, or custom modal, follow the existing interactive-mode lifecycle rather than leaving raw terminal state altered.

11.6 Keyboard input is parsed before it becomes a binding

`packages/tui/src/keys.ts` parses terminal input into key semantics and `packages/tui/src/keybindings.ts` maps semantic keybinding IDs to user-configurable keys. A binding ID is stable, namespaced policy such as:

```
tui.input.submit
tui.editor.cursorWordLeft
tui.select.confirm
app.interrupt
app.tools.expand
app.model.cycleForward
```

An extension should display hints with `keyHint("app.tools.expand", "to expand")` or inspect the injected `KeybindingsManager`. It should not hardcode raw `ctrl+x` checks. Hardcoding bypasses user configuration and breaks the mapping between the keybindings file, help text, and actual behavior.

11.7 Editor submission and queue semantics

The submission outcome comes first: when idle, regular Enter resolves `getUserInput()`, whose loop calls `session.prompt(text)`. While the agent is streaming, regular Enter sends a steering prompt for the active run, and Alt+Enter queues a follow-up prompt for after that run finishes.

The interactive editor is therefore more than a text box. Its submission handler also handles:

- built-in slash commands;
- `!/!!` user shell commands;
- message holding during compaction, which replaces older conversation context with a shorter summary to free context-window space;
- steering/follow-up delivery while an agent is active;
- normal queued input for the main `run()` loop;
- editor history, multiline text, image paste, autocomplete, and external edit.

The detailed handling stays in `InteractiveMode` so `AgentSession` can serve print/RPC clients without importing terminal components.

11.8 Transcript rendering from agent events

`InteractiveMode.subscribeToAgent()` receives coding-agent session events. `handleEvent()` dispatches behavior by type. In a typical tool-calling turn, the first occurrence of each event follows this runtime order:

Event	UI responsibility
<code>agent_start</code>	Clear pending tool bookkeeping and begin activity state
<code>message_start</code>	Create the row for a user or assistant message

Event	UI responsibility
<code>message_update</code>	Refresh the partial assistant, thinking, and tool-call display
<code>message_end</code>	Finalize the current message component
<code>tool_execution_start</code>	Start or recover the tool row created from the assistant's tool call
<code>tool_execution_update</code>	Refresh a streaming partial tool result
<code>tool_execution_end</code>	Finalize the tool result and remove it from pending bookkeeping
<code>agent_end</code>	Clear activity state and any leftover streaming/tool components

After tool execution, `message_start`/`message_end` carry the tool-result message, and another assistant turn may repeat the message lifecycle before `agent_end`. The UI works from events rather than polling the model. This preserves the same visual behavior whether an assistant emits text, thinking, tool calls, errors, or a synthetic retry/compaction message.

11.9 Tools in the transcript

Tool rows are composed by the interactive components around a tool definition's `renderCall` and `renderResult` slots. The context passed to those renderers includes current args, a row-local mutable state object, optional last component, partial/final flags, expanded state, show-images preference, cwd, timing, and an `invalidate()` method.

This produces a useful layering:

```
tool definition knows how to render its call/result data
interactive mode knows where the row belongs in chat and when it updates
TUI knows how to layout/diff-render the resulting components
```

For an extension tool, return a compact default component and make details available when expanded. For a high-frequency partial result, reuse `context.lastComponent` when it is already the same component class you would create and can be updated in place (for example a `Text` you can `setText` on). That reuse is the criterion for “safe”: same render slot, compatible instance, mutations only. Do not cast and mutate an unrelated previous component, and do not invent a new large component tree on every stream update when an existing instance can carry the change.

11.10 Extension UI surface

`ctx.ui` is an adapter owned by `InteractiveMode`. It gives extensions a controlled surface rather than direct access to private chat/editor fields:

API family	Examples
Dialogs	<code>select</code> , <code>confirm</code> , <code>input</code> , <code>editor</code>

API family	Examples
Notices/status	<code>notify</code> , <code>setStatus</code> , working message/indicator controls
Editor/widgets	<code>setEditorText</code> , <code>pasteToEditor</code> , <code>setWidget</code> , custom editor
Transcript/rendering	message/entry renderers, tool expansion controls
Look and completion	themes, autocomplete provider, custom footer
Complex interaction	<code>custom(factory)</code> and experimental overlays

`ctx.mode` identifies the host surface, while `ctx.hasUI` says whether a dialog-capable adapter is actually bound:

<code>ctx.mode</code>	Host surface	Normal <code>ctx.hasUI</code> behavior
"tui"	Interactive terminal	true ; dialogs and terminal components are available
"rpc"	Persistent protocol client	true ; supported dialogs/notices map to RPC messages
"json"	Machine-readable event output	false ; UI calls use noninteractive no-op/default behavior
"print"	One-shot text output	false ; UI calls use noninteractive no-op/default behavior

Use `ctx.hasUI` before operations that require a user response. Separately, require `ctx.mode === "tui"` for terminal-only behavior such as custom TUI components. An RPC UI is dialog-capable, but it is not an equivalent terminal component surface.

11.11 Custom components and focus

`ctx.ui.custom()` temporarily replaces the editor with a factory that receives the TUI instance, current theme, a keybindings manager, and a `done(value)` callback. The component handles input until it calls `done`.

```
const answer = await ctx.ui.custom<boolean>((_tui, _theme, keybindings, done) => {
  // Text(text, paddingX, paddingY): cell padding left/right then top/bottom
  const component = new Text("Enter confirms; Escape cancels", 1, 1);
  component.onKey = (key) => {
    if (keybindings.matches(key, "tui.select.confirm")) done(true);
    if (keybindings.matches(key, "tui.select.cancel")) done(false);
    return true;
  };
  return component;
});
```

The example uses named bindings, so a user who rebinds `confirm/cancel` receives the expected behavior. More complex overlays have focus/visibility APIs; use them only when an editor replacement/dialog cannot express the interaction.

11.12 Autocomplete composition

The TUI supplies autocomplete interfaces and a combined provider. Pi's interactive editor layers slash-command and path completion. An extension can add a provider by receiving the current provider and delegating when its own trigger syntax does not apply.

The reliable pattern is:

1. Inspect text before the cursor.
2. If extension syntax matches, return an explicit prefix and candidate list.
3. Otherwise call the previous provider.
4. Delegate completion application unless custom insertion is necessary.
5. Preserve existing file-completion decisions unless deliberately changing it.

This prevents an extension's `#issue` or `$variable` completion from disabling normal slash/path completion elsewhere in the line.

11.13 Themes and semantic styling

Theme functions provide semantic styles such as `accent`, `muted`, `dim`, `success`, `warning`, `error`, and `toolTitle`. Use `theme.fg(name, text)` plus style helpers like `theme.bold(text)` in renderer code.

Do not embed fixed ANSI colors in an extension renderer. A literal color may be unreadable in a user's light/custom theme and cannot participate in Pi's visual language. Semantic keys let an extension look native across themes.

11.14 Images and rendered width

Terminal width is measured in display cells, not JavaScript string length. Wide CJK characters, ANSI escape sequences, hyperlinks, and image rows make naive `text.length` calculations wrong. The TUI exports utilities such as `visibleWidth`, `truncateToWidth`, and ANSI-aware wrapping. Use them in custom components rather than slicing strings manually.

Likewise, terminal image support is capability- and cell-size-dependent. Use the TUI `Image` component/terminal image helpers rather than trying to print a provider image's base64 data as text.

11.15 Debugging visual issues

When an interactive feature is wrong, isolate its layer:

1. Capture the agent/session event sequence; verify data before rendering.
2. Inspect the tool/message renderer output as a component.
3. Check available width and ANSI-aware visible width.
4. Confirm the component invalidates after its state changes.
5. Test narrow terminal widths and long/CJK/ANSI-rich content.
6. Test focus and cancel paths for custom UI.
7. Use `/debug` for rendered TUI lines and the last model messages.

The correct fix is usually at the narrowest layer that owns the defect: a tool renderer for tool presentation, interactive mode for chat composition, or the TUI package for a general terminal layout/input primitive.

For a practical next step, follow [Tour 5: Follow one streamed token to the terminal](#), then capture the event/render trace described there while running `./pi-test.sh`.

12 Customization patterns and contributor workflow

This chapter turns the architecture into decisions. It also calls out the mistakes that most often create an extension which appears to work once but fails on reload, in parallel tool mode, or outside the interactive terminals.

12.1 Start with the decision, not the API

Use **configuration** for a stable value, an **extension** for behavior that reacts to events or needs Pi APIs, and a **core change** only when the behavior must be shared by every host. For example, changing a model default is configuration; asking for confirmation before `bash` is an extension; changing how every tool batch is scheduled belongs in the agent core.

The examples in this chapter are deliberately independent. Read the safety gate first, then the context and persistence patterns, and use the final verification ladder after adapting one pattern to your feature.

12.2 Configuration before code

Pi configuration is layered. By default, the coding-agent package uses `~/.pi/agent` as its global agent directory and `.pi` for project-local resources. The `piConfig.configDir` package setting makes the config directory rebrandable, so extension code should import `CONFIG_DIR_NAME` when constructing a project path instead of hardcoding `.pi`:

```
import { CONFIG_DIR_NAME } from "@earendil-works/pi-coding-agent";
import { join } from "node:path";

const projectConfigPath = join(ctx.cwd, CONFIG_DIR_NAME, "my-extension.json");
```

Use settings for stable choices such as providers, model selection, themes, keybindings, trusted packages, and resource locations. Use an extension when a choice must react to events, require a tool, retain state, or manipulate UI.

Project configuration and resources are an execution boundary. Before trust is resolved, Pi loads only context files plus global and explicitly supplied CLI extensions. Approving trust allows protected project settings and resources to load; declining trust skips them. Trust is an input-loading decision, not a sandbox for subsequent tool execution. A global extension should call `ctx.isProjectTrusted()` before honoring project-local configuration of its own.

12.3 Pattern: a safety gate around a built-in tool

This is appropriate for a team policy that requires confirmation before a destructive command. It observes the existing `bash` tool instead of replacing all shell behavior.

```
import { isToolCallEventType, type ExtensionAPI } from "@earendil-works/pi-coding-agent";

export default function register(pi: ExtensionAPI) {
  pi.on("tool_call", async (event, ctx) => {
    if (!isToolCallEventType("bash", event)) return;
    if (!event.input.command.includes("rm -rf")) return;

    // Noninteractive hosts cannot show a dialog; choose a safe default first.
    if (!ctx.hasUI) {
      return { block: true, reason: "Dangerous command blocked (no UI for confirmation)" };
    }

    const allowed = await ctx.ui.confirm("Dangerous command", "Allow rm -rf?");
    if (!allowed) return { block: true, reason: "Blocked by user" };
  });
}
```

The call is already schema-validated when the hook runs. The `isToolCallEventType` guard provides a typed `event.input` for a built-in tool. This pattern must handle non-TUI modes deliberately: `ctx.hasUI` can be false, so check it before `ctx.ui.confirm()` and choose a safe default or noninteractive policy. The shipped `permission-gate.ts` example follows the same order.

Do not mutate a tool's input merely to implement policy when blocking is more honest. If you do mutate, remember no second schema validation occurs.

12.4 Pattern: inject focused context, not a second system

`before_agent_start` is the correct point to add per-prompt instructions. It can add a custom message and/or replace the chained system prompt. The custom message is stored in the session and reaches the LLM; a notification is not.

```
import type { ExtensionAPI } from "@earendil-works/pi-coding-agent";

export default function register(pi: ExtensionAPI) {
  pi.on("before_agent_start", async (event) => {
    if (!event.prompt.startsWith("review ")) return;
    return {
      message: {
        customType: "review-mode",
        content: "Prioritize correctness, regressions, and concise findings.",
        display: true,
      },
    };
  });
}
```

```

    };
  });
}

```

Here, `display: true` shows the injected custom message in the transcript; the message is sent to the model whether or not it is displayed.

Multiple handlers see a chained system prompt. Prefer appending a narrow, conditional instruction over replacing the entire system prompt; replacement can accidentally discard skills, tools, context files, and user-provided instructions. For a message-level context transformation before *every* provider request, use the `context` event instead; see [the extension event reference](#) for when `context` is the right boundary versus prompt-scoped or provider hooks.

12.5 Pattern: state that survives restart and branching

For state derived from tool actions, put a serializable snapshot in each tool result's `details`. In `session_start`, initialize empty in-memory state. Scan `ctx.sessionManager.getBranch()` from root to the current leaf, validate matching tool-result entries, and apply their snapshots in order so the last one becomes the current state. `getBranch()` excludes sibling-branch entries, so this rebuild restores only the state associated with the selected branch. Rebuild on both `session_start` and `session_tree` so reload, resume, fork, and `/tree` navigation stay consistent:

```

function rebuildState(ctx: ExtensionContext) {
  state = emptyState();
  for (const entry of ctx.sessionManager.getBranch()) {
    if (entry.type !== "message") continue;
    const msg = entry.message;
    if (msg.role !== "toolResult" || msg.toolName !== "my_tool") continue;
    const snapshot = msg.details as MyState | undefined;
    if (snapshot) state = snapshot;
  }
}

pi.on("session_start", async (_event, ctx) => rebuildState(ctx));
pi.on("session_tree", async (_event, ctx) => rebuildState(ctx));

```

tool result details -> durable session entry -> branch reload -> extension rebuilds state

The shipped `todo.ts` extension implements this rebuild loop end to end.

For state that should be visible in the transcript but never sent to the model, use `pi.appendEntry("my-state", data)` and `pi.registerEntryRenderer("my-state", renderer)`. Do not use `sendMessage` for private bookkeeping: custom messages are model context.

For a read-modify-write operation, serialize mutations of the same file while leaving different files concurrent. `withFileMutationQueue` is exported from `@earendil-works/pi-coding-agent`:

```
import { withFileMutationQueue } from "@earendil-works/pi-coding-agent";
import { readFile, writeFile } from "node:fs/promises";

await withFileMutationQueue(filePath, async () => {
  const current = await readFile(filePath, "utf8");
  await writeFile(filePath, update(current));
});
```

12.6 Pattern: dynamic tools and active-tool policy

`pi.registerTool()` may be called after startup. Pi refreshes it immediately for the current session; it appears in `pi.getAllTools()` and can be enabled with `pi.setActiveTools()`. To avoid accidentally removing useful tools, start from the current selection:

```
const active = pi.getActiveTools();
pi.setActiveTools([...new Set([...active, "my_tool"])]);
```

Changing active tools affects the next turn and rebuilds the system prompt. `setActiveTools(["read", "bash"])` is a real policy change, not just a UI filter. Use it with a command/shortcut that clearly signals the new mode.

Command handlers receive `ExtensionCommandContext`: an `ExtensionContext` with user-initiated session controls such as `newSession`, `fork`, `navigateTree`, `switchSession`, `reload`, and `waitForIdle`. Put session replacement or navigation in a command handler. A normal event handler can run within the active agent loop, while these operations replace contexts and must be coordinated from an explicit user command; the old context is stale afterward.

When tools and session behavior are not enough, the next customization boundary is the model provider.

12.7 Pattern: provider customization

Use `pi.registerProvider()` to add a proxy, endpoint, model catalog, OAuth flow, or nonstandard stream implementation. Use a provider hook only for a small cross-cutting change such as tracing headers or payload inspection.

An extension factory has the shape `(pi: ExtensionAPI) => void | Promise<void>`. Use its `async` form when the provider must query a local model service before registration:

```
import type { ExtensionAPI, ExtensionFactory } from "@earendil-works/pi-coding-agent";

// Replace with a real local discovery call; shape is what registerProvider expects.
async function discoverLocalModels() {
  return [
    {
      id: "local-model",
```

```

    name: "Local Model",
    reasoning: false,
    input: ["text"] as const,
    cost: { input: 0, output: 0, cacheRead: 0, cacheWrite: 0 },
    contextWindow: 128000,
    maxTokens: 8192,
  },
];
}

const register: ExtensionFactory = async (pi: ExtensionAPI): Promise<void> => {
  const models = await discoverLocalModels();
  pi.registerProvider("local", {
    baseUrl: "http://localhost:11434/v1",
    api: "openai-completions",
    models,
  });
};

export default register;

```

Pi awaits the factory. Provider registrations made during extension loading are queued until the model registry is available, then applied before startup operations such as model listing continue. Registration from a later command takes effect immediately, but it cannot make the model available to startup operations that have already completed.

Provider configuration has security impact. Environment-variable references for `apiKey` keep secrets out of source. Avoid logging headers, payloads, or `ctx.getSystemPromptOptions()` because they can contain credentials, context files, skills, and user instructions.

12.8 Pattern: custom TUI that still works outside the TUI

Use `ctx.ui.setStatus` for a small persistent footer line, `setWidget` for editor-adjacent information, and renderers for transcript content. Use `ctx.ui.custom()` only when you need an interactive component with its own keyboard behavior.

An extension may run in four modes: `"tui"`, `"rpc"`, `"json"`, and `"print"`. Dialogs and custom component behavior do not have equivalent semantics in every mode. Design a noninteractive fallback first, then add an interactive dialog:

```

if (!ctx.hasUI) {
  return { block: true, reason: "This action requires interactive approval" };
}

```

For custom editors and UI components, use injected keybinding managers and named keybinding IDs. Do not hardcode raw control-key tests; that bypasses user configuration.

12.9 Pattern: package a team customization

An extension package can bundle extensions, skills, prompts, and themes. Put runtime dependencies in `dependencies`; Pi package installation uses a production-style install. Declare Pi core libraries as documented peer dependencies. Share a versioned package with explicit `pi` manifest paths when the team needs repeatability, rather than copying a large extension into every repository.

Use a project package for project-specific behavior only after the team trusts the project folder (or a saved trusted parent). That decision permits Pi to load project settings and resources, install missing packages configured by the project, and execute project extensions, including package-managed extensions. Use a global package for behavior independent of the current project. The scope determines both discovery and the security boundary.

12.10 Common mistakes and the correct boundary

Mistake	Why it fails	Better approach
Start a file watcher in the extension factory	Factory may run for a non-session command	Start in <code>session_start</code> ; close in <code>session_shutdown</code>
Keep all extension state in a closure	Reload/resume/fork loses it	Rebuild from session entries or tool <code>details</code>
Call session replacement from a normal event	It can run inside the active agent loop and leave stale contexts	Use a command and <code>ExtensionCommandContext</code>
Use <code>sendMessage</code> for private metadata	The model receives it	Use <code>appendEntry</code> and an entry renderer
Ignore <code>AbortSignal</code>	Esc cannot stop your nested work	Forward <code>ctx.signal</code> /tool signal
Mutate files in a custom tool without a queue	Parallel sibling calls can lose writes	Use <code>withFileMutationQueue</code> around read-modify-write
Override <code>read</code> or <code>bash</code> with a different result shape	Built-in UI/session behavior expects details	Preserve the documented built-in contract
Assume tool hooks observe sibling results	Parallel tools finish independently	Persist/coordinate after the batch or avoid the dependency
Assume <code>agent_end</code> means completely idle	<code>agent_end</code> marks one agent loop ending; retry, compaction, or a queued continuation may still run	Use <code>agent_settled</code> , which fires only after no automatic retry, compaction, or queued continuation remains, or command <code>waitForIdle()</code>
Test only <code>pi -e</code>	It does not prove discovery/reload behavior	Test from auto-discovery plus <code>/reload</code>

12.11 A practical verification ladder

Use evidence appropriate to what you changed:

1. Type-check the extension with the same TypeScript version/API contracts used by Pi, especially tool parameters and event return values. From an extension package with a `tsconfig.json`, run `npx tsc --noEmit -p tsconfig.json`.
2. Load it with `pi -e path/to/extension.ts` to isolate load errors.
3. Move it to an auto-discovered location and verify `session_start`.
4. Exercise the happy path and the relevant failure/deny/cancel path.
5. Run `/reload`, then repeat. Confirm no duplicate resource, stale closure, or leaked status remains.
6. For persistence, start a new process, resume/fork a session, and verify the intended branch state is restored.
7. For tools, test a malformed call/schema failure and a cancellation. For mutating tools, test concurrent sibling calls if applicable.
8. If you change repository code, run the repository-prescribed relevant check rather than a real-provider test. The coding-agent suite uses a faux provider under `packages/coding-agent/test/suite/harner` for deterministic tests.

The repository ships extension examples in `packages/coding-agent/examples/extensions/`. Start with the smallest match:

- `hello.ts`: a tool
- `permission-gate.ts`: a safety gate
- `input-transform.ts`: input routing
- `entry-renderer.ts`: transcript-only state
- `dynamic-tools.ts`: runtime tool changes
- `subagent/`: a larger composition

13 Testing, debugging, and contribution workflow

This chapter turns the system map from [Architecture](#) into a practical workflow for reproducing failures, choosing tests, and preparing a change for review. Pi’s repository rules distinguish code checks, targeted tests, interactive testing, and real-provider behavior. Use the narrowest validation path that exercises the behavior. A local state transition does not require a provider call.

Repository-wide contribution rules live in `AGENTS.md`. Coding-agent-specific development and extension guidance lives in `packages/coding-agent/docs/development.md` and `packages/coding-agent/docs/extension.md`. For runtime failure ownership and recovery behavior, see [Failure and recovery](#).

13.1 Three kinds of evidence

A **check** asks whether code satisfies static rules, a **test** asks whether a repeatable behavior still works, and a **smoke test** exercises a real host such as the interactive TUI. They answer different questions. For a tool change, start with a focused test for success and failure, then use the TUI only if the rendering or input path also changed.

When debugging, first write the expected boundary in one sentence: “the extension should block this validated `bash` call,” or “the model should receive this converted message.” Then inspect the first boundary where reality differs. The tables below turn that sentence into a repeatable workflow.

13.2 Localize the failure first

For a report such as “Pi ignored my extension” or “a tool result rendered wrong,” identify the first subsystem where observed behavior diverges from the expected path before changing code:

Symptom	Inspect first (in order)
Extension is absent	1) trust decision → 2) resource loader discovery → 3) factory load error
Command runs but prompt is unchanged	1) extension command event → 2) input handling → 3) before-agent path
Model gets wrong messages	1) session context → 2) <code>context</code> hook → 3) <code>convertToLlm</code> → 4) provider payload
Tool has invalid args	1) schema → 2) <code>prepareArguments</code> → 3) model tool-call block
Tool output is lost	1) truncation → 2) tool result → 3) parallel ordering → 4) renderer
Provider call fails	1) model registry/auth → 2) adapter response → 3) retry configuration

Symptom	Inspect first (in order)
UI is stale or clipped	1) session event → 2) renderer component → 3) TUI invalidation/width
Resume/fork loses state	1) session branch/context → 2) extension reconstruction → 3) stale replacement closure

This map prevents wasted work: editing the TUI cannot fix an agent that never emitted a message event, and editing a provider adapter cannot fix an extension blocked by project trust.

13.3 Static checking versus tests

After code changes, repository policy calls for `npm run check`. It formats and checks the workspace, verifies pinned dependencies/import rules/shrinkwrap state, and runs the root TypeScript check. It is a broad code-quality gate, not the test suite.

`npm run build` only compiles packages; it is not a test suite. Tests verify behavioral expectations. The full Vitest suite can activate end-to-end tests when provider or authentication environment variables are present. To avoid that and get broad non-LLM coverage, use `./test.sh` from the repository root. For a targeted coding-agent test, run the Vitest CLI from the package root as described by `AGENTS.md`.

13.4 Test at the correct package

Change area	First test location/type
Provider serialization/stream parsing	<code>packages/ai/test/</code> unit/regression tests
Agent tool scheduling/message events	<code>packages/agent</code> tests or harness tests
TUI width, key, component behavior	<code>packages/tui/test/</code>
Coding-agent sessions/config/extensions	<code>packages/coding-agent/test/</code>
Agent behavior without a paid provider	<code>packages/coding-agent/test/suite/</code> with faux provider

The coding-agent suite harness is intentional. For tests under `packages/coding-agent/test/suite/`, use `test/suite/harness.ts` and the faux provider. Real model keys introduce cost, nondeterminism, rate limits, and a different failure surface than the behavior being tested.

13.5 High-value test cases by subsystem

13.5.1 Extensions

- Discovery from global/project/CLI paths and trust gating.
- Async factory completion before provider/model use.
- Correct event ordering and return semantics.

- Tool block/allow behavior for policy hooks.
- Reload cleanup and state reconstruction.
- Non-interactive fallback when the extension context reports `ctx.hasUI === false`.
- Session replacement with a fresh `withSession` context.

13.5.2 Tools

- Schema validation and legacy `prepareArguments` shape.
- Success, thrown error, abort, and timeout.
- Bounded output and continuation/full-output notices.
- Parallel sibling behavior and same-file mutation queue.
- Renderer partial/final/expanded behavior.

13.5.3 Sessions and compaction

- JSONL loading/migration and malformed input.
- Tree path selection and label/session-info changes.
- Context rebuild with compaction and branch summaries.
- Model/thinking restoration.
- Resume/fork/new runtime cleanup and extension lifecycle.

13.5.4 Provider/model registry

- Custom models/overrides merge semantics.
- Auth source precedence and command/env configuration.
- Header merging and `authHeader` behavior.
- Dynamic register/unregister restoration.
- Compatibility flags for a protocol-specific regression.

13.6 Interactive TUI smoke testing

`AGENTS.md` documents a controlled tmux pattern for testing the interactive application. The important principle is to use a detached terminal with known dimensions, capture the pane after startup, send text/keys, then cleanly kill the session. This produces a repeatable artifact for editor, command, streaming, and rendering behavior without relying on the developer's current terminal state.

For a UI reproduction, record dimensions, theme/keybindings, exact input and keys, model/streaming state, and captured lines or a `/debug` log. Screenshots alone often omit what is needed to reproduce a visual bug.

13.7 /debug and diagnostic data

The hidden `/debug` command writes `~/.pi/agent/pi-debug.log` with rendered TUI lines containing ANSI codes and the last messages sent to the LLM. Comparing those two views can reveal whether a mismatch arose before or during rendering. The log may contain sensitive prompts, file contents, and tool results, so review and redact it before sharing any excerpt publicly.

Extension diagnostics should be equally disciplined. Log an event name, safe IDs, timing, and sanitized state—not API keys, auth headers, complete provider payloads, or full context-file content.

13.8 Error classification before retrying

Pi has several retry-like mechanisms with different ownership:

Failure/condition	Owner	Typical action
Invalid tool input or tool execution error	Agent loop/model	Return error tool result; model may recover
Transport/provider transient error	AgentSession/provider settings	Retry according to configured policy
Context overflow	AgentSession compaction policy	Compact and retry if applicable
User abort	Active run abort controller	Stop abort-aware work; do not auto-retry blindly
Extension hook error	Extension runner	Log; tool-call errors fail safe by blocking
Project not trusted	Trust manager	Do not load project code/resources

Misclassifying these leads to bad fixes. Retrying an invalid schema will not correct model arguments; compacting a missing API key is pointless.

13.9 Source navigation habits

The repository is large enough that broad search results can mislead. A useful sequence is to find a public symbol with `rg`, read its contract and focused call sites, identify which package owns the next step, and then read a nearby test or example. Change one subsystem at a time and add a regression where needed.

Examples are valuable for extensions. Start with `packages/coding-agent/examples/extensions/hello.ts`, then choose a focused example such as `permission-gate.ts`, `dynamic-tools.ts`, `entry-renderer.ts`, or `custom-provider-*` instead of copying a large multi-feature extension.

13.10 Git hygiene in a shared worktree

Multiple concurrent agent sessions may edit different files in the same working tree. Stage only explicit paths you changed in your session; never use broad staging commands. Inspect `git status` before staging/committing, and do not reset, clean, stash, or check out unrelated work. `AGENTS.md` also prohibits commits unless the user asks.

For generated model metadata, change the generator source rather than editing `packages/ai/src/models.generate` by hand. That policy preserves the link between catalog data and its generation process.

13.11 Documentation as a tested interface

This guide is source-grounded documentation. Keep its code examples subject to the same review questions as implementation:

1. Does every exported symbol exist at the documented package path?
2. Does the example respect actual `async/cancellation` semantics?
3. Does a lifecycle claim match the runner/agent implementation?
4. Does a configuration example state `merge/replace/trust` semantics?
5. Does a visual description name the responsible renderer or component?

When an API changes, update its focused chapter and the prompt lifecycle chapter together. The lifecycle is the integration test for the book: it reveals contradictions between isolated component explanations.

13.12 Pre-review checklist

Before requesting review for a code or extension change:

1. Reproduce the original behavior with a minimal scenario.
2. State the package and public contract that own the fix.
3. Add/run the narrowest regression test or controlled interactive proof.
4. Run the repository-required static check for code changes.
5. Check trust/security implications of new configuration or package loading.
6. Test cancellation, error, and reload/replacement paths where relevant.
7. Inspect diffs for generated/lockfile/dependency changes you did not intend.

This workflow keeps extension changes reviewable: each change has a clear runtime owner and a validation step that produces concrete evidence.

14 Guided source tours and architecture exercises

The earlier chapters describe Pi's architecture. This chapter turns that map into repeatable reading exercises. Each tour has a question, an entry point, the important control-flow boundary, and a concrete outcome. Work through them with the source open; the purpose is to learn how the modules connect, not to memorize file names. All paths in this chapter are relative to the repository root.

14.1 How to do a source tour

Each tour has four parts: a **question**, a starting file, a short list of boundaries to trace, and an **outcome** you can explain in your own words. Before opening the source, predict the path. Then verify the prediction with one symbol search and one nearby test or example. For instance, before Tour 1 you should predict that `cli.ts` parses arguments and hands control to a mode; the tour shows where that prediction is right and where it is incomplete.

14.2 Tour 1: What launches when pi starts?

Question: Which module turns process arguments into an interactive agent?

Start at `packages/coding-agent/src/cli.ts`. It assigns `process.title` to `APP_NAME`, sets `process.env.PI_CODING_AGENT` to `"true"`, configures the HTTP dispatcher early, and calls `main(process.argv.slice(2))`. Then follow `packages/coding-agent/src/main.ts`.

Trace these checkpoints:

1. `parseArgs` turns raw argv into a typed argument object.
2. `resolveAppMode` selects RPC, JSON, print, or interactive behavior from explicit flags and TTY state.
3. Startup settings/trust/session inputs are prepared.
4. `createAgentSessionRuntime` creates services and a session.
5. `InteractiveMode`, `runRpcMode`, or `runPrintMode` receives that runtime.

Outcome: You should be able to answer why a piped stdin invocation may use print mode and why interactive mode can keep the caller's cwd while resolving Pi's own assets from its package configuration.

14.3 Tour 2: Why can a session change cwd safely?

Question: What is recreated when `/resume` opens a session from another working directory?

Start at `packages/coding-agent/src/core/agent-session-runtime.ts`. Read `switchSession`, then the shared `teardown/apply` helpers. Follow the `createRuntime` factory passed from `packages/coding-agent/src/main.ts`.

Draw this before reading implementation:

```
old session + old cwd services
-> session shutdown/dispose
-> SessionManager.open(target)
-> target cwd services
-> target AgentSession
-> rebind interactive UI
```

Check which objects are cwd-bound: `SettingsManager`, `DefaultResourceLoader`, and the runtime's session/model configuration all depend on the effective cwd. The shared global auth storage can remain reusable, but its resolution is still consulted by the new model registry.

Outcome: You should understand why extension code must not reuse an old session context after a replacement and why `withSession` exists.

14.4 Tour 3: How do resources reach a system prompt?

Question: Where do `AGENTS.md`, skills, prompts, and tool guidance join?

Start at `packages/coding-agent/src/core/resource-loader.ts`, then find callers of `resourceLoader.getSystem`, `getSkills`, and `getAgentsFiles` in `packages/coding-agent/src/core/agent-session.ts`. Read `_rebuildSystemPrompt` and then `packages/coding-agent/src/core/system-prompt.ts`.

Write the data as a structured object before it becomes text:

```
cwd
context files
skills
custom/append prompt
selected tools
tool snippets
tool guidelines
-> BuildSystemPromptOptions -> string
```

Now inspect `setActiveToolsByName`. It rebuilds the prompt because a changed tool selection must change the instructions available to the model.

Outcome: You can explain why adding a tool has both a runtime registry effect and a prompt/context effect.

14.5 Tour 4: What happens to a user message before a provider sees it?

Question: Which transformations can change user text?

Read `AgentSession.prompt` in `packages/coding-agent/src/core/agent-session.ts` top to bottom. List each return/branch in order:

1. Extension command resolution.
2. `input` event and possible transform/handled result.
3. Skill/template expansion.
4. Streaming steer/follow-up queue behavior.
5. Auth/model checks and possible compaction.
6. User/custom message construction.
7. `before_agent_start` message/system-prompt modifications.
8. `_runAgentPrompt`.

Then read the `Agent` call path and `streamAssistantResponse` to see the later context transform plus `convertToLlm` boundary.

Outcome: You can identify where `input`, `before_agent_start`, `context`, and `before_provider_request` run, then narrow your investigation to the hook that sees the representation you need to change.

14.6 Tour 5: Follow one streamed token to the terminal

Question: How does a provider delta become a changed terminal row?

This tour is the concrete entry for the Terminal UI and interactive mode). Start in `packages/agent/src/agent-loop` at `streamAssistantResponse`. Find how `text_delta`, `thinking_delta`, and `toolcall_delta` replace the partial assistant message and emit `message_update`. Then walk this bridge without inventing extra stages:

1. `Agent.processEvents` in `packages/agent/src/agent.ts` stores the partial assistant message and awaits registered listeners.
2. `AgentSession._handleAgentEvent` in `packages/coding-agent/src/core/agent-session.ts` re-emits a session `message_update` (including extension handlers) and fans it out through `AgentSession.subscribe`.
3. `InteractiveMode.subscribeToAgent` in `packages/coding-agent/src/modes/interactive/interactive-mode.ts` calls `handleEvent`.
4. The `message_update` branch updates `AssistantMessageComponent` (`packages/coding-agent/src/modes/interactive/assistant-message.ts`) via `updateContent`, creates or refreshes `ToolExecutionComponent` rows when tool-call content appears (`packages/coding-agent/src/modes/interactive/components/tool-execution.ts`), and calls `ui.requestRender()` so the TUI package diffs the terminal frame.

Do not try to read `interactive-mode.ts` linearly; search for the event type and the component method it calls.

Outcome: You can locate a rendering bug as one of four categories: provider event normalization, agent partial state, session event bridge, or interactive component update.

14.7 Tour 6: Follow one tool call through validation

Question: Why cannot an extension tool execute arbitrary model JSON?

Start at `executeToolCalls` in `packages/agent/src/agent-loop.ts`. Follow one call through:

```
assistant content block
-> active tool lookup
-> prepareArguments
-> validateToolArguments
-> beforeToolCall
-> execute
-> afterToolCall
-> ToolResultMessage
```

Next open `packages/coding-agent/examples/extensions/hello.ts` and identify where its `TypeBox` schema becomes both a static parameter type and a runtime validator. Compare it with `packages/coding-agent/src/core/tools/read.ts`, which adds operations and renderers but retains the same fundamental definition shape.

Outcome: You can explain why a schema belongs in an extension even when TypeScript already knows the desired params type.

14.8 Tour 7: Study parallel tool execution without guessing

Question: What ordering is guaranteed when an assistant asks for two tools?

Read `executeToolCallsParallel` and contrast it with the sequential path. Write a timeline for two calls A and B:

```
start(A), validate/gate(A)
start(B), validate/gate(B)
execute(A) and execute(B) concurrently
end events in completion order
final ToolResultMessage(A), ToolResultMessage(B) in source order
```

Compare that timeline with the documented guarantees in `packages/coding-agent/docs/extensions.md` under `tool_execution_start` / `tool_execution_update` / `tool_execution_end`: start events are source-ordered during preflight, update/end events can finish in completion order, and final `toolResult` messages are still emitted in source order. Then inspect `packages/coding-agent/src/core/tools/fileMutationQueue` (with `FileMutationQueue`) to see how same-file writes gain a narrower serialization guarantee without making unrelated tools sequential.

Outcome: You can design an extension tool that works correctly under the default mode rather than accidentally relying on timing.

14.9 Tour 8: Explain a model request without opening a network trace

Question: Where do key, headers, model metadata, and payload come from?

First open `packages/coding-agent/src/core/sdk.ts`. This module is the coding-agent bridge from model-registry configuration to the AI package's streaming API. In its `stream` function, trace how it asks the model registry for auth, combines environment/header settings, emits `before_provider_headers`, and then calls `streamSimple`.

Follow into `packages/ai/src/compat.ts`. It chooses a built-in model dispatcher or resolves the API provider by `model.api`. Open one adapter such as `packages/ai/src/api/openai-completions.ts` or `packages/ai/src/api/anthropic-messages.ts` and identify its serialization and event-stream parser entry points.

Outcome: You can distinguish configuration bugs (registry), protocol bugs (adapter), and runtime-loop bugs (agent) without a speculative rewrite.

14.10 Tour 9: Trace a resume-context discrepancy

Question: Why did a resumed model refer to an old branch or summary?

Open `packages/coding-agent/src/core/session-manager.ts` and read `buildContextEntries` plus `buildSessionContext`. Use a small hand-drawn tree with a compaction entry and a branch summary. Trace the active leaf backwards, then identify which entries become context messages and which are metadata only.

Cross-check with `packages/coding-agent/docs/session-format.md`. Use `getBranch`, `buildContextEntries`, and `buildSessionContext` from an extension command to inspect an actual session instead of parsing JSONL manually at first.

Outcome: You can prove whether a context difference is persistence policy, compaction, branch navigation, or an extension's reconstruction mistake.

14.11 Tour 10: Add a narrowly scoped extension feature

Question: How do you add a feature without coupling to internals?

Pick one small behavior such as a status widget, a custom command, or a read-only tool. Start from the closest example. Name the lifecycle event/API that owns it, then write a one-file extension under a global auto-discovery directory. Verify startup, action, `/reload`, and noninteractive behavior. `/reload` reloads keybindings, extensions, skills, prompts, themes, and context files without quitting the process; treat it as a full runtime/resource refresh (`AgentSession.reload`), so do not reuse a pre-reload extension context.

Before writing code, classify the behavior by the narrowest public seam that owns it. Use this table to make that decision, then confirm the selected API in the extension documentation or nearest example:

Desired result	Narrow public seam
Ask model to use a new operation	<code>registerTool</code>
Let user invoke a task	<code>registerCommand</code>
Block/mutate a tool call	<code>on("tool_call")</code>
Add context per user prompt	<code>on("before_agent_start")</code>
Change provider endpoint/models	<code>registerProvider</code>
Store private session state	<code>appendEntry</code>
Render a custom persistent record	<code>registerEntryRenderer</code>
Add a small always-visible indicator	<code>ctx.ui.setStatus</code> or <code>widget</code>

Outcome: You practice the extension API as a set of explicit seams rather than as permission to reach into `InteractiveMode` or `AgentSession` private state.

14.12 A weekly reading routine for new contributors

The terminal user interface (TUI) is the interactive layer that turns session events into terminal components and rendered rows. Treat it as its own reading boundary rather than as another name for the entire coding-agent package.

Large codebases become legible through repeated, bounded questions. A practical routine is:

Session	Focus	Deliverable to yourself
1	CLI startup and services	One startup diagram with package boundaries
2	Agent loop and messages	One provider/tool turn timeline
3	Sessions/compaction	One active-branch/context sketch
4	Built-in tools	One tool contract comparison
5	Extensions (Tour 10)	A working one-file extension plus <code>/reload</code> and noninteractive checks
6	TUI (Tour 5)	Trace one <code>message_update</code> from <code>streamAssistantResponse</code> through <code>InteractiveMode.handleEvent</code> to <code>AssistantMessageComponent.updateContent</code>
7	Provider config (Tour 8)	A local/custom provider configuration review

The guide is intentionally organized so each exercise has a corresponding chapter. Revisit the prompt lifecycle after every tour: it is the shortest integration model of the entire project.

15 Extension event and API reference

This chapter is a practical reference for choosing extension hooks. It does not replace the source types in `packages/coding-agent/src/core/extensions/types.ts` or the exhaustive `packages/coding-agent/docs/extensions.md`; use those when writing code. Its goal is to make event choice, return value, context, and lifecycle safety explicit before implementation.

15.1 Use this chapter as an index

An **event** is a notification with a payload and a timing guarantee. An event is **observation-only** when a handler can inspect it but cannot change the operation. A **return value** is a handler's supported way to continue, transform, or block work. Start with the first table: choose the latest event that still contains the representation you need, then read that event's subsection for ordering, mode, and error behavior.

For example, use `input` to change submitted text, `context` to change the message view before every provider request, and `tool_call` to gate validated tool arguments. Those three examples look similar from the UI but occur at different boundaries.

15.2 Choose an event by the representation you need

The same user action appears in several representations. Select the latest boundary that still contains the information you need, because later hooks are more specific and less likely to disturb unrelated behavior. The rows below follow the normal prompt/turn pipeline from submitted text to tool output; `user_bash` and durable session operations are separate paths shown last.

Need to inspect/change	Event/API	Representation
Raw submitted text before skills/templates	<code>input</code>	Text/images plus source/delivery mode
Prompt-specific system context	<code>before_agent_start</code>	Expanded prompt and system-prompt options
Message history before every LLM call	<code>context</code>	Deep-copied agent messages
Final HTTP headers	<code>before_provider_headers</code>	Mutable header map
Provider wire payload	<code>before_provider_request</code>	Adapter-specific payload
Provider status/response headers	<code>after_provider_response</code>	HTTP metadata before stream consumption
Validated tool arguments	<code>tool_call</code>	Tool name and mutable input
Streaming/final tool execution row	<code>tool_execution_start</code> / <code>_update</code> / <code>_end</code>	Observation-only lifecycle payloads

Need to inspect/change	Event/API	Representation
Tool execution output (patchable)	<code>tool_result</code>	Content/details/ <code>isError</code> patch opportunity
Final tool transcript message	<code>message_start</code> / <code>message_end</code> with <code>role: "toolResult"</code>	Durable tool-result message after patching
User typed ! command	<code>user_bash</code>	User command and context-exclusion choice
Cancel or customize compaction	<code>session_before_compact</code>	Prep metadata; may cancel or supply summary
Observe completed compaction	<code>session_compact</code>	Saved compaction entry and source flags
Cancel or summarize tree navigation	<code>session_before_tree</code>	Branch preparation; may cancel or supply summary
Observe tree navigation	<code>session_tree</code>	New leaf, old leaf, and summary outcome
Initiate session replacement from an extension	Command context (<code>newSession</code> , <code>fork</code> , <code>navigateTree</code> , <code>switchSession</code> , <code>reload</code>)	User-triggered only; not ordinary event handlers

The rows describe different capabilities, not aliases. For example, a context filter changes Pi's model-message view, whereas provider hooks operate only at the serialized request boundary. Durable session hooks change neither unless their operation rebuilds the active context.

15.3 Startup and trust events

15.3.1 `project_trust`

This event occurs before Pi decides whether project-local dynamic resources may run. It is visible only to user/global and CLI-provided extensions, not project extensions. The handler must return a trust decision:

```
pi.on("project_trust", async (event, ctx) => {
  if (!ctx.hasUI) return { trusted: "undecided" };
  const yes = await ctx.ui.confirm("Trust project?", event.cwd);
  return yes ? { trusted: "yes", remember: true } : { trusted: "undecided" };
});
```

Return `"undecided"` when your extension does not own the policy. A yes/no decision suppresses later handlers and the built-in flow. Only use `remember` when it is appropriate to persist a global trust decision for this user.

15.3.2 resources_discover

This fires after `session_start` so an extension can return additional skill, prompt, and theme paths. It receives `reason: "startup" | "reload"`. It is a discovery hook, not an instruction to read arbitrary project paths without checking trust. Use the surrounding extension/session context's trust state for project-local behavior.

15.4 Session events

15.4.1 session_start and session_shutdown

Use `session_start` to initialize state that requires an active session: restore durable data, add UI status, start a watcher, or open a connection. Its reason identifies startup, reload, new, resume, or fork. Use `session_shutdown` to close that same resource. Cleanup must be idempotent because reload, replacement, and quit may all trigger it.

Do not start long-lived resources in the factory. A factory may run for a command that never creates a session, while session events make lifecycle ownership explicit.

15.4.2 session_before_switch and session_before_fork

These are cancellation gates. A switch handler sees whether the operation is `new` or `resume` plus an optional target path. A fork handler sees entry ID and position (`before` for fork or `at` for clone). Return `{ cancel: true }` to block a user action after a confirmation or policy check:

```
pi.on("session_before_switch", async (event, ctx) => {
  if (!ctx.hasUI) return;
  const ok = await ctx.ui.confirm(
    "Change session?",
    event.reason === "new" ? "Start a new session?" : "Open another session?",
  );
  if (!ok) return { cancel: true };
});
```

Do not call session-changing methods such as `newSession`, `fork`, `navigateTree`, `switchSession`, or `reload` from an ordinary event handler. If an extension must initiate one, use a command handler with the command-context APIs described later in this chapter.

15.4.3 Compaction and tree hooks

Compaction replaces older messages on the active branch in model context with a durable summary while retaining a recent suffix; it does not delete the underlying session entries. `session_before_compact` can cancel that operation or return a custom summary plus the required kept-entry/token metadata. `session_compact` observes the durable result and whether it came from an extension.

```

pi.on("session_before_compact", async (event, ctx) => {
  const { preparation, reason, willRetry } = event;
  // reason: "manual" | "threshold" | "overflow"
  // willRetry is true for overflow recovery that retries the aborted turn.

  if (reason === "manual" && !ctx.hasUI) {
    return { cancel: true };
  }

  return {
    compaction: {
      summary: `Kept from ${preparation.firstKeptEntryId}`,
      firstKeptEntryId: preparation.firstKeptEntryId,
      tokensBefore: preparation.tokensBefore,
    },
  };
});

pi.on("session_compact", (event) => {
  // event.compactionEntry, event.fromExtension, event.reason, event.willRetry
});

```

A session is a tree of durable entries linked by `id` and `parentId`; the current leaf selects the root-to-leaf branch used for context. `/tree` navigation moves that leaf within the same session file and may summarize the branch being left. `session_before_tree` can cancel or provide that branch summary; `session_tree` observes the selected leaf and summary outcome.

```

pi.on("session_before_tree", async (event) => {
  const { preparation } = event;
  // Summary is only used when the user requested one for this navigation.
  if (!preparation.userWantsSummary) return;
  return {
    summary: {
      summary: `Leaving ${preparation.oldLeafId} for ${preparation.targetId}`,
      details: {},
    },
  };
});

pi.on("session_tree", (event) => {
  // event.newLeafId, event.oldLeafId, event.summaryEntry, event.fromExtension
});

```

Use these hooks for policy or summary customization. Use `context` if the goal is only to hide/filter data for one provider call; a context hook does not change durable compaction or tree state.

15.4.4 Session information

`session_info_changed` fires after `/name` or `pi.setSessionName`. It is a good place to update a title/status integration. It is not a model message and does not require rebuilding the system prompt.

15.5 Input and agent-run events

15.5.1 input

Input handlers run after extension command lookup and before skill/template expansion. The result controls the next step:

Result	Effect
no result / <code>continue</code>	Leave input unchanged
<code>transform</code> with text/images	Pass modified input into later expansion
<code>handled</code>	Stop input processing; no agent run starts

Transforms chain in extension load order. Keep transforms simple and make source-aware decisions with `event.source` (`interactive`, `rpc`, or `extension`). During streaming, `event.streamingBehavior` distinguishes steer and follow-up queue delivery.

15.5.2 before_agent_start

This runs after normal input expansion but before the low-level agent loop. It is the best prompt-scoped seam. A result may add a custom persistent message and/or replace the chained system prompt. Later handlers receive system-prompt changes from earlier handlers.

The event exposes `systemPromptOptions`, including custom prompt, selected tools, snippets/guidelines, cwd, context files, and skills. Treat it as sensitive extension-local data. It may contain full instructions that should not be copied into command descriptions or telemetry.

15.5.3 agent_start, agent_end, and agent_settled

A low-level agent run begins at `agent_start`, contains one or more turns, and ends at `agent_end`. That end does not prove application idleness: coding-agent may start another run for an automatic retry, compact-and-retry recovery, or a queued continuation. `agent_settled` fires only when that outer processing cycle has no automatic work left. Use settled for a status integration that must not clear itself between runs.

15.5.4 `turn_start`, `turn_end`, and `messages`

Within a run, each turn is one LLM request and assistant response plus the tool calls/results produced by that response. If tools run and the agent continues, the next provider call is another turn in the same run. `turn_end` includes the assistant message and final tool results. Message events give fine-grained lifecycle: user/assistant/tool-result messages emit start/end; assistant streaming also emits update events. `message_end` can replace the finalized message while preserving its role.

Message replacement is powerful. Preserve required fields and do not change a role merely to repurpose a transcript; provider conversion and session logic rely on role semantics.

15.6 Provider events

These hooks are protocol boundaries, not portable message APIs. The payload is `unknown`, its shape varies by adapter, and payload rewrites are not reflected by `ctx.getSystemPrompt()`. Guard payload logic by provider/model, test it against that provider's exact wire protocol, and avoid logging bodies or headers without redacting prompts, credentials, and provider metadata.

15.6.1 `before_provider_headers`

Pi assembles outgoing headers, then gives handlers a mutable map. Assign a string to add/override, or `null` to delete. The hook runs once per provider request; retries reuse the same headers rather than invoking it again.

Use it for a stable request correlation header. Do not use it for a credential refresh protocol that must run anew on every retry unless you explicitly own that behavior.

15.6.2 `before_provider_request`

This sees the adapter-specific payload after Pi has converted context, system prompt, and tools. Return `undefined` to keep it or another value to replace it for later handlers and the actual request. Use it only for a narrow serialization compatibility patch or a sanitized development trace.

15.6.3 `after_provider_response`

This fires after the HTTP response arrives and before its body stream is consumed. It receives normalized status and headers when available. Some provider transports abstract away headers; absence is a provider limitation, not an extension failure.

15.7 Tool events and parallelism

The tool execution path separates observation, policy, and patching:

Event	Role	Handler may
<code>tool_execution_start</code>	Observation: tool is about to run	Read <code>toolCallId</code> , <code>toolName</code> , <code>args</code>
<code>tool_call</code>	Policy gate after validation	Mutate <code>event.input</code> in place, or return <code>{ block: true, reason? }</code>
<code>tool_execution_update</code>	Observation: partial/streaming output	Read <code>partialResult</code>
<code>tool_result</code>	Result-patching middleware	Return partial <code>{ content?, details?, isError? }</code>
<code>tool_execution_end</code>	Observation: finalized execute result	Read <code>result</code> and <code>isError</code>

`tool_execution_*` handlers are observation-only: they do not cancel execution or rewrite output. Do not confuse the extension event `tool_result` with `toolResult` messages. `tool_result` is an extension hook that can patch the result before it is committed (`content`, `details`, and/or `isError`; omitted fields keep their current values). A `toolResult` message is the durable transcript entry (`role`: `"toolResult"`) that appears later as `message_start`/`message_end` and in the session file.

The default parallel execution order is subtle:

```
source-order tool_execution_start and tool_call preflight
-> concurrent execute / tool_execution_update
-> completion-order tool_result then tool_execution_end
-> source-order final toolResult messages (message_start / message_end)
```

Therefore, a `tool_call` handler sees session state synchronized through the assistant message but not necessarily sibling tool results. Use a typed helper for built-ins:

```
import { isToolCallEventType } from "@earendil-works/pi-coding-agent";

pi.on("tool_call", (event) => {
  if (!isToolCallEventType("read", event)) return;
  // event.input.path is typed for the built-in read tool.
});
```

For a custom tool, derive the TypeBox input type with `Static` from the `typebox` package (a direct dependency of `coding-agent`; not re-exported from `@earendil-works/pi-coding-agent`) and supply explicit generic parameters to the guard:

```
import { Type, type Static } from "typebox";
import { isToolCallEventType } from "@earendil-works/pi-coding-agent";

const myToolSchema = Type.Object({ action: Type.String() });
export type MyToolInput = Static<typeof myToolSchema>;

pi.on("tool_call", (event) => {
```

```

    if (isToolCallEventType<"my_tool", MyToolInput>("my_tool", event)) {
        // event.input.action is typed
    }
});

```

Direct `event.toolName === name` narrowing is not sufficient because arbitrary custom names overlap built-in literals.

15.8 Extension context reference

All handlers receive a context with these recurring values:

Field/method	Practical meaning
<code>cwd</code>	Current session working directory
<code>mode / hasUI</code>	Execution surface and interaction availability
<code>ui</code>	Controlled interactive UI adapter
<code>sessionManager</code>	Read-only access to durable tree/context state
<code>modelRegistry / model</code>	Available/current model and authentication view
<code>signal</code>	Active agent signal during turn-related work, otherwise usually undefined
<code>isIdle, abort, hasPendingMessages</code>	Flow-control observations/actions
<code>getContextUsage</code>	Context-window usage/estimate
<code>compact</code>	Trigger compaction with callbacks
<code>getSystemPrompt</code>	Current Pi system-prompt string at this lifecycle point

The scope of `signal` is important. A command handler executed while idle does not receive an active turn signal. A `tool_result` handler usually does (`ctx.signal` is the abort signal for that work). Pass it to nested asynchronous work only when it is defined and relevant.

15.9 Command context reference

Command handlers receive `ExtensionCommandContext`, which adds operations that would be unsafe from arbitrary events:

Method	Safe use
<code>waitForIdle</code>	Finish automatic work before a session mutation
<code>newSession</code>	Create a replacement session, optional setup/handoff
<code>fork</code>	Fork/clone from an entry
<code>navigateTree</code>	Move to a tree point with optional summary
<code>switchSession</code>	Open another session path
<code>reload</code>	Run <code>/reload</code> lifecycle

Method	Safe use
<code>getSystemPromptOptions</code>	Inspect base prompt inputs in a user-triggered command

Replacement methods accept `withSession`. Use only the callback's context for session-bound work. The old extension instance may have already run shutdown cleanup, and captured old session objects will throw or represent old state.

15.10 Registration API quick decisions

API	Use when	Avoid when
<code>registerTool</code>	Model needs a new operation	User-only interaction is enough
<code>registerCommand</code>	User needs an explicit action	Input syntax is a natural transform
<code>registerShortcut</code>	User needs a configurable keyboard action	You can use a command/menu instead
<code>registerFlag</code>	Startup behavior needs a CLI option	State can live in settings
<code>sendMessage</code>	Model needs a custom context message	State should remain private
<code>sendUserMessage</code>	Agent should receive a genuine user message	You only need a silent state update
<code>appendEntry</code>	Durable data should not affect LLM context	You need model-visible instructions
<code>registerProvider</code>	Endpoint/models/auth/streaming change	A header-only trace is enough

15.11 Model-facing output truncation

Custom tool output is not truncated automatically. Apply `truncateHead` to file/search-style output or `truncateTail` to logs and command output. Their shared defaults, from `DEFAULT_MAX_LINES` / `DEFAULT_MAX_BYTES` in `packages/coding-agent/src/core/tools/truncate.ts`, are 2,000 lines and 50 KB (50 * 1024 bytes), whichever is reached first:

```
import { formatSize, truncateHead } from "@earendil-works/pi-coding-agent";

const clipped = truncateHead(output);
let text = clipped.content;
if (clipped.truncated) {
  text += `\n\n[Output truncated: ${clipped.outputLines} of ${clipped.totalLines} lines`;
  text += `; ${formatSize(clipped.outputBytes)} of ${formatSize(clipped.totalBytes)}.`;
  text += " Re-run with a narrower range.";
}
```

A truncation marker must say what was retained and how to retrieve the rest: provide an offset/narrower-query instruction, or save the full output to a temp file when retaining it is safe and include that path. If individual lines can be enormous, pre-truncate them with `truncateLine` (which limits characters) or provide specialized handling; `truncateHead` returns no partial first line when that line alone exceeds its byte limit. Document limits in the tool description.

15.12 Safety rules for extension authors

1. Start background resources in `session_start`, clean them in `session_shutdown`.
2. Treat project-local config as untrusted until `isProjectTrusted()` says otherwise.
3. Forward cancellation signals.
4. Serialize file mutations with the shared queue.
5. Truncate model-facing output explicitly and make continuation actionable.
6. Use `mode/hasUI` guards for user prompts and custom components.
7. Reconstruct persistent state from session entries.
8. Use fresh context after session replacement/reload.
9. Prefer public APIs/types over internal imports.

These rules cover the lifecycle errors that are hardest to discover from a happy-path extension demo.

16 Run modes, SDK embedding, and operational boundaries

The `pi` binary is one host for the coding-agent runtime. The same session construction and event model can also serve print, JSON, RPC, and programmatic SDK use. Understanding the modes prevents terminal assumptions from leaking into automation and helps you select an integration surface deliberately.

Start with `packages/coding-agent/src/main.ts`, `packages/coding-agent/src/modes/`, and `packages/coding-agent/src/core/sdk.ts`. For SDK examples, see `packages/coding-agent/examples/sdk/` and `packages/coding-agent/docs/sdk.md`. Canonical CLI docs for the noninteractive hosts are `packages/coding-agent/docs/json.md` and `packages/coding-agent/docs/rpc.md`.

16.1 Start with the host, not the flag

A **host** is the program that drives Pi. A **mode** describes how that host communicates: a TUI has a terminal, print mode returns text once, JSON emits machine-readable events, and RPC keeps a process open behind a protocol. An **SDK embedding** calls the runtime from another program instead of starting the `pi` binary.

The same agent/session runtime serves all four modes, but UI assumptions do not transfer automatically. First choose the host, then check `ctx.mode` and `ctx.hasUI`, and only then design extension behavior.

16.2 How Pi chooses a mode

`packages/coding-agent/src/main.ts` resolves application mode from explicit arguments and terminal state:

Application mode	Typical trigger	Runtime host/path
Interactive	TTY stdin/stdout without print, JSON, or RPC selection	<code>InteractiveMode</code>
Print	<code>-p</code> or non-TTY stdin/stdout	<code>runPrintMode(..., { mode: "text" })</code>
JSON	<code>--mode json</code>	<code>runPrintMode(..., { mode: "json" })</code>
RPC	<code>--mode rpc</code>	<code>runRpcMode(...)</code>

The distinction is not cosmetic. It changes whether Pi can prompt a user, how output is emitted, and which extension UI methods are meaningful. All modes still rely on the coding-agent session/runtime instead of implementing separate provider/tool loops.

16.3 Extension mode behavior

An extension context exposes `mode` and `hasUI`. `ctx.mode` is a string whose `ExtensionMode` type is the literal union `"tui" | "rpc" | "json" | "print"`. `ctx.hasUI` is not a synonym for “interactive terminal”; it reports whether the extension runner is bound to a real `ExtensionUIContext` rather than the no-op context used when there is no host UI protocol.

Application surface	<code>ctx.mode</code>	<code>ctx.hasUI</code>	Design implication
Interactive terminal	<code>"tui"</code>	true	Full components, dialogs, editor behavior
RPC	<code>"rpc"</code>	true	Protocol-backed dialogs/notices exist; custom terminal UI is limited
JSON	<code>"json"</code>	false	No UI protocol; do not wait for a user prompt or render components
Print	<code>"print"</code>	false	Agent runs, then process exits; UI calls are no-ops/fallbacks

That table is the practical distinction between JSON and RPC for extension authors. JSON and print share the single-run print host and bind a no-op UI, so dialog methods either no-op or resolve without a user. RPC binds a protocol UI so `select/confirm/input/editor` become request/response messages over stdin/stdout, while terminal-only methods such as `custom()` remain unavailable. Guard on both:

- use `ctx.hasUI` before waiting for a user decision;
- use `ctx.mode === "tui"` for terminal components and other TUI-only behavior.

A policy tool that requires confirmation needs an explicit noninteractive decision when `!ctx.hasUI`, such as blocking with a clear reason, rather than awaiting `ctx.ui.confirm()`.

16.4 Print mode as a single-run host

Print mode is appropriate for scripts that want one response rather than a long-lived terminal session. `packages/coding-agent/src/main.ts` prepares initial text/images from arguments, files, and stdin, then passes that input to `runPrintMode` in `packages/coding-agent/src/modes/print-mode.ts` with text output selected. The normal `AgentSession.prompt()` preprocessing remains active, so extensions, context, tools, provider selection, and session policy can still participate. After the prompts finish, the host writes only the text blocks from the final assistant message and exits.

For extension authors, that output rule means a notification or custom UI value is not part of the script's result. Put required information into model-facing content so it can appear in the final response, or persist it through an explicit durable channel. Do not depend on delayed background work or UI state surviving the response.

16.5 JSON mode and machine-readable automation

JSON mode is not a separate session host. `packages/coding-agent/src/main.ts` calls `runPrintMode` with `mode: "json"`. That path writes the session header, when present, followed by subscribed session events as newline-delimited JSON. It therefore provides a single-run event stream rather than print mode's final assistant text or RPC's persistent command channel. See also `packages/coding-agent/docs/json.md`.

16.5.1 Session header on the JSON stream

The first JSONL object is the same session-file header that begins a persisted session, when one is available. In source it is `SessionHeader` from `packages/coding-agent/src/core/session-manager.ts`: metadata immediately written by `runPrintMode` before subscribed events, not a conversation entry.

```
{"type":"session","version":3,"id":"<session-uuid>","timestamp":"<iso-time>","cwd":"/work/project"
```

Fields:

- **type**: always `"session"` for the header;
- **version**: optional session format version (current writes use 3; older v1 headers may omit it, and loaders treat missing `version` as 1);
- **id**: durable session identifier;
- **timestamp**: session creation time;
- **cwd**: working directory recorded for the session;
- **parentSession**: optional source session path when a session is forked.

This header has no `parentId` and is not part of the message tree. It is the metadata line documented in [Sessions, branches, and durable context](#) and `packages/coding-agent/docs/session-format.md`. Subsequent JSON-mode lines are `AgentSessionEvent` objects as the single-run prompt proceeds.

In extension code, avoid `stdout`. Arbitrary `console.log` or other writes can corrupt the JSONL stream consumed by automation. Prefer documented event/UI mechanisms, or emit controlled diagnostics on `stderr` when that is acceptable for the embedding host.

The same caution applies to provider payload tracing. A raw JSON dump may contain private context and may interfere with automation output. Build a sanitized, opt-in diagnostic command instead.

16.6 RPC mode as a persistent process boundary

RPC mode keeps a process alive for command/request exchange. Start it with:

```
pi --mode rpc [options]
```

Implementation and types live under `packages/coding-agent/src/modes/rpc/`, with the host entry in `packages/coding-agent/src/modes/rpc/rpc-mode.ts`, framing in `packages/coding-agent/src/modes/rpc/j` command/response types in `packages/coding-agent/src/modes/rpc/rpc-types.ts`, and a typed client in `packages/coding-agent/src/modes/rpc/rpc-client.ts`. The runtime and session still own model/tool state; RPC is a transport around that behavior, not an alternate agent implementation. The full command and event catalog is `packages/coding-agent/docs/rpc.md`.

16.6.1 Framing and object shapes

- Transport is strict JSONL over stdin/stdout.
- The only record delimiter is LF (`\n`). Clients may strip a trailing `\r` so CRLF-delimited input is accepted, but must not treat Unicode line separators such as `U+2028/U+2029` as record boundaries. Those characters are valid inside JSON strings, so a reader that only splits on LF stays correct; Node `readline` is not protocol-compliant because it also splits on `U+2028/U+2029` and can break a single logical JSON record.
- Commands are JSON objects on stdin with a `type` field and an optional `id` for request/response correlation.
- Command replies are objects with `type`: `"response"`, `command`, `success`, optional `data/error`, and the same `id` when one was supplied.
- Session events are streamed to stdout as plain event objects. They do not carry command `id` values.
- Extension UI dialog traffic uses `type`: `"extension_ui_request"` on stdout and, for dialogs, matching `type`: `"extension_ui_response"` objects on stdin.

16.6.2 Lifecycle in overview

1. The process starts, takes over stdout for protocol traffic, binds the extension/runtime host with `mode`: `"rpc"`, and waits for commands on stdin.
2. A client sends commands such as `prompt`, `steer`, `follow_up`, `abort`, `get_state`, `get_messages`, model/thinking controls, compaction/retry controls, session navigation around entries, or `bash`.
3. For `prompt`, the immediate `response` means the prompt was accepted, queued, or handled; later progress appears as events. For streaming agents, `prompt` requires `streamingBehavior` (`"steer"` or `"followUp"`) when the agent is already running, or the command fails before acceptance.
4. While work proceeds, the host streams session events (`agent_start`, turns/messages/tools, compaction/retry updates, and eventually `agent_settled` when no automatic continuation remains).
5. If an extension uses dialog/fire-and-forget UI methods, the host emits `extension_ui_request`. Dialog methods block until the client answers with the matching `extension_ui_response` or a timeout/default resolves them; fire-and-forget methods do not wait.
6. The process remains available for further commands until the client closes the stream or shutdown is requested.

Extensions can use `ctx.hasUI` in RPC mode for operations that map to protocol UI messages. Dialog methods (`select`, `confirm`, `input`, `editor`) and fire-and-forget methods such as `notify/setStatus` are functional there. Terminal-only methods are not: `ctx.ui.custom()` returns `undefined`, and other TUI-only surface controls are no-ops or degraded. Design commands/tools with a structured noninteractive fallback when they might also run through JSON/print.

In extension code, avoid `stdout`. RPC clients parse every `stdout` line as protocol JSON; incidental writes corrupt the transport. Prefer extension UI methods, session/event channels, or `stderr` for diagnostics.

16.7 The packages/ai boundary

`packages/ai` is the lower-level, provider-neutral layer for model definitions, authentication support, request serialization, and normalized streaming events. It does not own session persistence, extension loading, built-in tools, compaction/retry policy, system-prompt assembly, or any of the run-mode hosts described above.

Choose that package when the work is about model/protocol behavior. Stay in coding-agent modes/SDK when the work is about host policy around those model calls.

16.8 The public SDK entry point

`createAgentSession(options?)` in `packages/coding-agent/src/core/sdk.ts` is the composition entry point for a programmatic host. It can create defaults or accept caller-provided `cwd`, agent directory, model, session manager, settings manager, model registry, resource loader, scoped models, tool restrictions, custom tools, and a session start event.

The SDK does several pieces of application policy on purpose:

```
resolve cwd and agent directory
-> auth/model/settings/session services
-> resource loader (unless caller supplies one)
-> restore/choose model and thinking level
-> create Agent with coding-agent conversion/provider hooks
-> create AgentSession and extension runtime
```

The result gives an `AgentSession`, extension load result, and possible model fallback message. A host can subscribe to the session, call `prompt`, provide custom tools, inspect sessions, or embed its own UI.

16.9 Why SDK callers should not reimplement packages/coding-agent/src/main.ts

It is tempting to instantiate `Agent` directly because its constructor is small. That skips coding-agent semantics including resource loading, session persistence, model restore/default selection, built-in tool

definitions, extension runner integration, system-prompt building, retry, and compaction. Those policies are exactly what `createAgentSession` and the CLI mode hosts assemble on top of the `packages/ai` stream/provider primitives.

Use `Agent` directly only when you deliberately own all of those policies. Use `createAgentSession` when you want Pi's coding-agent behavior in a new host. Use the CLI/RPC surface when a separate process boundary is preferable.

16.10 Custom tools supplied by an SDK host

`createAgentSession` accepts custom tool definitions alongside built-in-tool selection. These tools enter the same registry/extension wrapping flow as an extension-registered tool. Give them a clear source identity, schema, durable details shape, and prompt metadata where appropriate.

An SDK host can use this for application-specific operations without requiring an extension file on disk. The tradeoff is scope: a tool supplied by one host does not automatically appear in a user's ordinary `pi` invocation.

16.11 Initial messages and attachments

The CLI's initial-message machinery can combine explicit prompt text, piped stdin, file arguments, and supported images. Interactive mode feeds the resulting message/images through the same session prompt path used after the editor submits text. This keeps extension `input/before_agent_start` behavior consistent between startup prompts and later user prompts.

When embedding, construct user content as text/image blocks and let the session/agent conversion path handle the provider representation. Do not pre-serialize provider-specific image payloads in a generic SDK host.

16.12 Operational safety boundaries

Pi runs with the operating-system permissions of its process. It does not add a built-in filesystem/process/network permission sandbox. Treat containerization or other OS-level sandboxing as the recommended way to create stronger isolation when automation or untrusted tools require it. That recommendation matches the root `README.md` containerization guidance and `packages/coding-agent/docs/containerization.md`.

This has mode-specific consequences:

- A print/CI invocation can execute the same tools as an interactive one.
- Trust gates project extensions/resources, but do not sandbox global code.
- A custom provider can transmit context to its configured endpoint.
- `bash`, `write`, and extension `exec` run with host process authority.

Use OS/container boundaries for security enforcement. Use extension tool gates for workflow policy and user confirmation. Do not confuse a model instruction or a theme with a security control.

16.13 Operational observability

16.13.1 Behavior provided by Pi

Startup diagnostics are collected during service/session construction and reported by the application layer. Extension load failures are startup errors and include a hint to retry without extensions. Settings and model load issues surface as diagnostics, while the output guard protects protocol output from ordinary writes. Timing instrumentation also exists, including optional startup timing output.

16.13.2 Host-level recommendations

For a production-like integration, capture:

- selected mode and cwd;
- selected model/provider/API type, without credentials;
- session ID/path and persistence state;
- extension/resource load diagnostics;
- agent/turn/tool timing and completion reason;
- provider status metadata when safely available;
- cancellation/retry/compaction decisions.

This is integration guidance rather than a claim that Pi emits every item as a ready-made telemetry record. Choose fields that make incidents diagnosable without collecting full user prompts.

16.14 Choosing an integration approach

Goal	Best starting point
Human coding in a terminal	Interactive CLI
One scripted answer	Print mode
Machine-readable event/result stream	JSON mode
Long-lived external controller	RPC mode
Node application sharing Pi semantics	<code>createAgentSession</code> SDK
New model protocol/provider behavior	Provider extension or <code>packages/ai</code> contribution
Custom execution policy/UI in existing Pi	Extension

Choosing the boundary early is an architectural decision. It determines who owns terminal state, persistence, authentication, user confirmation, and process lifetime.

16.15 Operational checklist

Before putting Pi into automation or another host:

1. Choose the mode/SDK boundary intentionally.
2. Set a trust policy appropriate for noninteractive projects.

3. Select or restrict models/tools explicitly if policy requires it.
4. Store credentials in supported auth/config mechanisms, not prompt text.
5. Ensure output/logging does not corrupt JSON/RPC protocols.
6. Forward cancellation from the host into Pi's abortable work.
7. Decide session persistence and cleanup behavior.
8. Add an OS/container sandbox if untrusted code or high-risk tools are in scope.

These operational decisions are the final layer of the architecture: Pi's internal boundaries are valuable only when the host uses them deliberately. For concrete recovery behavior at these boundaries, continue with [Failure and recovery reference](#); for the provider layer behind every mode, see [Models, providers, and authentication](#).

17 Failure and recovery reference

Pi turns many failures into explicit protocol data instead of allowing an exception to escape through every layer. This operational reference is for extension authors and contributors debugging failures or designing recovery; it is a symptom-to-owner lookup, not a step-by-step tutorial.

17.1 How to use the reference

An **error** means an operation failed. An **abort** means work was stopped by an explicit cancellation request. A **retry** starts another attempt, while **recovery** can also mean returning an actionable tool result or falling back to a safe renderer. Start with the first layer where the symptom appears, then follow the owning layer's recovery action; do not retry before deciding which kind of failure you have.

For example, malformed tool arguments should become a tool error the model can correct, while a missing API key must be fixed in model/auth configuration. Both may look like “the prompt failed,” but they have different owners.

17.2 Failure flow by runtime layer

```
configuration/trust failure
  -> startup diagnostic or ignored project resource

model/provider failure
  -> final assistant message: error or aborted
  -> AgentSession retry/compaction policy may decide next action

tool validation/execution failure
  -> error ToolResultMessage
  -> model receives the error and can choose a recovery action

extension hook failure
  -> runner reports error; tool_call fails safe by blocking

rendering failure
  -> fallback renderer or interactive diagnostic path
```

Each outcome keeps recovery at the layer that can still make a safe decision. Startup rejects or ignores invalid configuration because no reliable runtime exists yet. A provider failure becomes a terminal assistant message because there is no trusted assistant result to continue from, while a tool failure

becomes a `ToolResultMessage` because it is useful input to the model’s next request. An extension hook fails closed where it controls an operation so a broken policy hook cannot accidentally allow that operation. Rendering alone falls back because presentation failure should not discard otherwise valid result data.

17.3 Startup and configuration failures

Symptom	Owning layer	Expected behavior	First recovery action
Malformed <code>settings.json</code> (global <code>~/.pi/agent/settings.json</code> or project <code>.pi/settings.json</code> or <code>models.json</code>)	Settings/model registry	Diagnostic/load error; built-ins remain available where possible	Correct JSON; inspect reported scope/path
Project extension unavailable	Trust/resource loader	Project resources ignored when untrusted	Save/override trust intentionally, then restart/reload
Extension factory fails	Resource loader/application	Startup diagnostics; Pi suggests retry without extensions	Run with <code>--no-extensions/-ne</code> (or load a single path via <code>--extension/-e</code>) to isolate path/import failure
No model selected	Session setup	Prompt is rejected before agent loop	Configure/choose model and auth
Auth missing	Model registry/session	Prompt is rejected with provider guidance	<code>/login</code> , configure key, or select authenticated model

Do not work around a trust failure by making a global extension read an untrusted project’s configuration. That moves a deliberate security boundary into code outside Pi’s trust boundary, on an uncontrolled path.

17.4 Provider failures

Provider adapters report normal failure/abort outcomes as final assistant messages on the stream rather than as raw exceptions. `Agent.runWithLifecycle()` also converts unexpected throws to a final assistant message whose `stopReason` is `error` or `aborted`, followed by turn/agent end events. The transcript and extensions therefore see a normal terminal event sequence even when the network fails.

`AgentSession` evaluates the final assistant message. Depending on settings and error classification, it may prepare an agent-level retry, auto-compact and retry after an overflow, or finish automatic recovery and emit `agent_settled`. Provider SDK retries and agent-level retries are separate knobs;

an excessively long provider retry can delay agent/session-layer retry, compaction, or settle decisions until the provider layer finishes.

For an extension, the correct response to an `agent_end` error is normally to observe/report it, not to immediately create another competing prompt. Use `agent_settled` or a deliberate command action once the built-in policy has finished.

17.5 Tool failures

Tool processing has several failure points:

Point	Result returned to model
Tool name is not active/found	Error result naming the missing tool
<code>prepareArguments</code> or schema validation fails	Error result containing the failure reason
<code>tool_call</code> policy denies/blocks the call	Error result with the deny/block reason
Tool execute throws	Error result with thrown error text
Abort occurs before/during work	Error result indicating cancellation
<code>tool_result</code> hook throws	Error result rather than a malformed patched result

The loop emits a tool-result message after execution and `tool_result` hooks finish (successful or error-path finalization). A model can inspect the result, modify its arguments, call another tool, or explain the failure. Make tool error strings actionable: include an invalid path, missing exact text, exit code, timeout, or next command when safe. Do not put credentials or other secret values in a tool error message; the model and transcript both receive that text.

17.6 Cancellation versus timeout

An abort signal expresses user/application cancellation. A timeout expresses a tool/provider policy deadline. They can lead to similar text in the transcript, but extension code should preserve the distinction:

- Forward abort signals into `fetch`, process execution, and model helpers.
- Clean up partial state deterministically when cancellation happens.
- Use a tool timeout only when the work has a clear bounded deadline.
- Do not convert user cancellation into an automatic retry.

The built-in bash tool kills its process tree for abort and timeout, collects available output, and returns an error with the appropriate status. A custom tool that launches a child process should provide equivalent cleanup rather than leaving a detached process behind.

17.7 Recovery after compaction and tree movement

Compaction/retry is recovery from context pressure, not from every provider error. `firstKeptEntryId` is the ID of the earliest session entry retained verbatim after compaction; entries before it are represented by the summary. A custom compaction extension that intercepts `session_before_compact` must return a `SessionBeforeCompactResult`. To replace the summary, set `compaction` to a `CompactionResult` shaped as `{ summary, firstKeptEntryId, tokensBefore, details? }`. To abort compaction, return `{ cancel: true }`. To keep the default summary from `prepareCompaction/compact`, omit `compaction`. Changing or omitting the `firstKeptEntryId` boundary can make context rebuilding retain the wrong entries or omit the intended ones.

Tree navigation changes the active leaf. An extension that reconstructs state from `getEntries()` instead of the active branch may accidentally recover state from an abandoned path. This can look like a failure after `/tree` even though session persistence behaved correctly.

17.8 UI and renderer fallback

Renderers are presentation code. If a custom tool/message renderer throws, Pi falls back to basic rendering for that slot where supported. Treat a fallback as a diagnostic, not evidence that result data is correct. Inspect the original content/details and renderer context, then make the renderer handle partial, error, and expanded states explicitly.

For a TUI-only issue, replay the same fixture at a narrow width (for example, 40 columns) and a normal width, then test both collapsed and expanded tool output. Narrow width forces wrapping and clipping paths, while toggling output exercises both renderer states. A failure that appears only when ANSI-styled text wraps usually points to a display-column calculation (escape sequences occupy bytes but no visible columns), not to agent or session data.

17.9 Incident checklist

For a reliable bug report or regression test, record:

1. Mode, cwd, trust state, session ID/path, and selected model.
2. The input/event that triggered the problem.
3. Whether it was idle, streaming, compacting, or replacing a session.
4. The relevant final stop reason/tool result error.
5. Sanitized provider status/headers if applicable.
6. Active tools and extension/resource provenance.
7. Whether Esc/reload/new/resume changes the behavior.
8. A focused test, session tree sketch, or terminal capture proving the result.

This checklist establishes a factual boundary before a fix is proposed. For rendering incidents, pair it with [Tour 5: Follow one streamed token to the terminal](#). For compaction or branch-context incidents, use [Tour 9: Trace a resume-context discrepancy](#).

18 Glossary

Agent The stateful class in `packages/agent` that owns a model, system prompt, messages, active tools, queues, and one active run. A *run* is one continuous loop cycle from a submitted prompt through assistant responses and tool calls until **Steering** and **Follow-up** queues have nothing left to deliver and the agent goes idle.

AgentSession The wrapper in `packages/coding-agent` around an **Agent**. It preprocesses prompts, coordinates resources, configures tools and the system prompt, bridges extension events, handles retries and **Compaction**, and records events through its **SessionManager**. An **AgentSessionRuntime** owns the live instance.

AgentSessionRuntime The lifecycle owner of the current **AgentSession** and the cwd-bound service bundle that backs it: settings, **Resource loader**, model registry, and authentication. On new, resume, or fork it shuts down the old session and services for the previous cwd, creates a matching bundle for the destination, and rebinds hosts to the replacement; it does not mutate an existing session into a new identity. See **SessionManager**.

Assistant message event stream Async stream from `packages/ai` that normalizes provider streaming events into start, deltas, completion, and error states.

Compaction Summarizing older context into a durable session entry so recent context can fit in a model window. It is not simple transcript deletion.

Context The provider-facing bundle of system prompt, converted messages, and active tools. It is distinct from the broader persisted session held by a **SessionManager**.

Extension A runtime-loaded TypeScript/JavaScript factory module that receives **ExtensionAPI** and registers behavior. Loaded by the **Resource loader** under the resolved **Trust policy**.

Extension context The `ctx` passed to hooks, tools, and commands, containing mode, cwd, UI, session access, and the model registry. Its optional `signal` is an **AbortSignal** during active turn work so nested I/O can cancel with the agent; it is usually `undefined` while idle.

Follow-up A queued user or custom message delivered only when the current run has no more tool calls or **Steering** messages and would otherwise stop. Use it when the current run should finish first; use **Steering** when new guidance should affect the ongoing run at its next turn boundary.

JSONL JSON Lines: a text format with one complete JSON value per line. Pi uses it both for append-only session files and to frame JSON-mode and RPC streams.

Model registry Service in `packages/coding-agent` that knows models, providers, authentication, OAuth, and dynamic provider registration.

Resource loader Service in `packages/coding-agent` that discovers and loads extensions, skills, prompt templates, themes, context files, and packages according to the resolved **Trust policy** and each resource category's rules. Its results feed **AgentSession** construction, including

system-prompt inputs; the usual implementation is reloaded with the rest of the cwd-bound bundle when an **AgentSessionRuntime** replaces a session.

SessionManager The persistence component used by an **AgentSession**. It stores the session as an append-only **JSONL** tree, tracks the selected leaf and branches, and reconstructs the active model **Context**. An **AgentSessionRuntime** replaces it as part of replacing the current session.

Steering A queued message injected after the current assistant response and all tool calls from that response finish, but before the next provider request. Use it when new guidance should affect the ongoing run at its next turn boundary; use a **Follow-up** when the current run should finish first.

Tool A model-callable operation with a TypeBox parameter schema and async execute function. Built-in and extension tools share the agent-loop boundary.

Tool result A durable message produced by a tool execution. Its content goes to the model; details carry structured metadata for renderers/state.

Trust policy The startup decision that determines whether project-local configuration and resources may be loaded or executed, including settings, packages, extensions, skills, prompts, and themes. The **Resource loader** applies that decision when discovering project resources. Context files such as **AGENTS.md** are a documented exception and load unless context loading is disabled.

TUI Terminal user interface. Pi's reusable TUI primitives live in `packages/tui`; `packages/coding-agent` composes them into the interactive application.